

UNIT-I**INTRODUCTION- THE HUMAN AND COMPUTER**

The Human: Introduction- Input /Output Channels- Human Memory- Thinking: Reasoning and problem solving- Individual differences- Psychology and the design of interactive systems- The Computer: Introduction Text entry devices- Positioning- Pointing and drawing – Paper: Printing and Scanning- Memory- Processing and networks.

The Human: Introduction

Humans are limited in their capacity to process information. This has important implications for design.

Information is received and responses given via a number of input and output channels:

- visual channel
- auditory channel
- haptic channel
- movement.

Information is stored in memory:

- sensory memory
- short-term (working) memory
- long-term memory.

Information is processed and applied:

- reasoning
- problem solving
- skill acquisition
- error.

Emotion influences human capabilities.

Users share common capabilities but are individuals with differences, which should not be ignored.

1.1 INTRODUCTION

This chapter is the first of four in which we introduce some of the ‘foundations’ of HCI. We start with the human, the central character in any discussion of interactive systems. The human, the user, is, after all, the one whom computer systems are designed to assist. The requirements of the user should therefore be our first priority.

In this chapter we will look at areas of human psychology coming under the general banner of cognitive psychology. This may seem a far cry from designing and building interactive computer systems, but it is not. In order to design something for someone, we need to understand their capabilities and limitations. We need to know if there are things that they will find difficult or, even, impossible. It will also help us to know what people find easy and how we can help them by encouraging these things. We will look at aspects of cognitive psychology which have a bearing on the use of computer systems: how humans perceive the world around them, how they store and process information and solve problems, and how they physically manipulate objects. We have already said that we will restrict our study to those things that are relevant to HCI. One way to structure this discussion is to think of the user in a way that highlights these aspects. In other words, to think of a simplified model of what is actually going on. Many models have been proposed and it useful to consider one of the most influential in passing, to understand the context of the discussion that is to follow. In 1983, Card, Moran and Newell described the Model Human Processor, which is a simplified view of the human processing involved in interacting with computer systems. The model comprises three subsystems: the perceptual system, handling sensory stimulus from the outside world, the motor system, which controls actions, and the cognitive system, which provides the processing needed to connect the two. Each of these subsystems has its own processor and memory, although obviously the complexity of these varies depending on the complexity of the tasks the subsystem has to perform. The model also includes a number of principles of operation which dictate the behavior of the systems under certain conditions.

We will use the analogy of the user as an information processing system, but in our model make the analogy closer to that of a conventional computer system. Information comes in, is stored and processed, and information is passed out. We will therefore discuss three components of this system: input–output, memory and processing. In the human, we are dealing with an intelligent information-processing system, and processing therefore includes problem solving, learning, and,

consequently, making mistakes. This model is obviously a simplification of the real situation, since memory and processing are required at all levels, as we have seen in the Model Human Processor. However, it is convenient as a way of grasping how information is handled by the human system. The human, unlike the computer, is also influenced by external factors such as the social and organizational environment, and we need to be aware of these influences as well. We will ignore such factors for now and concentrate on the human's information processing capabilities only. In this chapter, we will first look at the human's input-output channels, the senses and responders or effectors. This will involve some low-level processing. Secondly, we will consider human memory and how it works. We will then think about how humans perform complex problem solving, how they learn and acquire skills, and why they make mistakes. Finally, we will discuss how these things can help us in the design of computer systems.

1.2 INPUT/OUTPUT CHANNELS

A person's interaction with the outside world occurs through information being received and sent: input and output. In an interaction with a computer the user receives information that is output by the computer, and responds by providing input to the computer – the user's output becomes the computer's input and vice versa. Consequently, the use of the terms input and output may lead to confusion so we shall blur the distinction somewhat and concentrate on the channels involved. This blurring is appropriate since, although a particular channel may have a primary role as input or output in the interaction, it is more than likely that it is also used in the other role. For example, sight may be used primarily in receiving information from the computer, but it can also be used to provide information to the computer, for example by fixating on a particular screen point when using an eye gaze system.

Input in the human occurs mainly through the senses and output through the motor control of the effectors. There are **five major senses: sight, hearing, touch, taste and smell**. Of these, the first three are the most important to HCI. Taste and smell do not currently play a significant role in HCI, and it is not clear whether they could be exploited at all in general computer systems, although they could have a role to play in more specialized systems (smells to give warning of malfunction, for example) or in augmented reality systems. However, vision, hearing and touch are central.

Similarly, there are a number of effectors, including the limbs, fingers, eyes, head and vocal system. In the interaction with the computer, the fingers play the primary role, through typing or

mouse control, with some use of voice, and eye, head and body position.

Imagine using a personal computer (PC) with a mouse and a keyboard. The application you are using has a graphical interface, with menus, icons and windows. In your interaction with this system, you receive information primarily by sight, from what appears on the screen. However, you may also receive information by ear: for example, the computer may ‘beep’ at you if you make a mistake or to draw attention to something, or there may be a voice commentary in a multimedia presentation. Touch plays a part too in that you will feel the keys moving (also hearing the ‘click’) or the orientation of the mouse, which provides vital feedback about what you have done. You yourself send information to the computer using your hands, either by hitting keys or moving the mouse. Sight and hearing do not play a direct role in sending information in this example, although they may be used to receive information from a third source (for example, a book, or the words of another person) which is then transmitted to the computer.

In this section we will look at the main elements of such an interaction, first considering the role and limitations of the three primary senses and going on to consider motor control.

Vision

Human vision is a highly complex activity with a range of physical and perceptual limitations, yet it is the primary source of information for the average person. We can roughly divide visual perception into two stages: the physical reception of the stimulus from the outside world, and the processing and interpretation of that stimulus. On the one hand the physical properties of the eye and the visual system mean that there are certain things that cannot be seen by the human; on the other the interpretative capabilities of visual processing allow images to be constructed from incomplete information. We need to understand both stages as both influence what can and cannot be perceived visually by a human being, which in turn directly affects the way that we design computer systems. We will begin by looking at the eye as a physical receptor, and then go on to consider the processing involved in basic vision.

The human eye

Vision begins with light. The eye is a mechanism for receiving light and transforming it into electrical energy. Light is reflected from objects in the world and their image is focused upside down on the back of the eye. The receptors in the eye transform it into electrical signals which are passed to the brain.

to respond. Each of these stages takes time, which can be roughly divided into reaction time and movement time.

Movement time is dependent largely on the physical characteristics of the subjects: their age and fitness, for example. Reaction time varies according to the sensory channel through which the stimulus is received. A person can react to an auditory signal in approximately 150 ms, to a visual signal in 200 ms and to pain in 700 ms. However, a combined signal will result in the quickest response. Factors such as skill or practice can reduce reaction time, and fatigue can increase it.

A second measure of motor skill is accuracy. One question that we should ask is whether speed of reaction results in reduced accuracy. This is dependent on the task and the user. In some cases, requiring increased reaction time reduces accuracy. This is the premise behind many arcade and video games where less skilled users fail at levels of play that require faster responses. However, for skilled operators this is not necessarily the case. Studies of keyboard operators have shown that, although the faster operators were up to twice as fast as the others, the slower ones made 10 times the errors.

Speed and accuracy of movement are important considerations in the design of interactive systems, primarily in terms of the time taken to move to a particular target on a screen. The target may be a button, a menu item or an icon, for example. The time taken to hit a target is a function of the size of the target and the distance that has to be moved. This is formalized in Fitts' law [135]. There are many variations of this formula, which have varying constants, but they are all very similar. One common form is

$$\text{Movement time} = a + b \log_2(\text{distance/size} + 1)$$

where a and b are empirically determined constants.

This affects the type of target we design. Since users will find it more difficult to manipulate small objects, targets should generally be as large as possible and the distance to be moved as small as possible. This has led to suggestions that pie-chart-shaped menus are preferable to lists since all options are equidistant. However, the trade-off is increased use of screen estate, so the choice may not be so simple. If lists are used, the most frequently used options can be placed closest to the user's start point (for example, at the top of the menu).

1.3 HUMAN MEMORY

Indeed, much of our everyday activity relies on memory. As well as storing all our factual knowledge, our memory contains our knowledge of actions or procedures.

It allows us to repeat actions, to use language, and to use new information received via our senses. It also gives us our sense of identity, by preserving information from our past experiences.

But how does our memory work? How do we remember arbitrary lists such as those generated in the memory game? Why do some people remember more easily than others? And what happens when we forget?

In order to answer questions such as these, we need to understand some of the capabilities and limitations of human memory. Memory is the second part of our model of the human as an information-processing system. However, as we noted earlier, such a division is simplistic since, as we shall see, memory is associated with each level of processing. Bearing this in mind, we will consider the way in which memory is structured and the activities that take place within the system.

It is generally agreed that there are three types of memory or memory function: sensory buffers, short-term memory or working memory, and long-term memory. There is some disagreement as to whether these are three separate systems or different functions of the same system. We will not concern ourselves here with the details of this debate, which is discussed in detail by Baddeley [21], but will indicate the evidence used by both sides as we go along. For our purposes, it is sufficient to note three separate types of memory. These memories interact, with information being processed and passed between memory stores, as shown in Figure 1.3

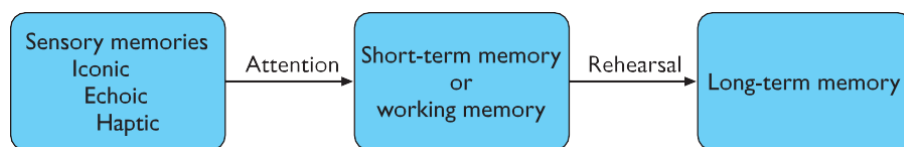


Figure 1.3 Model of the Structure of memory

1.3.1 Sensory memory

The sensory memories act as buffers for stimuli received through the senses. A sensory memory exists for each sensory channel: iconic memory for visual stimuli, echoic memory for aural stimuli and haptic memory for touch. These memories are constantly overwritten by new information coming in on these channels.

We can demonstrate the existence of iconic memory by moving a finger in front of the eye. Can you see it in more than one place at once? This indicates a persistence of the image after the stimulus has been removed. A similar effect is noticed most vividly at firework displays where moving sparklers leave a persistent image. Information remains in iconic memory very briefly, in the order of 0.5 seconds.

Similarly, the existence of echoic memory is evidenced by our ability to ascertain the direction from which a sound originates. This is due to information being received by both ears. However, since this information is received at different times, we must store the stimulus in the meantime. Echoic memory allows brief 'play-back' of information. Have you ever had someone ask you a question when you are reading? You ask them to repeat the question, only to realize that you know what was asked after all. This experience, too, is evidence of the existence of echoic memory.

Information is passed from sensory memory into short-term memory by attention, thereby filtering the stimuli to only those which are of interest at a given time. Attention is the concentration of the mind on one out of a number of competing stimuli or thoughts. It is clear that we are able to focus our attention selectively, choosing to attend to one thing rather than another. This is due to the limited capacity of our sensory and mental processes. If we did not selectively attend to the stimuli coming into our senses, we would be overloaded. We can choose which stimuli to attend to, and this choice is governed to an extent by our arousal, our level of interest or need. This explains the cocktail party phenomenon mentioned earlier: we can attend to one conversation over the background noise, but we may choose to switch our attention to a conversation across the room if we hear our name mentioned. Information received by sensory memories is quickly passed into a more permanent memory store, or overwritten and lost.

1.3.2. Short-term memory

Short-term memory or working memory acts as a 'scratch-pad' for temporary recall of information. It is used to store information which is only required fleetingly. For example, calculate the multiplication 35×6 in your head. The chances are that you will have done this calculation in stages, perhaps 5×6 and then 30×6 and added the results; or you may have used the fact that $6 = 2 \times 3$ and calculated $2 \times 35 = 70$ followed by 3×70 . To perform calculations such as this we need to store the intermediate stages for use later. Or consider reading. In order to comprehend this sentence, you need to hold in your mind the beginning of the sentence as you read the rest. Both of these tasks use short-term memory.

Short-term memory can be accessed rapidly, in the order of 70 ms. However, it also decays rapidly, meaning that information can only be held there temporarily, in the order of 200 ms.

Short-term memory also has a limited capacity. There are two basic methods for measuring memory capacity. The first involves determining the length of a sequence which can be remembered in order. The second allows items to be freely recalled in any order. Using the first

Long-term memory processes

So much for the structure of memory, but what about the processes which it uses? There are three main activities related to long-term memory: storage or remembering of information, forgetting and information retrieval. We shall consider each of these in turn.

First, how does information get into long-term memory and how can we improve this process? Information from short-term memory is stored in long-term memory by rehearsal. The repeated exposure to a stimulus or the rehearsal of a piece of information transfers it into long-term memory.

This process can be optimized in a number of ways. Ebbinghaus performed numerous experiments on memory, using himself as a subject. In these experiments he tested his ability to learn and repeat nonsense syllables, comparing his recall minutes, hours and days after the learning process. He discovered that the amount learned was directly proportional to the amount of time spent learning. This is known as the total time hypothesis. However, experiments by Baddeley and others suggest that learning time is most effective if it is distributed over time. For example, in an experiment in which Post Office workers were taught to type, those whose training period was divided into weekly sessions of one hour performed better than those who spent two or four hours a week learning (although the former obviously took more weeks to complete their training). This is known as the distribution of practice effect.

However, repetition is not enough to learn information well. If information is not meaningful it is more difficult to remember. This is illustrated by the fact that it is more difficult to remember a set of words representing concepts than a set of words representing objects. Try it. First try to remember the words in list A and test yourself.

List A: Faith Age Cold Tenet Quiet Logic Idea Value Past Large

Now try list B.

List B: Boat Tree Cat Child Rug Plate Church Gun Flame Head

The second list was probably easier to remember than the first since you could visualize the objects in the second list.

Sentences are easier still to memorize. Bartlett performed experiments on remembering meaningful information (as opposed to meaningless such as Ebbinghaus used). In one such experiment he got subjects to learn a story about an unfamiliar culture and then retell it. He found that subjects would retell the story replacing unfamiliar words and concepts with words which

were meaningful to them. Stories were effectively translated into the subject's own culture. This is related to the semantic structuring of long-term memory: if information is meaningful and familiar, it can be related to existing structures and more easily incorporated into memory.

1.4. THINKING: REASONING AND PROBLEM SOLVING

1.4.1 Reasoning

Reasoning is the process by which we use the knowledge we have to draw conclusions or infer something new about the domain of interest. There are a number of different types of reasoning: deductive, inductive and abductive. We use each of these types of reasoning in everyday life, but they differ in significant ways.

Deductive reasoning

Deductive reasoning derives the logically necessary conclusion from the given pre-mises. For example,

If it is Friday then she will go to work It is Friday

Therefore, she will go to work.

It is important to note that this is the logical conclusion from the premises; it does not necessarily have to correspond to our notion of truth. So, for example,

If it is raining then the ground is dry It is raining

Therefore, the ground is dry.

is a perfectly valid deduction, even though it conflicts with our knowledge of what is true in the world.

Deductive reasoning is therefore often misapplied. Given the premises

Some people are babies Some babies' cry

many people will infer that 'Some people cry'. This is in fact an invalid deduction since we are not told that all babies are people. It is therefore logically possible that the babies who cry are those who are not people.

It is at this point, where truth and validity clash, that human deduction is poorest. One explanation for this is that people bring their world knowledge into the reasoning process. There is good reason

for this. It allows us to take short cuts which make dialog and interaction between people informative but efficient. We assume a certain amount of shared knowledge in our dealings with each other, which in turn allows us to interpret the inferences and deductions implied by others. If validity rather than truth was preferred, all premises would have to be made explicit.

Inductive reasoning

Induction is generalizing from cases we have seen to infer information about cases we have not seen. For example, if every elephant we have ever seen has a trunk, we infer that all elephants have trunks. Of course, this inference is unreliable and cannot be proved to be true; it can only be proved to be false. We can disprove the inference simply by producing an elephant without a trunk. However, we can never prove it true because, no matter how many elephants with trunks we have seen or are known to exist, the next one we see may be trunkless. The best that we can do is gather evidence to support our inductive inference.

In spite of its unreliability, induction is a useful process, which we use constantly in learning about our environment. We can never see all the elephants that have ever lived or will ever live, but we have certain knowledge about elephants which we are prepared to trust for all practical purposes, which has largely been inferred by induction. Even if we saw an elephant without a trunk, we would be unlikely to move from our position that 'All elephants have trunks', since we are better at using positive than negative evidence.

Abductive reasoning

The third type of reasoning is abduction. Abduction reasons from a fact to the action or state that caused it. This is the method we use to derive explanations for the events we observe. For example, suppose we know that Sam always drives too fast when she has been drinking. If we see Sam driving too fast, we may infer that she has been drinking. Of course, this too is unreliable since there may be another reason why she is driving fast: she may have been called to an emergency, for example.

In spite of its unreliability, it is clear that people do infer explanations in this way, and hold onto them until they have evidence to support an alternative theory or explanation. This can lead to problems in using interactive systems. If an event always follows an action, the user will infer that the event is caused by the action unless evidence to the contrary is made available. If, in fact, the event and the action are unrelated, confusion and even error often result.

Errors and mental models

Human capability for interpreting and manipulating information is quite impressive. However, we do make mistakes. Some are trivial, resulting in no more than temporary inconvenience or annoyance. Others may be more serious, requiring substantial effort to correct. Occasionally an error may have catastrophic effects, as we see when ‘human error’ results in a plane crash or nuclear plant leak.

Why do we make mistakes and can we avoid them? In order to answer the latter part of the question we must first look at what is going on when we make an error. There are several different types of error. As we saw in the last section some errors result from changes in the context of skilled behavior. If a pattern of behavior has become automatic and we change some aspect of it, the more familiar pattern may break through and cause an error. A familiar example of this is where we intend to stop at the shop on the way home from work but in fact drive past. Here, the activity of driving home is the more familiar and overrides the less familiar intention.

Other errors result from an incorrect understanding, or model, of a situation or system. People build their own theories to understand the causal behavior of systems. These have been termed mental models. They have a number of characteristics. Mental models are often partial: the person does not have a full understanding of the working of the whole system. They are unstable and are subject to change. They can be internally inconsistent, since the person may not have worked through the logical consequences of their beliefs. They are often unscientific and may be based on superstition rather than evidence. Often, they are based on an incorrect interpretation of the evidence.

Emotion

So far in this chapter we have concentrated on human perceptual and cognitive abilities. But human experience is far more complex than this. Our emotional response to situations affects how we perform. For example, positive emotions enable us to think more creatively, to solve complex problems, whereas negative emotion pushes us into narrow, focused thinking. A problem that may be easy to solve when we are relaxed, will become difficult if we are frustrated or afraid.

Psychologists have studied emotional response for decades and there are many theories as to what is happening when we feel an emotion and why such a response occurs. More than a century ago, William James proposed what has become known as the James–Lange theory (Lange was a

contemporary of James whose theories were similar): that emotion was the interpretation of a physiological response, rather than the other way around. So while we may feel that we respond to an emotion, James contended that we respond physiologically to a stimulus and interpret that as emotion:

Common sense says, we lose our fortune, are sorry and weep; we meet a bear, are frightened and run; we are insulted by a rival, are angry and strike. The hypothesis here... is that we feel sorry because we cry, angry because we strike, afraid because we tremble.

Others, however, disagree. Cannon, for example, argued that our physiological processes are in fact too slow to account for our emotional reactions, and that the physiological responses for some emotional states are too similar (e.g. anger and fear), yet they can be easily distinguished. Experience in studies with the use of drugs that stimulate broadly the same physiological responses as anger or fear seems to support this: participants reported physical symptoms but not the emotion, which suggests that emotional response is more than a recognition of physiological changes.

1.5 INDIVIDUAL DIFFERENCES

We have made the assumption that everyone has similar capabilities and limitations and that we can therefore make generalizations. To an extent this is true: the psychological principles and properties that we have discussed apply to the majority of people. Notwithstanding this, we should remember that, although we share processes in common, humans, and therefore users, are not all the same. We should be aware of individual differences so that we can account for them as far as possible within our designs. These differences may be long term, such as sex, physical capabilities and intellectual capabilities. Others are shorter term and include the effect of stress or fatigue on the user. Still others change through time, such as age.

These differences should be taken into account in our designs. It is useful to consider, for any design decision, if there are likely to be users within the target group who will be adversely affected by our decision. At the extremes a decision may exclude a section of the user population. For example, the current emphasis on visual interfaces excludes those who are visually impaired, unless the design also makes use of the other sensory channels. On a more mundane level, designs should allow for users who are under pressure, feeling ill or distracted by other concerns: they should not push users to their perceptual or cognitive limits.

1.6 PSYCHOLOGY AND THE DESIGN OF INTERACTIVE SYSTEMS

So far, we have looked briefly at the way in which humans receive, process and store information, solve problems and acquire skill. But how can we apply what we have learned to designing interactive systems? Sometimes, straightforward conclusions can be drawn. For example, we can deduce that recognition is easier than recall and allow users to select commands from a set (such as a menu) rather than input them directly. However, in the majority of cases, application is not so obvious or simple. In fact, it may be dangerous, leading us to make generalizations which are not valid. In order to apply a psychological principle or result properly in design, we need to understand its context, both in terms of where it fits in the wider field of psychology and in terms of the details of the actual experiments, the measures used and the subjects involved, for example. This may appear daunting, particularly to the novice designer who wants to acknowledge the relevance of cognitive psychology but does not have the background to derive appropriate conclusions. Fortunately, principles and results from research in psychology have been distilled into guidelines for design, models to support design and techniques for evaluating design.

1.6.1 Guidelines

Throughout this chapter we have discussed the strengths and weaknesses of human cognitive and perceptual processes but, for the most part, we have avoided attempting to apply these directly to design. This is because such an attempt could only be partial and simplistic, and may give the impression that this is all psychology has to offer.

However, general design principles and guidelines can be and have been derived from the theories we have discussed. Some of these are relatively straightforward: for instance, recall is assisted by the provision of retrieval cues so interfaces should incorporate recognizable cues wherever possible. Others are more complex and context dependent.

1.6.2 Models to support design

As well as guidelines and principles, psychological theory has led to the development of analytic and predictive models of user behavior. Some of these include a specific model of human problem solving, others of physical activity, and others attempt a more comprehensive view of cognition. Some predict how a typical computer user would behave in a given situation, others analyze why particular user behavior occurred. All are based on cognitive theory.

1.6.3 Techniques for evaluation

In addition to providing us with a wealth of theoretical understanding of the human user, psychology also provides a range of empirical techniques which we can employ to evaluate our designs and our systems. In order to use these effectively we need to understand the scope and benefits of each method.

1.7.THE COMPUTER

In order to understand how humans interact with computers, we need to have an understanding of both parties in the interaction. The previous chapter explored aspects of human capabilities and behavior of which we need to be aware in the context of human–computer interaction; this chapter considers the computer and associated input–output devices and investigates how the technology influences the nature of the interaction and style of the interface.

We will concentrate principally on the traditional computer but we will also look at devices that take us beyond the closed world of keyboard, mouse and screen. As well as giving us lessons about more traditional systems, these are increasingly becoming important application areas in HCI. When we interact with computers, what are we trying to achieve? Consider what happens when we interact with each other – we are either passing information to other people, or receiving information from them. Often, the information we receive is in response to the information that we have recently imparted to them, and we may then respond to that. Interaction is therefore a process of information transfer. Relating this to the electronic computer, the same principles hold: interaction is a process of information transfer, from the user to the computer and from the computer to the user.

1.7.1 TEXT ENTRY DEVICES

Whether writing a book like this, producing an office memo, sending a thank you letter after your birthday, or simply sending an email to a friend, entering text is one of our main activities when using the computer. The most obvious means of text entry is the plain keyboard, but there are several variations on this: different keyboard layouts, ‘chord’ keyboards that use combinations of fingers to enter letters, and phone key pads. Handwriting and speech recognition offer more radical alternatives.

1.7.1.1 The alphanumeric keyboard

The keyboard is still one of the most common input devices in use today. It is used for entering textual data and commands. The vast majority of keyboards have a standardized layout, and are known by the first six letters of the top row of alphabetical keys, QWERTY. There are alternative designs which have some advantages over the QWERTY layout, but these have not been able to overcome the vast technological inertia of the QWERTY keyboard. These alternatives are of two forms: 26 key layouts and chord keyboards. A 26 key layout rearranges the order of the alphabetic keys, putting the most commonly used letters under the strongest fingers, or adopting simpler practices. In addition to QWERTY, we will discuss two 26 key layouts, alphabetic and DVORAK, and chord keyboards.

The QWERTY keyboard

The layout of the digits and letters on a QWERTY keyboard is fixed, but non-alphanumeric keys vary between keyboards. For example, there is a difference between key assignments on British and American keyboards. The standard layout is also subject to variation in the placement of brackets, backslashes and suchlike. In addition, different national keyboards include accented letters and the traditional French layout places the main letters in different locations – the top line starts AZERTY.

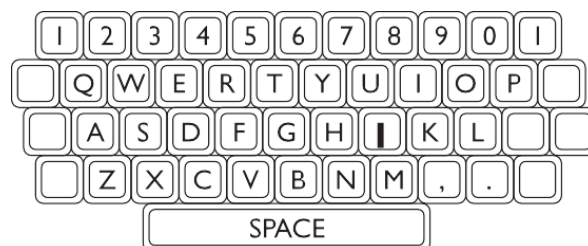


Figure 1.6 The standard QWERTY keyboard

Ease of learning – alphabetic keyboard

One of the most obvious layouts to be produced is the alphabetic keyboard, in which the letters are arranged alphabetically across the keyboard. It might be expected that such a layout would make it quicker for untrained typists to use, but this is not the case. Studies have shown that this keyboard is not faster for properly trained typists, as we may expect, since there is no inherent advantage to this layout. And even for novice or occasional users, the alphabetic layout appears to make very little difference to the speed of typing. These keyboards are used in some pocket electronic personal organizers, perhaps because the layout looks simpler to use than the QWERTY one. Also, it

Most phones have at least two modes for the numeric buttons: one where the keys mean the digits (for example when entering a phone number) and one where they mean letters (for example when typing an SMS message). Some have additional modes to make entering accented characters easier. Also, a special mode or setting is needed for capital letters although many phones use rules to reduce this, for example automatically capitalizing the initial letter in a message and letters following full stops, question marks and exclamation marks.



Figure 1.8. Mobile phone keypad

1.7.1.4 Handwriting recognition

Handwriting is a common and familiar activity, and is therefore attractive as a method of text entry. If we were able to write as we would when we use paper, but with the computer taking this form of input and converting it to text, we can see that it is an intuitive and simple way of interacting with the computer. However, there are a number of disadvantages with handwriting recognition. Current technology is still fairly inaccurate and so makes a significant number of mistakes in recognizing letters, though it has improved rapidly. Moreover, individual differences in handwriting are enormous, and make the recognition process even more difficult. The most significant information in handwriting is not in the letter shape itself but in the stroke information – the way in which the letter is drawn. This means that devices which support handwriting recognition must capture the stroke information, not just the final character shape. Because of this, online recognition is far easier than reading handwritten text on paper. Further complications arise because letters within words are shaped and often drawn very differently depending on the actual word; the context can help determine the letter's identity, but is often unable to provide enough information.

1.7.1.5 Speech recognition

Speech recognition is a promising area of text entry, but it has been promising for a number of years and is still only used in very limited situations. There is a natural enthusiasm for being able to talk to the machine and have it responded to commands, since this form of interaction is one with which we are very familiar. Successful recognition rates of over 97% have been reported, but since this represents one letter in error in approximately every 30, or one spelling mistake every six or so words, this is still unacceptable (sic)! Note also that this performance is usually quoted only for a restricted vocabulary of command words. Trying to extend such systems to the level of understanding natural language, with its inherent vagueness, imprecision and pauses, opens up many more problems that have not been satisfactorily solved even for keyboard-entered natural language. Moreover, since every person speaks differently, the system has to be trained and tuned to each new speaker, or its performance decreases. Strong accents, a cold or emotion can also cause recognition problems, as can background noise. This leads us on to the question of practicality within an office environment: not only may the background level of noise cause errors, but if everyone in an open-plan office were to talk to their machine, the level of noise would dramatically increase, with associated difficulties. Confidentiality would also be harder to maintain.

1.8. POSITIONING, POINTING AND DRAWING

Central to most modern computing systems is the ability to point at something on the screen and thereby manipulate it, or perform some function. There has been a long history of such devices, in particular in computer-aided design (CAD), where positioning and drawing are the major activities. Pointing devices allow the user to point, position and select items, either directly or by manipulating a pointer on the screen. Many pointing devices can also be used for free-hand drawing although the skill of drawing with a mouse is very different from using a pencil. The mouse is still most common for desktop computers, but is facing challenges as laptop and hand-held computing increase their market share. Indeed, these words are being typed on a laptop with a touchpad and no mouse.

1.8.1. The mouse

The Mouse has become a major component of the majority of desktop computer systems sold today, and is the little box with the tail connecting it to the machine in our basic computer system picture. It is a small, palm-sized box housing a weighted ball – as the box is moved over the tabletop, the ball is rolled by the table and so rotates inside the housing. This rotation is detected

by small rollers that are in contact with the ball, and these adjust the values of potentiometers. If you remove the ball occasionally to clear dust you may be able to see these rollers. The changing values of these potentiometers can be directly related to changes in position of the ball. The potentiometers are aligned in different directions so that they can detect both horizontal and vertical motion. The relative motion information is passed to the computer via a wire attached to the box, or in some cases using wireless or infrared, and moves a pointer on the screen, called the cursor. The whole arrangement tends to look rodent-like, with the box acting as the body and the wire as the tail; hence the term ‘mouse’. In addition to detecting motion, the mouse has typically one, two or three buttons on top. These are used to indicate selection or to initiate action. Single-button mice tend to have similar functionality to multi-button mice, and achieve this by instituting different operations for a single and a double button click. A ‘double-click’ is when the button is pressed twice in rapid succession. Multi-button mice tend to allocate one operation to each particular button.

1.8.2. Touchpad

Touchpads are touch-sensitive tablets usually around 2–3 inches (50–75 mm) square. They were first used extensively in Apple PowerBook portable computers but are now used in many other notebook computers and can be obtained separately to replace the mouse on the desktop. They are operated by stroking a finger over their surface, rather like using a simulated trackball. The feel is very different from other input devices, but as with all devices users quickly get used to the action and become proficient.

Because they are small it may require several strokes to move the cursor across the screen. This can be improved by using acceleration settings in the software linking the trackpad movement to the screen movement. Rather than having a fixed ratio of pad distance to screen distance, this varies with the speed of movement. If the finger moves slowly over the pad then the pad movements map to small distances on the screen. If the finger is moving quickly the same distance on the touchpad moves the cursor a long distance. For example, on the trackpad being used when writing this section, a very slow movement of the finger from one side of the trackpad to the other moves the cursor less than 10% of the width of the screen. However, if the finger is moved very rapidly from side to side, the cursor moves the whole width of the screen. In fact, this form of acceleration setting is also used in other indirect positioning devices including the mouse. Fine settings of this sort of parameter makes a great difference to the ‘feel’ of the device.

accurate device, but the more accurate versions can be expensive. It is fine for selection but not for drawing since the eye does not move in smooth lines. Also, in real applications it can be difficult to distinguish deliberately gazing at something and accidentally glancing at it.

Such systems have been used in military applications, notably for guiding air-to-air missiles to their targets, but are starting to find more peaceable uses, for disabled users and for workers in environments where it is impossible for them to use their hands. The rarity of the eyegaze is due partly to its novelty and partly to its expense, and it is usually found only in certain domain-specific applications. Within HCI it is particularly useful as part of evaluation as one is able to trace exactly where the user is looking. As prices drop and the technology becomes less intrusive, we may see more applications using eyegaze, especially in virtual reality and augmented reality areas.

1.8.9. Cursor keys and discrete positioning

All of the devices we have discussed are capable of giving near continuous 2D positioning, with varying degrees of accuracy. For many applications we are only interested in positioning within a sequential list such as a menu or amongst 2D cells as in a spreadsheet. Even for moving within text discrete up/down left/right keys can sometimes be preferable to using a mouse.

Cursor keys are available on most keyboards. Four keys on the keyboard are used to control the cursor, one each for up, down, left and right. There is no standardized layout for the keys. Some layouts are shown in Figure 2.7, but the most common now is the inverted 'T'.

Cursor keys used to be more heavily used in character-based systems before windows and mice were the norm. However, when logging into remote machines such as web servers, the interface is often a virtual character-based terminal within a telnet window. In such applications it is common to find yourself in a 1970s world of text editors controlled sometimes using cursor keys and sometimes by more arcane combinations of control keys!

Small devices such as mobile phones, personal entertainment and television remote controls often require discrete control, either dedicated to a particular function such as volume, or for use as general menu selection. The satellite TV remote control has dedicated '+/-' buttons for controlling volume and stepping between channels. It also has a central cursor pad that is used for on-screen menus. The mobile phone has a single central joystick-like device. This can be pushed left/right, up/down to navigate within the small 3*3 array of graphical icons as well as select from text menus.

1.9. PAPER: PRINTING AND SCANNING

Some years ago, a recurrent theme of information technology was the paperless office. In the paperless office, documents would be produced, dispatched, read and filed online. The only time electronic information would be committed to paper would be when it went out of the office to ordinary customers, or to other firms who were laggards in this technological race. This vision was fueled by rocketing property prices, and the realization that the floor space for a wastepaper basket could cost thousands in rent each year. Some years on, many traditional paper files are now online, but the desire for the completely paperless office has faded. Offices still have wastepaper baskets, and extra floor space is needed for the special computer tables to house 14-inch color monitors.

Printing

If anything, computer systems have made it easier to produce paper documents. It is so easy to run off many copies of a letter (or book), in order to get it looking 'just right'. Older printers had a fixed set of characters available on a printhead. These varied from the traditional line printer to golf-ball and daisy-wheel printers. To change a typeface or the size of type meant changing the printhead, and was an awkward, and frequently messy, job, but for many years the daisy-wheel printer was the only means of producing high-quality output at an affordable price. However, the drop in the price of laser printers coupled with the availability of other cheap high-quality printers means that daisy-wheels are fast becoming a rarity.

All of the popular printing technologies, like screens, build the image on the paper as a series of dots. This enables, in theory, any character set or graphic to be printed limited only by the resolution of the dots. This resolution is measured in dots per inch (dpi). Imagine a sheet of graph paper, and building up an image by putting dots at the intersection of each line. The number of lines per inch in each direction is the resolution in dpi. For some mechanical printers this is slightly confused: the dots printed may be bigger than the gaps, neighboring printheads may not be able to print simultaneously and may be offset relative to one another (a diamond-shaped rather than rectangular grid). These differences do not make too much difference to the user, but mean that, given two printers at the same nominal resolution, the output of one looks better than that of the other, because it has managed the physical constraints better.

The most common types of dot-based printers are dot-matrix printers, ink-jet printers and laser printers. These are listed roughly in order of increasing resolution and quality, where dot-matrix printers typically have a resolution of 80–120 dpi rising to about 300–600 dpi for ink-jet printers

and 600–2400 dpi for laser printers. By varying the quantity of ink and quality of paper, ink-jet printers can be used to print photo-quality prints from digital photographs.

As well as resolution, printers vary in speed and cost. Typically, office-quality ink-jet or laser printers produce between four and eight pages per minute. Dot-matrix printers are more often rated in characters per second (cps), and typical speeds may be 200 cps for draft and 50 cps for letter-quality print. In practice, this means no more than a page or so per minute. These are maximum speeds for simple text, and printers may operate much more slowly for graphics.

Color ink-jet printers are substantially cheaper than even monochrome laser printers. However, the recurrent costs of consumables may easily dominate this initial cost. Both jet and laser printers have special-purpose parts (print cartridges, toner, print drums), which need to be replaced every few thousand sheets; and they must also use high-grade paper. It may be more difficult to find suitable grades of recycled paper for laser printers.

Fonts and page description languages

Some printers can act in a mode whereby any characters sent to them are printed, typewriter style, in a single font. Another case, simple in theory, is when you have a bitmap picture and want to print it. The dots to print are sent to the printer, and no further interpretation is needed. However, in practice, it is rarely so simple.

Many printed documents are far more complex – they incorporate text in many different fonts and many sizes, often italicized, emboldened and underlined. Within the text you will find line drawings, digitized photographs and pictures generated from ‘paint’ packages, including the ubiquitous ‘clip art’. Sometimes the computer does all the work, converting the page image into a bitmap of the right size to be sent to the printer. Alternatively, a description of the page may be sent to the printer. At the simplest level, this will include commands to set the print position on the page, and change the font size.

More sophisticated printers can accept a page description language, the most common of which is PostScript. This is a form of programming language for printing. It includes some standard programming constructs, but also some special ones: paths for drawing lines and curves, sophisticated character and font handling and scaled bitmaps. The idea is that the description of a page is far smaller than the associated bitmap, reducing the time taken to send the page to the printer. A bitmap version of an A4 laser printer page at 300 dpi takes 8 Mbytes; to send this down

small images such as photographs are quite common, and as long as one direction is less than the width of the scanner, they can be read in one pass.

Scanners work by shining a beam of light at the page and then recording the intensity and color of the reflection. Like photocopiers, the color of the light that is shone means that some colors may appear darker than others on a monochrome scanner. For example, if the light is pure red, then a red image will reflect the light completely and thus not appear on the scanned image.

Like printers, scanners differ in resolution, commonly between 600 and 2400 dpi, and like printers the quoted resolution needs careful interpretation. Many have a lower resolution scan head but digitally interpolate additional pixels – the same is true for some digital cameras. Monochrome scanners are typically only found in multi-function devices, but color scanners usually have monochrome modes for black and white or grayscale copying. Scanners will usually return up to 256 levels of gray or RGB (red, green, blue) color. If a pure monochrome image is required (for instance, from a printed page), then it can threshold the grayscale image; that is, turn all pixels darker than some particular value black, and the rest white.

Scanners are used extensively in desktop publishing (DTP) for reading in hand-drawn pictures and photographs. This means that cut and paste can be performed electronically rather than with real glue. In addition, the images can be rotated,

scaled and otherwise transformed, using a variety of image manipulation software tools. Such tools are becoming increasingly powerful, allowing complex image transformations to be easily achieved; these range from color correction, through the merging of multiple images to the application of edge-detection and special effects filters. The use of multiple layers allows photomontage effects that would be impossible with traditional photographic or paper techniques. Even where a scanned image is simply going to be printed back out as part of a larger publication, some processing typically has to be performed to match the scanned colors with those produced during printing. For film photographs there are also special film scanners that can scan photographic negatives or color slides. Of course, if the photographs are digital no scanning is necessary.

Another application area is in document storage and retrieval systems, where paper documents are scanned and stored on computer rather than (or sometimes as well as) in a filing cabinet. The costs of maintaining paper records are enormous, and electronic storage can be cheaper, more reliable and more flexible. Storing a bitmap image is neither most useful (in terms of access

methods), nor space efficient (as we will see later), so scanning may be combined with optical character recognition to obtain the text rather than the page image of the document.

Optical character recognition (OCR) is the process whereby the computer can ‘read’ the characters on the page. It is only comparatively recently that print could be reliably read, since the wide variety of typefaces and print sizes makes this more difficult than one would imagine – it is not simply a matter of matching a character shape to the image on the page. In fact, OCR is rather a misnomer nowadays as, although the document is optically scanned, the OCR software itself operates on the bitmap image. Current software can recognize ‘unseen’ fonts and can even produce output in word-processing formats, preserving super- and subscripts, centering, italics and so on.

Another important area is electronic publishing for multimedia and the world wide web. Whereas in desktop publishing the scanned image usually ends up (after editing) back on paper, in electronic publishing the scanned image is destined to be viewed on screen. These images may be used simply as digital photographs or may be made active, whereby clicking on some portion of the image causes pertinent information to be displayed (see Chapter 3 for more on the point-and-click style of interaction). One big problem when using electronic images is the plethora of formats for storing graphics. Another problem is the fact that different computers can display different numbers of colors and that the appearance of the same image on different monitors can be very different.

The importance of electronic publishing and also the ease of electronically manipulating images for printing have made the digital camera increasingly popular. Rather than capturing an image on film, a digital camera has a small light-sensitive chip that can directly record an image into memory.

1.10. MEMORY

Like human memory, we can think of the computer’s memory as operating at different levels, with those that have the faster access typically having less capacity. By analogy with the human memory, we can group these into short-term and long-term memories (STM and LTM), but the analogy is rather weak – the capacity of the computer’s STM is a lot more than seven items! The different levels of computer memory are more commonly called primary and secondary storage. The details of computer memory are not in themselves of direct interest to the user interface designer. However, the limitations in capacity and access methods are important constraints on the sort of interface that can be designed. After some fairly basic information, we will put the raw memory capacity into perspective with the sort of information which can be stored, as well as

again seeing how advances in technology offer more scope for the designer to produce more effective interfaces. In particular, we will see how the capacity of typical memory copes with video images as these are becoming important as part of multimedia applications.

1.10.1. RAM and short-term memory (STM)

At the lowest level of computer memory are the registers on the computer chip, but these have little impact on the user except in so far as they affect the general speed of the computer. Most currently active information is held in silicon-chip random access memory (RAM). Different forms of RAM differ as to their precise access times, power consumption and characteristics. Typical access times are of the order of 10 nanoseconds, that is a hundred-millionth of a second, and information can be accessed at a rate of around 100 Mbytes (million bytes) per second. Typical storage in modern personal computers is between 64 and 256 Mbytes.

Most RAM is volatile, that is its contents are lost when the power is turned off. However, many computers have small amount of non-volatile RAM, which retains its contents, perhaps with the aid of a small battery. This may be used to store setup information in a large computer, but in a pocket, organizer will be the whole memory. Non-volatile RAM is more expensive so is only used where necessary, but with many notebook computers using very low-power static RAM, the divide is shrinking. By strict analogy, non-volatile RAM ought to be classed as LTM, but the important thing we want to emphasize is the gulf between STM and LTM in a traditional computer system.

In PDAs the distinctions become more confused as the battery power means that the system is never completely off, so RAM memory effectively lasts forever. Some also use flash memory, which is a form of silicon memory that sits between fixed content ROM (read-only memory) chips and normal RAM. Flash memory is relatively slow to write, but once written retains its content even with no power whatsoever. These are sometimes called silicon disks on PDAs. Digital cameras typically store photographs in some form of flash media and small flash-based devices are used to plug into a laptop or desktop's USB port to transfer data.

1.10.2. Disks and long-term memory (LTM)

For most computer users the LTM consists of disks, possibly with small tapes for backup. The existence of backups, and appropriate software to generate and retrieve them, is an important area for user security. However, we will deal mainly with those forms of storage that impact the interactive computer user.

image itself. In addition, an icon may have a ‘hot-spot’ for use as a cursor. If you think of all the ways of encoding these features, or leaving them implicit, and then consider all the combinations of these different encodings, you can see why there are problems. And all this before we have even considered the effects of compression! There is, in fact, a whole software industry producing packages that convert from one format to another.

Given the range of storage standards (or rather lack of standards), there is no easy advice as to which is best, but if you are writing a new word processor and are about to decide how to store the document on disk, think, just for a moment, before defining yet another format.

1.10.6. Methods of access

Standard database access is by special key fields with an associated index. The user has to know the key before the system can find the information. A telephone directory is a good example of this. You can find out someone’s telephone number if you know their name (the key), but you cannot find the name given the number. This is evident in the interface of many computer systems. So often, when you contact an organization, they can only help you if you give your customer number, or last order number. The usability of the system is seriously impaired by a shortsighted reliance on a single key and index. In fact, most database systems will allow multiple keys and indices, allowing you to find a record given partial information. So these problems are avoidable with only slight foresight.

There are valid reasons for not indexing on too many items. Adding extra indices adds to the size of the database, so one has to balance ease of use against storage cost. However, with ever-increasing disk sizes, this is not a good excuse for all but extreme examples. Unfortunately, brought up on lectures about algorithmic efficiency, it is easy for computer scientists to be stingy with storage. Another, more valid, reason for restricting the fields you index is privacy and security. For example, telephone companies will typically hold an online index that, given a telephone number, would return the name and address of the subscriber, but to protect the privacy of their customers, this information is not divulged to the general public.

It is often said that dictionaries are only useful for people who can spell. Bad spellers do not know what a word looks like so cannot look it up to find out. Not only in spelling packages, but in general, an application can help the user by matching badly spelt versions of keywords. One example of this is do what I mean (DWIM) used in several of Xerox PARC’s experimental programming environments. If a command name is misspelt the system prompts the user with a

close correct name. Menu-based systems make this less of an issue, but one can easily imagine doing the same with, say, file selection. Another important instance of this principle is Soundex, a way of indexing words, especially names. Given a key, Soundex finds those words which sound similar. For example, given McCloud, it would find MacCleod. These are all examples of forgiving systems, and in general one should aim to accommodate the user's mistakes. Again, there are exceptions to this: you do not want a bank's automated teller machine (ATM) to give money when the PIN number is almost correct!

Not all databases allow long passages of text to be stored in records, perhaps setting a maximum length for text strings, or demanding the length be fixed in advance. Where this is the case, the database seriously restricts interface applications where text forms an important part. At the other extreme, free text retrieval systems are centered on unformatted, unstructured text. These systems work by keeping an index of every word in every document, and so you can ask 'give me all documents with the words "human" and "computer" in them'. Programs, such as versions of the UNIX 'grep' command, give some of the same facilities by quickly scanning a list of files for a certain word, but are much slower. On the web, free text search is of course the standard way to find things using search engines.

1.11. PROCESSING AND NETWORKS

Computers that run interactive programs will process in the order of 100 million instructions per second. It sounds a lot and yet, like memory, it can soon be used up. Indeed, the first program written by one of the authors (some while ago) 'hung' and all attempts to debug it failed. Later calculation showed that the program would have taken more than the known age of the universe to complete! Failures need not be as spectacular as that to render a system unusable. Consider, for example, one drawing system known to the authors. To draw a line you press down the mouse button at one end, drag the mouse and then release the mouse button at the other end of the line – but not too quickly. You have to press down the button and then actually hold your hand steady for a moment, otherwise the line starts half way! For activities involving the user's hand-eye coordination, delays of even a fraction of a second can be disastrous.

1.11.1. Effects of finite processor speed

As we can see, speed of processing can seriously affect the user interface. These effects must be taken into account when designing an interactive system. There are two sorts of faults due to processing speed: those when it is too slow, and those when it is too fast!

We saw one example of the former above. This was a functional fault, in that the program did the wrong thing. The system is supposed to draw lines from where the mouse button is depressed to where it is released. However, the program gets it wrong – after realizing the button is down, it does not check the position of the mouse fast enough, and so the user may have moved the mouse before the start position is registered. This is a fault at the implementation stage of the system rather than of the design. But to be fair, the programmer may not be given the right sort of information from lower levels of system software.

A second fault due to slow processing is where, in a sense, the program does the right thing, but the feedback is too slow, leading to strange effects at the interface. In order to avoid faults of the first kind, the system buffers the user input; that is, it remembers keypresses and mouse buttons and movement. Unfortunately, this leads to problems of its own. One example of this sort of problem is cursor tracking, which happens in character-based text editors. The user is trying to move backwards on the same line to correct an error, and so presses the cursor-left key. The cursor moves and when it is over the correct position, the user releases the key. Unfortunately, the system is behind in responding to the user, and so has a few more cursor-left keys

to process – the cursor then overshoots. The user tries to correct this by pressing the cursor-right key, and again overshoots. There is typically no way for the user to tell whether the buffer is empty or not, except by interacting very slowly with the system and observing that the cursor has moved after every keypress.

A similar problem, icon wars, occurs on window systems. The user clicks the mouse on a menu or icon, and nothing happens; for some reason the machine is busy or slow. So the user clicks again, tries something else – then, suddenly, all the buffered mouse clicks are interpreted and the screen becomes a blur of flashing windows and menus. This time, it is not so much that the response is too slow – it is fast enough when it happens – but that the response is variable. The delays due to swapping programs in and out of main memory typically cause these problems.

Furthermore, a style of interaction that is optimal on one machine may not be so on a slower machine. In particular, mouse-based interfaces cannot tolerate delays between actions and feedback of more than a fraction of a second, otherwise the immediacy required for successful interaction is lost. If these responses cannot be met then a more old-fashioned, command-based interface may be required.

Whereas it is immediately obvious that slow responses can cause problems for the user, it is not

UNIT – II

INTERACTION AND INTERFACES

The Interaction: Introduction – Models of interaction – Frameworks and HCI – Ergonomics – Interaction Styles – Elements of the WIMP Interface – Interactivity – The context of the interaction – Experience – Engagement and fun – Paradigms: Introduction – Paradigm for interaction

2.1 THE INTERACTION: INTRODUCTION

Interaction models help us to understand what is going on in the interaction between user and system. They address the translations between what the user wants and what the system does. There are a number of ways in which the user can communicate with the system. At one extreme is batch input, in which the user provides all the information to the computer at once and leaves the machine to perform the task. This approach does involve an interaction between the user and computer but does not support many tasks well. At the other extreme are highly interactive input devices and paradigms, such as direct manipulation and the applications of virtual reality. Here the user is constantly providing instruction and receiving feed- back. These are the types of interactive system we are considering.

In this chapter, we consider the communication between user and system: the interaction. We will look at some models of interaction that enable us to identify and evaluate components of the interaction, and at the physical, social and organizational issues that provide the context for it. We will also survey some of the different styles of interaction that are used and consider how well they support the user.

2.2. MODELS OF INTERACTION

Interaction involves at least two participants: the user and the system. Both are complex, as we have seen, and are very different from each other in the way that they communicate and view the domain and the task. The interface must therefore effectively translate between them to allow the interaction to be successful. This translation can fail at a number of points and for a number of reasons. The use of models of interaction can help us to understand exactly what is going on in the interaction and identify the likely root of difficulties. They also provide us with a framework to compare different interaction styles and to consider interaction problems.

We begin by considering the most influential model of interaction, Norman's execution–evaluation cycle; then we look at another model which extends the ideas of Norman's cycle. Both of these models describe the interaction in terms of the goals and actions of the user. We will therefore briefly discuss the terminology used and the assumptions inherent in the models, before describing the models themselves.

2.2.1. The terms of interaction

Traditionally, the purpose of an interactive system is to aid a user in accomplishing goals from some application domain. A domain defines an area of expertise and knowledge in some real-world activity. Some examples of domains are graphic design, authoring and process control in a factory. A domain consists of concepts that highlight its important aspects. In a graphic design domain, some of the important concepts are geometric shapes, a drawing surface and a drawing utensil. Tasks are operations to manipulate the concepts of a domain. A goal is the desired output from a performed task. For example, one task within the graphic design domain is the construction of a specific geometric shape with particular attributes on the drawing surface. A related goal would be to produce a solid red triangle centered on the canvas. An intention is a specific action required to meet the goal.

Task analysis involves the identification of the problem space for the user of an interactive system in terms of the domain, goals, intentions and tasks. We can use our knowledge of tasks and goals to assess the interactive system that is designed to support them. The concepts used in the design of the system and the description of the user are separate, and so we can refer to them as distinct components, called the System and the User, respectively. The System and User are each described by means of a language that can express concepts relevant in the domain of the application. The System's language we will refer to as the core language and the User's language we will refer to as the task language. The core language describes computational attributes of the domain relevant to the System state, whereas the task language describes psychological attributes of the domain relevant to the User state. The system is assumed to be some computerized application, in the context of this book, but the models apply equally to non-computer applications. It is also a common assumption that by distinguishing between user and system we are restricted to single-user applications. This is not the case. However, the emphasis is on the view of the interaction from a single user's perspective. From this point of view, other users, such as those in a multi-party conferencing system, form part of the system.

achieve that goal. The only way the user can manipulate the machine is through the Input, and so the task must be articulated within the input language. The input language is translated into the core language as operations to be performed by the System. The System then transforms itself as described by the operations; the execution phase of the cycle is complete and the evaluation phase now begins. The System is in a new state, which must now be communicated to the User. The current values of system attributes are rendered as concepts or features of the Output. It is then up to the User to observe the Output and assess the results of the interaction relative to the original goal, ending the evaluation phase and, hence, the interactive cycle. There are four main translations involved in the interaction: articulation, performance, presentation and observation. The User's formulation of the desired task to achieve some goal needs to be articulated in the input language. The tasks are responses of the User and they need to be translated to stimuli for the Input. As pointed out above, this articulation is judged in terms of the coverage from tasks to input and the relative ease with which the translation can be accomplished. The task is phrased in terms of certain psychological attributes that highlight the important features of the domain for the User. If these psychological attributes map clearly onto the input language, then articulation of the task will be made much simpler.

Figure 2.1 The general interaction framework

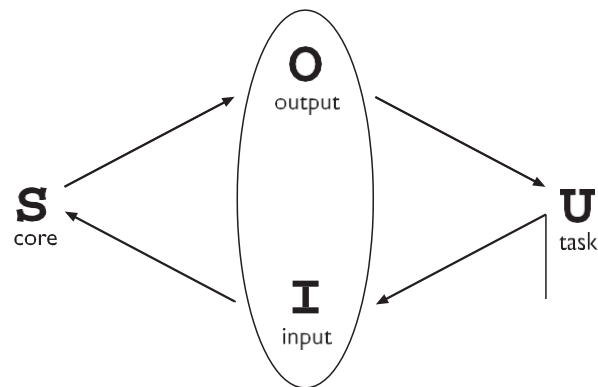
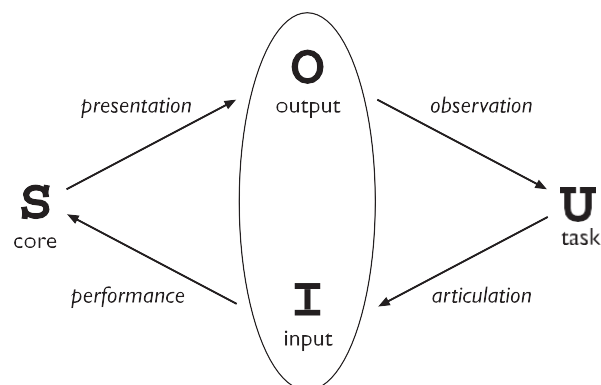


Figure 2.2 Translations between components



An example of a poor mapping, as pointed out by Norman, is a large room with overhead lighting controlled by a bank of switches. It is often desirable to control the lighting so that only one section of the room is lit. We are then faced with the puzzle of determining which switch controls which lights. The result is usually repeated trials and frustration. This arises from the difficulty of articulating a goal (for example, 'Turn on the lights in the front of the room') in an input language that consists of a linear row of switches, which may or may not be oriented to reflect the room layout.

Conversely, an example of a good mapping is in virtual reality systems, where input devices such as data gloves are specifically geared towards easing articulation by making the user's psychological notion of gesturing an act that can be directly realized at the interface. Direct manipulation interfaces, such as those found on common desktop operating systems like the Macintosh and Windows, make the articulation of some file handling commands easier. On the other hand, some tasks, such as repetitive file renaming or launching a program whose icon is not visible, are not at all easy to articulate with such an interface.

At the next stage, the responses of the Input are translated to stimuli for the System. Of interest in assessing this translation is whether the translated input language can reach as many states of the System as is possible using the System stimuli directly. For example, the remote-control units for some compact disc players do not allow the user to turn the power off on the player unit; hence the off state of the player cannot be reached using the remote control's input language. On the panel of the compact disc player, however, there is usually a button that controls the power. The ease with which this translation from Input to System takes place is of less importance because the effort is not expended by the user. However, there can be a real effort expended by the designer and programmer. In this case, the ease of the translation is viewed in terms of the cost of implementation.

Once a state transition has occurred within the System, the execution phase of the interaction is complete and the evaluation phase begins. The new state of the System must be communicated to the User, and this begins by translating the System responses to the transition into stimuli for the Output component. This presentation translation must preserve the relevant system attributes from the domain in the limited expressiveness of the output devices. The ability to capture the domain concepts of the System within the Output is a question of expressiveness for this translation.

For example, while writing a paper with some word-processing package, it is necessary at times to see both the immediate surrounding text where one is currently composing, say, the current paragraph, and a wider context within the whole paper that cannot be easily displayed on one screen.

Ultimately, the user must interpret the output to evaluate what has happened. The response from the Output is translated to stimuli for the User which trigger assessment. The observation translation will address the ease and coverage of this final translation. For example, it is difficult to tell the time accurately on an unmarked analog clock, especially if it is not oriented properly. It is difficult in a command line interface to determine the result of copying and moving files in a hierarchical file system. Developing a website using a markup language like HTML would be virtually impossible without being able to preview the output through a browser.

Assessing overall interaction

The interaction framework is presented as a means to judge the overall usability of an entire interactive system. In reality, all of the analysis that is suggested by the framework is dependent on the current task (or set of tasks) in which the User is engaged. This is not surprising since it is only in attempting to perform a particular task within some domain that we are able to determine if the tools we use are adequate. For example, different text editors are better at different things. For a particular editing task, one can choose the text editor best suited for interaction relative to the task. The best editor, if we are forced to choose only one, is the one that best suits the tasks most frequently performed. Therefore, it is not too disappointing that we cannot extend the interaction analysis beyond the scope of a particular task.

2.3. FRAMEWORKS AND HCI

As well as providing a means of discussing the details of a particular interaction, frameworks provide a basis for discussing other issues that relate to the interaction. The ACM SIGCHI Curriculum Development Group presents a framework similar to that presented here, and uses it to place different areas that relate to HCI.

In Figure 3.3 these aspects are shown as they relate to the interaction framework. In particular, the field of ergonomics addresses issues on the user side of the interface, covering both input and output, as well as the user's immediate context. Dialog design and interface styles can be placed

particularly along the input branch of the framework, addressing both articulation and performance. However, dialog is most usually associated with the computer and so is biased to that side of the framework.

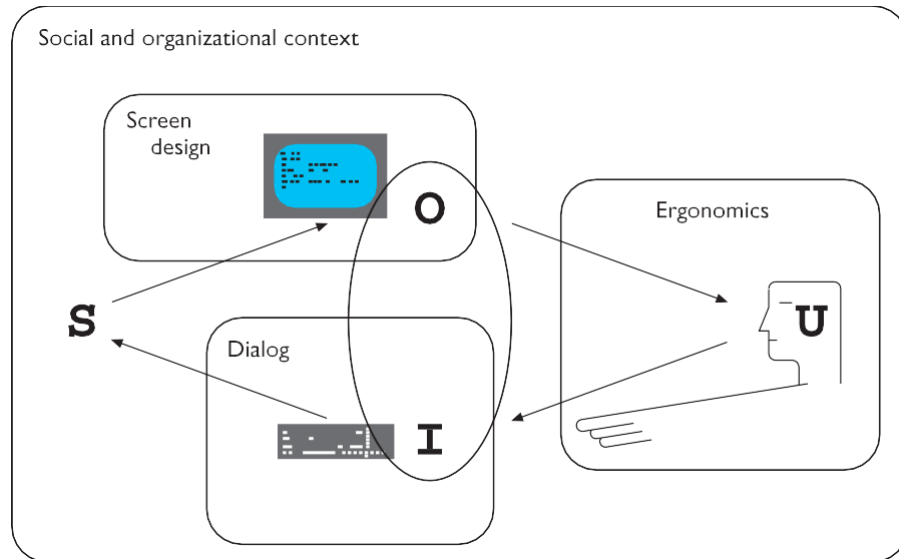


Figure 2.3 A framework for human-computer interaction. Adapted from ACM SIGCHI Curriculum Development Group

Presentation and screen design relates to the output branch of the framework. The entire framework can be placed within a social and organizational context that also affects the interaction. Each of these areas has important implications for the design of interactive systems and the performance of the user.

2.4. ERGONOMICS

Ergonomics (or human factors) is traditionally the study of the physical characteristics of the interaction: how the controls are designed, the physical environment in which the interaction takes place, and the layout and physical qualities of the screen. A primary focus is on user performance and how the interface enhances or detracts from this. In seeking to evaluate these aspects of the interaction, ergonomics will certainly also touch upon human psychology and system constraints. It is a large and established field, which is closely related to but distinct from HCI, and full coverage would demand a book in its own right. Here we consider a few of the issues addressed by ergonomics as an introduction to the field. We will briefly look at the arrangement of controls and displays, the physical environment, health issues and the use of color. These are by no means

exhaustive and are intended only to give an indication of the types of issues and problems addressed by ergonomics.

2.4.1. Arrangement of controls and displays

The exact organization that this will suggest will depend on the domain and the application, but possible organizations include the following:

functional controls and displays are organized so that those that are functionally related are placed together;

sequential controls and displays are organized to reflect the order of their use in a typical interaction (this may be especially appropriate in domains where a particular task sequence is enforced, such as aviation);

frequency controls and displays are organized according to how frequently they are used, with the most commonly used controls being the most easily accessible.

In addition to the organization of the controls and displays in relation to each other, the entire system interface must be arranged appropriately in relation to the user's position. So, for example, the user should be able to reach all controls necessary and view all displays without excessive body movement. Critical displays should be at eye level. Lighting should be arranged to avoid glare and reflection distorting displays. Controls should be spaced to provide adequate room for the user to manoeuvre.

2.4.2. The physical environment of the interaction

As well as addressing physical issues in the layout and arrangement of the machine interface, ergonomics is concerned with the design of the work environment itself. Where will the system be used? By whom will it be used? Will users be sitting, standing or moving about? Again, this will depend largely on the domain and will be more critical in specific control and operational settings than in general computer use. However, the physical environment in which the system is used may influence how well it is accepted and even the health and safety of its users. It should therefore be considered in all design.

The first consideration here is the size of the users. Obviously, this is going to vary considerably. However, in any system the smallest user should be able to reach all the controls (this may include a user in a wheelchair), and the largest user should not be cramped in the environment.

In particular, all users should be comfortably able to see critical displays. For long periods of use, the user should be seated for comfort and stability. Seating should provide back support. If required to stand, the user should have room to move around in order to reach all the controls.

2.4.3. Health issues

Perhaps we do not immediately think of computer use as a hazardous activity but we should bear in mind possible consequences of our designs on the health and safety of users. Leaving aside the obvious safety risks of poorly designed safety-critical systems (aircraft crashing, nuclear plant leaks and worse), there are a number of factors that may affect the use of more general computers. Again, these are factors in the physical environment that directly affect the quality of the interaction and the user's performance:

Physical position as we noted in the previous section, users should be able to reach all controls comfortably and see all displays. Users should not be expected to stand for long periods and, if sitting, should be provided with back support. If a particular position for a part of the body is to be adopted for long periods (for example, in typing) support should be provided to allow rest.

Temperature Although most users can adapt to slight changes in temperature without adverse effect, extremes of hot or cold will affect performance and, in excessive cases, health. Experimental studies show that performance deteriorates at high or low temperatures, with users being unable to concentrate efficiently.

Lighting The lighting level will again depend on the work environment. However, adequate lighting should be provided to allow users to see the computer screen without discomfort or eyestrain. The light source should also be positioned to avoid glare affecting the display.

Noise Excessive noise can be harmful to health, causing the user pain, and in acute cases, loss of hearing. Noise levels should be maintained at a comfortable level in the work environment. This does not necessarily mean no noise at all. Noise can be a stimulus to users and can provide needed confirmation of system activity.

Time The time users spend using the system should also be controlled. As we saw in the previous chapter, it has been suggested that excessive use of CRT displays can be harmful to users, particularly pregnant women.

2.4.4. The use of color

In this section we have concentrated on the ergonomics of physical characteristics of systems, including the physical environment in which they are used. However, ergonomics has a close relationship to human psychology in that it is also concerned with the perceptual limitations of humans. For example, the use of color in displays is an ergonomics issue. As we saw in Chapter 1, the visual system has some limitations with regard to color, including the number of colors that are distinguishable and the relatively low blue acuity. We also saw that a relatively high proportion of the population suffers from a deficiency in color vision. Each of these psychological phenomena leads to ergonomic guidelines; some examples are discussed below.

Colors used in the display should be as distinct as possible and the distinction should not be affected by changes in contrast. Blue should not be used to display critical information. If color is used as an indicator, it should not be the only cue: additional coding information should be included.

The colors used should also correspond to common conventions and user expectations. Red, green and yellow are colors frequently associated with stop, go and standby respectively. Therefore, red may be used to indicate emergency and alarms; green, normal activity; and yellow, standby and auxiliary function. These conventions should not be violated without very good cause.

However, we should remember that color conventions are culturally determined. For example, red is associated with danger and warnings in most western cultures, but in China it symbolizes happiness and good fortune. The color of mourning is black in some cultures and white in others. Awareness of the cultural associations of color is particularly important in designing systems and websites for a global market.

2.5. INTERACTION STYLES

Interaction can be seen as a dialog between the computer and the user. The choice of interface style can have a profound effect on the nature of this dialog. Here we introduce the most common interface styles and note the different effects these have on the interaction. There are a number of common interface styles including

- command line interface
- menus
- natural language

- question/answer and query dialog
- form-fills and spreadsheets
- WIMP
- point and click
- three-dimensional interfaces.

2.5.1. Command line interface

```
sable.soc.staffs.ac.uk> javac HelloWorldApp
javac: invalid argument: HelloWorldApp
use: javac [-g][-O][-classpath path][-d dir] file.java...
sable.soc.staffs.ac.uk> javac HelloWorldApp.java
sable.soc.staffs.ac.uk> java HelloWorldApp
Hello world!!
sable.soc.staffs.ac.uk>
```

Figure 2.4. Command line interface

The command line interface (Figure 3.7) was the first interactive dialog style to be commonly used and, in spite of the availability of menu-driven interfaces, it is still widely used. It provides a means of expressing instructions to the computer directly, using function keys, single characters, abbreviations or whole-word commands. In some systems the command line is the only way of communicating with the system, especially for remote access using telnet. More commonly today it is supplementary to menu-based interfaces, providing accelerated access to the system's functionality for experienced users.

Command line interfaces are powerful in that they offer direct access to system functionality (as opposed to the hierarchical nature of menus), and can be combined to apply a number of tools to the same data. They are also flexible: the command often has a number of options or parameters that will vary its behavior in some way, and it can be applied to many objects at once, making it useful for repetitive tasks. However, this flexibility and power brings with it difficulty in use and learning. Commands must be remembered, as no cue is provided in the command line to indicate which command is needed. They are therefore better for expert users than for novices. This problem can be alleviated a little by using consistent and meaningful commands and abbreviations.

easy movement around the form and allow some fields to be left blank. They also require correction facilities, as users may change their minds or make a mistake about the value that belongs in each field. The dialog style is useful primarily for data entry applications and, as it is easy to learn and use, for novice users. However, assuming a design that allows flexible entry, form filling is also appropriate for expert users.

2.5.6. The WIMP interface

Currently many common environments for interactive computing are examples of the WIMP interface style, often simply called windowing systems. WIMP stands for windows, icons, menus and pointers (sometimes windows, icons, mice and pull-down menus), and is the default interface style for the majority of interactive computer systems in use today, especially in the PC and desktop workstation arena. Examples of WIMP interfaces include Microsoft Windows for IBM PC compatibles, MacOS for Apple Macintosh compatibles and various X Windows-based systems for UNIX.

2.5.7. Point-and-click interfaces

In most multimedia systems and in web browsers, virtually all actions take only a single click of the mouse button. You may point at a city on a map and when you click a window opens, showing you tourist information about the city. You may point at a word in some text and when you click you see a definition of the word. You may point at a recognizable iconic button and when you click some action is performed. This point-and-click interface style is obviously closely related to the WIMP style.

It clearly overlaps in the use of buttons, but may also include other WIMP elements. However, the philosophy is simpler and more closely tied to ideas of hypertext. In addition, the point-and-click style is not tied to mouse-based interfaces, and is also extensively used in touchscreen information systems. In this case, it is often combined with a menu-driven interface.

The point-and-click style has been popularized by world wide web pages, which incorporate all the above types of point-and-click navigation: highlighted words, maps and iconic buttons.

2.5.8. Three-dimensional interfaces

There is an increasing use of three-dimensional effects in user interfaces. The most obvious example is virtual reality, but VR is only part of a range of 3D techniques available to the interface designer.

The simplest technique is where ordinary WIMP elements, buttons, scroll bars, etc., are given a

3D appearance using shading, giving the appearance of being sculpted out of stone. By unstated convention, such interfaces have a light source at their top right. Where used judiciously, the raised areas are easily identifiable and can be used to highlight active areas. Unfortunately, some interfaces make indiscriminate use of sculptural effects, on every text area, border and menu, so all sense of differentiation is lost.

2.6. ELEMENTS OF WIMP INTERFACE

We have already noted the four key features of the WIMP interface that give it its name – windows, icons, pointers and menus – and we will now describe these in turn. There are also many additional interaction objects and techniques commonly used in WIMP interfaces, some designed for specific purposes and others more general. We will look at buttons, toolbars, palettes and dialog boxes.

Together, these elements of the WIMP interfaces are called widgets, and they comprise the toolkit for interaction between user and system.

3.6.1.Windows

Windows are areas of the screen that behave as if they were independent terminals in their own right. A window can usually contain text or graphics, and can be moved or resized. More than one window can be on a screen at once, allowing separate tasks to be visible at the same time. Users can direct their attention to the different windows as they switch from one thread of work to another.

If one window overlaps the other, the back window is partially obscured, and then refreshed when exposed again. Overlapping windows can cause problems by obscuring vital information, so windows may also be tiled, when they adjoin but do not overlap each other. Alternatively, windows may be placed in a cascading fashion, where each new window is placed slightly to the left and below the previous window. In some systems this layout policy is fixed, in others it can be selected by the user.

Usually, windows have various things associated with them that increase their usefulness. Scrollbars are one such attachment, allowing the user to move the contents of the window up and down, or from side to side. This makes the window behave as if it were a real window onto a much larger world, where new information is brought into view by manipulating the scrollbars.

There is usually a title bar attached to the top of a window, identifying it to the user, and there may be special boxes in the corners of the window to aid resizing, closing, or making as large as possible.

In addition, some systems allow windows within windows. For example, in Microsoft Office applications, such as Excel and Word, each application has its own window and then within this each document has a window. It is often possible to have different layout policies within the different application windows.

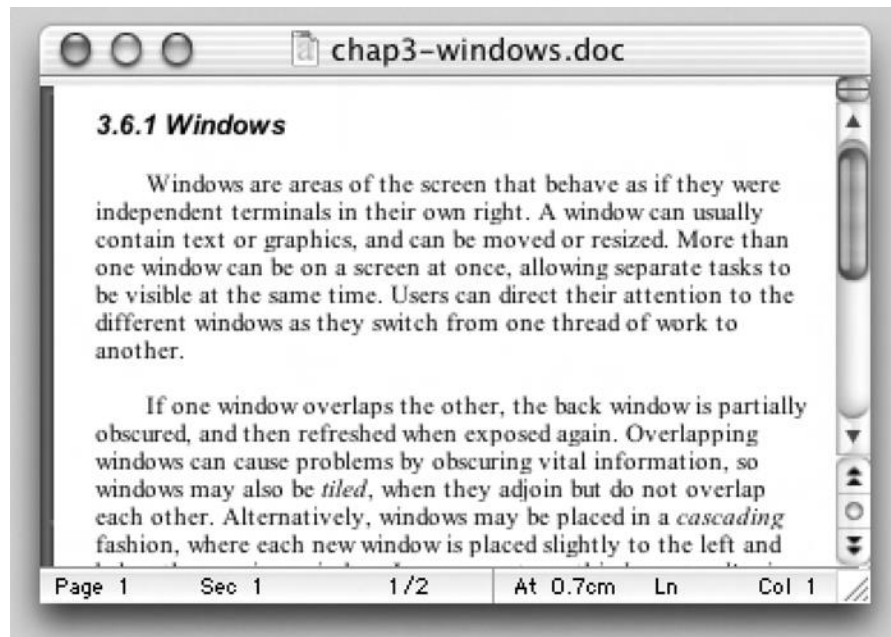


Figure 2.6. A typical window.

2.6.2. Icons

Windows can be closed and lost forever, or they can be shrunk to some very reduced representation. A small picture is used to represent a closed window, and this representation is known as an icon. By allowing icons, many windows can be available on the screen at the same time, ready to be expanded to their full size by clicking on the icon. Shrinking a window to its icon is known as iconifying the window. When a user temporarily does not want to follow a particular thread of dialog, he can suspend that dialog by iconifying the window containing the dialog. The icon saves space on the screen and serves as a reminder to the user that he can subsequently resume the dialog by opening up the window. a few examples of some icons used in a typical windowing system (MacOS X).

Icons can also be used to represent other aspects of the system, such as a waste- basket for throwing unwanted files into, or various disks, programs or functions that are accessible to the user. Icons can take many forms: they can be realistic representations of the objects that they

2.6.6. Toolbars

Many systems have a collection of small buttons, each with icons, placed at the top or side of the window and offering commonly used functions. The function of this toolbar is similar to a menu bar, but as the icons are smaller than the equivalent text more functions can be simultaneously displayed. Sometimes the content of the toolbar is fixed, but often users can customize it, either changing which functions are made available, or choosing which of several predefined toolbars is displayed.

2.6.7. Palettes

In many application programs, interaction can enter one of several modes. The defining characteristic of modes is that the interpretation of actions, such as keystrokes or gestures with the mouse, changes as the mode changes. For example, using the standard UNIX text editor vi, keystrokes can be interpreted either as operations to insert characters in the document (insert mode) or as operations to perform file manipulation (command mode). Problems occur if the user is not aware of the current mode. Palettes are a mechanism for making the set of possible modes and the active mode visible to the user. A palette is usually a collection of icons that are reminiscent of the purpose of the various modes. An example in a drawing package would be a collection of icons to indicate the pixel color or pattern that is used to fill in objects, much like an artist's palette for paint.

Some systems allow the user to create palettes from menus or toolbars. In the case of pull-down menus, the user may be able 'tear off' the menu, turning it into a palette showing the menu items. In the case of toolbars, he may be able to drag the toolbar away from its normal position and place it anywhere on the screen. Tear-off menus are usually those that are heavily graphical anyway, for example line-style or color selection in a drawing package.

2.6.8. Dialog boxes

Dialog boxes are information windows used by the system to bring the user's attention to some important information, possibly an error or a warning used to prevent a possible error. Alternatively, they are used to invoke a sub dialog between user and system for a very specific task that will normally be embedded within some larger tasks. For example, most interactive applications result in the user creating some file that will have to be named and stored within the filing system. When the user or system wants to save the file, a dialog box can be used to allow the user to name the file and indicate where it is to be located within the filing system. When the

save sub dialog is complete, the dialog box will disappear. Just as windows are used to separate the different threads of user–system dialog, so too are dialog boxes used to factor out auxiliary task threads from the main task dialog.

2.7. INTERACTIVITY

It is worth remembering that interactivity is the defining feature of an interactive system. This can be seen in many areas of HCI. For example, the recognition rate for speech recognition is too low to allow transcription from tape, but in an airline reservation system, so long as the system can reliably recognize yes and no it can reflect back its understanding of what you said and seek confirmation. Speech-based input is difficult, speech-based interaction easier. Also, in the area of information visualization the most exciting developments are all where users can interact with a visualization in real time, changing parameters and seeing the effect.

Interactivity is also crucial in determining the ‘feel’ of a WIMP environment. All WIMP systems appear to have virtually the same elements: windows, icons, menus, pointers, dialog boxes, buttons, etc. However, the precise behavior of these elements differs both within a single environment and between environments. For example, we have already discussed the different behavior of pull-down and fall-down menus. These look the same, but fall-down menus are more easily invoked by accident (and not surprisingly the windowing environments that use them have largely fallen into disuse!). In fact, menus are a major difference between the MacOS and Microsoft Windows environments: in MacOS you have to keep the mouse depressed through- out menu selection; in Windows you can click on the menu bar and a pull-down menu appears and remains there until an item is selected or it is cancelled. In older computer systems, the order of interaction was largely determined by the machine. You did things when the computer was ready. In WIMP environments, the user takes the initiative, with many options and often many applications simultaneously available. The exceptions to this are pre-emptive parts of the interface, where the system for various reasons wrests the initiative away from the user, perhaps because of a problem or because it needs information in order to continue.

The major example of this is modal dialog boxes. It is often the case that when a dialog box appears the application will not allow you to do anything else until the dialog box has been completed or cancelled. In some cases this may simply block the application, but you can perform tasks in other applications. In other cases you can do nothing at all until the dialog box has been completed. An especially annoying example is when the dialog box asks a question, perhaps simply for

confirmation of an action, but the information you need to answer is hidden by the dialog box!

There are occasions when modal dialog boxes are necessary, for example when a major fault has been detected, or for certain kinds of instructional software. However, the general philosophy of modern systems suggests that one should minimize the use of pre-emptive elements, allowing the user maximum flexibility.

Interactivity is also critical in dealing with errors. We discussed slips and mistakes earlier in the chapter, and some ways to try to prevent these types of errors. The other way to deal with errors is to make sure that the user or the system is able to tell when errors have occurred. If users can detect Errors, then they can correct them. So, even if errors occur, the interaction as a whole succeeds. This ability to detect and correct is important both at the small scale of button presses and keystrokes and also at the large scale. For example, if you have sent a client a letter and expect a reply, you can put in your diary a note on the day you expect a reply. If the other person forgets to reply or the letter gets lost in the post you know to send a reminder or ring when the due day passes.

2.8.THE CONTEXT OF THE INTERACTION

We have been considering the interaction between a user and a system, and how this is affected by interface design. This interaction does not occur within a vacuum. We have already noted some of the physical factors in the environment that can directly affect the quality of the interaction. This is part of the context in which the interaction takes place. But this still assumes a single user operating a single, albeit complex, machine. In reality, users work within a wider social and organizational context. This provides the wider context for the interaction, and may influence the activity and motivation of the user. Here we will confine our discussion to the influence social and organizational factors may have on the user's interaction with the system. These may not be factors over which the designer has control. However, it is important to be aware of such influences to understand the user and the work domain fully.

The presence of other people in a work environment affects the performance of the worker in any task. In the case of peers, competition increases performance, at least for known tasks. Similarly the desire to impress management and superiors improves performance on these tasks. However, when it comes to acquisition of new skills, the presence of these groups can inhibit performance, owing to the fear of failure. Consequently, privacy is important to allow users the opportunity to experiment.

In order to perform well, users must be motivated. There are a number of possible sources of

motivation, as well as those we have already mentioned, including fear, allegiance, ambition and self-satisfaction. The last of these is influenced by the user's perception of the quality of the work done, which leads to job satisfaction. If a system makes it difficult for the user to perform necessary tasks, or is frustrating to use, the user's job satisfaction, and consequently performance, will be reduced.

The user may also lose motivation if a system is introduced that does not match the actual requirements of the job to be done. Often systems are chosen and introduced by managers rather than the users themselves. In some cases, the manager's perception of the job may be based upon observation of results and not on actual activity. The system introduced may therefore impose a way of working that is unsatisfactory to the users. If this happens there may be three results: the system will be rejected, the users will be resentful and unmotivated, or the user will adapt the intended interaction to his own requirements. This indicates the importance of involving actual users in the design process.

On the other hand, the introduction of new technology may prove to be a motivation to users, particularly if it is well designed, integrated with the user's current work, and challenging. Providing adequate feedback is an important source of motivation for users. If no feedback is given during a session, the user may become bored, unmotivated or, worse, unsure of whether the actions performed have been successful. In general, an action should have an obvious effect to prevent this confusion and to allow early recovery in the case of error. Similarly, if system delays occur, feedback can be used to prevent frustration on the part of the user – the user is then aware of what is happening and is not left wondering if the system is still working.

2.9. EXPERIENCE, ENGAGEMENT AND FUN

Ask many in HCI about usability and they may use the words 'effective' and 'efficient'. Some may add 'satisfaction' as well. This view of usability seems to stem mainly from the Taylorist tradition of time and motion studies: if you can get the worker to pull the levers and turn the knobs in the right order then you can shave 10% off production costs.

However, users no longer see themselves as cogs in a machine. Increasingly, applications are focused outside the closed work environment: on the home, leisure, entertainment, shopping. It is not sufficient that people can use a system, they must want to use it.

Even from a pure economic standpoint, your employees are likely to work better and more effectively if they enjoy what they are doing!

2.9.1. Understanding experience

Shopping is an interesting example to consider. Most internet stores allow you to buy things, but do you go shopping? Shopping is as much about going to the shops, feeling the clothes, being with friends. You can go shopping and never intend to spend money. Shopping is not about an efficient financial transaction; it is an experience.

But experience is a difficult thing to pin down; we understand the idea of a good experience, but how do we define it and even more difficult how do we design it?

Csikszentmihalyi [82] looked at extreme experiences such as climbing a rock face in order to understand that feeling of total engagement that can sometimes happen. He calls this flow and it is perhaps related to what some sportspeople refer to as being ‘in the zone’. This sense of flow occurs when there is a balance between anxiety and boredom. If you do something that you know you can do it is not engaging; you may do it automatically while thinking of something else, or you may simply become bored. Alternatively, if you do something completely outside your abilities you may become anxious and, if you are half way up a rock face, afraid. Flow comes when you are teetering at the edge of your abilities, stretching yourself to or a little beyond your limits.

In education there is a similar phenomenon. The zone of proximal development is those things that you cannot quite do yourself, but you can do with some support, whether from teachers, fellow pupils, or electronic or physical materials. Learning is at its best in this zone. Notice again this touching of limits.

Of course, this does not fully capture the sense of experience, and there is an active subfield of HCI researchers striving to make sense of this, building on the work of psychologists and philosophers on the one hand and literary analysis, film making and drama on the other.

2.9.2. Designing experience

Some of the authors were involved in the design of virtual Christmas crackers. These are rather like electronic greetings cards, but are based on crackers. For those who have not come across them, Christmas crackers are small tubes of paper between 8 and 12 inches long (20–30 cm). Inside there are a small toy, a joke or motto and a paper hat. A small strip of card is threaded through, partly coated with gunpowder. When two people at a party pull the cracker, it bursts apart with a small bang from the gunpowder and the contents spill out.



The virtual cracker does not attempt to fully replicate each aspect of the physical characteristics and process of pulling the cracker, but instead seeks to reproduce the experience. To do this the original crackers experience was deconstructed and each aspect of the experience produced in a similar, but sometimes different, way in the new media. Table 2.1 shows the aspects of the experience deconstructed and reconstructed in the virtual cracker.

For example, the cracker contents are hidden inside; no one knows what toy or joke will be inside. Similarly, when you create a virtual cracker you normally cannot see the contents until the recipient has opened it. Even the recipient initially sees a page with just an image of the cracker; it is only after the recipient has clicked on the ‘open’ icon that the cracker slowly opens and you get to see the joke, web toy and mask.

	Real cracker	Virtual cracker
Surface elements		
Design	Cheap and cheerful	Simple page/graphics
Play	Plastic toy and joke	Web toy and joke
Dressing up	Paper hat	Mask to cut out
Experienced effects		
Shared	Offered to another	Sent by email, message
Co-experience	Pulled together	Sender can't see content until opened by recipient
Excitement	Cultural connotations	Recruited expectation
Hiddenness	Contents inside	First page – no contents
Suspense	Pulling cracker	Slow... page change
Surprise	Bang (when it works)	WAV file (when it works)

Table 2.1. The crackers experience

The mask is also worth looking at. The first potential design was to have a picture of a face with a hat on it – well, it wouldn't rank highly on excitement! The essential feature of the paper hat is that you can dress up. An iconic hat hardly does that.

2.9.3. Physical design and engagement

Designers are faced with many constraints:

Ergonomic You cannot physically push buttons if they are too small or too close.

Physical The size or nature of the device may force certain positions or styles of control, for example, a dial like the one on the washing machine would not fit on the MiniDisc controller; high-voltage switches cannot be as small as low-voltage ones.

Legal and safety Cooker controls must be far enough from the pans that you do not burn yourself, but also high enough to prevent small children turning them on.

Context and environment The microwave's controls are smooth to make them easy to clean in the kitchen.

Aesthetic The controls must look good.

Economic It must not cost too much!

These constraints are themselves often contradictory and require trade-offs to be made. For example, even within the safety category front-mounted controls are better in that they can be turned on or off without putting your hands over the pans and hot steam, but back-mounted controls are further from children's grasp. The MiniDisc player is another example; it physically needs to be small, but this means there is not room for all the controls you want given the minimum size that can be manipulated. In the case of the cooker there is no obvious best solution and so different designs favor one or the other. In the case of the MiniDisc player the end knob is multi-function. This means the knob is ergonomically big enough to turn and physically small enough to fit, but at the cost of a more complex interaction style.

To add to this list of constraints there is another that makes a major impact on the ease of use and also the ability of the user to become engaged with the device, for it to become natural to use:

Fluidity The extent to which the physical structure and manipulation of the device naturally relate to the logical functions it supports.

2.9.4. Managing value

If we want people to want to use a device or application, we need to understand their personal values. Why should they want to use it? What value do they get from using it? Now when we say value here we don't mean monetary value, although that may be part of the story, but all the things that drive a person. For some people this may include being nice to colleagues, being ecologically friendly, being successful in their career. Whatever their personal values are, if we ask someone to do something or use something they are only likely to do it if the value to them exceeds the cost.

This is complicated by the fact that for many systems the costs such as purchase cost, download time of a free application, learning effort are incurred up front, whereas often the returns – faster work, enjoyment of use – are seen later. In economics, businesses use a measure called 'net present value' to calculate what a future gain is worth today; because money can be invested,

£100 today is worth the same as perhaps £200 in five years' time. Future gain is discounted. For human decision making, future gains are typically discounted very highly; many of us are bad at saving for tomorrow or even keeping the best bits of our dinner until last. This means that not only must we understand people's value systems, but we must be able to offer gains sooner as well as later, or at least produce a very good demonstration of potential future gains so that they have a perceived current value.

When we were preparing the website for the second edition of this book, we thought very hard about how to give things that were of value to those who had the book, and also to those who hadn't. The latter is partly because we are all academics and researchers in the field and so want to contribute to the HCI community, but also of course we would like lots of people to buy the book. One option we thought of was to put the text online, which would be good for people without the book, but this would have less value to people who have the book (they might even be annoyed that those who hadn't paid should have access). The search mechanism was the result of this process (Figure 2.8). It gives value to those who have the book because it is a way of finding things. It is of value to those who don't because it acts as a sort of online encyclopedia of HCI.

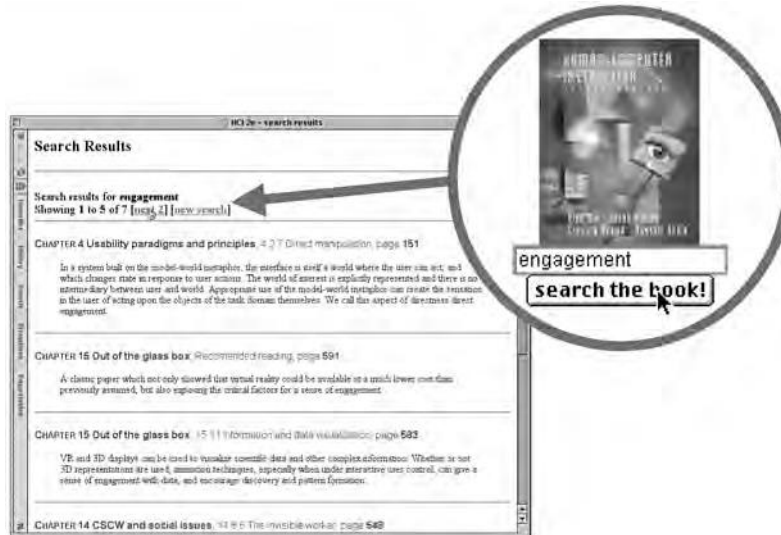


Figure 2.8. The web-based book search facility

2.10. PARADIGMS: INTRODUCTION

The primary objective of an interactive system is to allow the user to achieve particular goals in some application domain, that is, the interactive system must be usable. The designer of an interactive system, then, is posed with two open questions:

1. How can an interactive system be developed to ensure its usability?
2. How can the usability of an interactive system be demonstrated or measured?

One approach to answering these questions is by means of example, in which successful interactive systems are commonly believed to enhance usability and, therefore, serve as paradigms for the development of future products.

We believe that we now build interactive systems that are more usable than those built in the past. We also believe that there is considerable room for improvement in designing more usable systems in the future.

The great advances in computer technology have increased the power of machines and enhanced the bandwidth of communication between humans and computers. The impact of technology alone, however, is not sufficient to enhance its usability. As our machines have become more powerful, the key to increased usability has come from the creative and considered application of the technology to accommodate and augment the power of the human. Paradigms for interaction have for the most part been dependent upon technological advances and their creative application to enhance interaction.

In this chapter, we investigate some of the principal historical advances in interactive designs. What is important to notice here is that the techniques and designs mentioned are recognized as major improvements in interaction, though it is sometimes hard to find a consensus for the reason behind the success. It is even harder to predict ahead what the new paradigms will be. Often new paradigms have arisen through exploratory designs that have then been seen, after the fact, to have created a new base point for future design.

2.11.PARADIGM FOR INTERACTION

2.11.1.Time Sharing

In the 1940s and 1950s, the significant advances in computing consisted of new hardware technologies. Mechanical relays were replaced by vacuum electron tubes. Tubes were replaced by transistors, and transistors by integrated chips, all of which meant that the amount of sheer computing power was increasing by orders of magnitude. By the 1960s it was becoming apparent that the explosion of growth in computing power would be wasted if there was not an equivalent explosion of ideas about how to channel that power. One of the leading advocates of research into human-centered applications of computer technology was J. C. R. Licklider, who became the director of the Information Processing Techniques Office of the US Department of Defense's

Advanced Research Projects Agency (ARPA). It was Licklider's goal to finance various research centers across the United States in order to encourage new ideas about how best to apply the burgeoning computing technology.

One of the major contributions to come out of this new emphasis in research was the concept of time sharing, in which a single computer could support multiple users. Previously, the human (or more accurately, the programmer) was restricted to batch sessions, in which complete jobs were submitted on punched cards or paper tape to an operator who would then run them individually on the computer. Time-sharing systems of the 1960s made programming a truly interactive venture and brought about a subculture of programmers known as 'hackers' – single-minded masters of detail who took pleasure in understanding complexity. Though the purpose of the first interactive time-sharing systems was simply to augment the programming capabilities of the early hackers, it marked a significant stage in computer applications for human use. Rather than rely on a model of interaction as a pre-planned activity that resulted in a complete set of instructions being laid out for the computer to follow, truly interactive exchange between programmer and computer was possible. The computer could now project itself as a dedicated partner with each individual user and the increased throughput of information between user and computer allowed the human to become a more reactive and spontaneous collaborator. Indeed, with the advent of time sharing, real human–computer interaction was now possible.

2.11.2.Video display units

As early as the mid-1950s researchers were experimenting with the possibility of presenting and manipulating information from a computer in the form of images on a video display unit (VDU). These display screens could provide a more suitable medium than a paper printout for presenting vast quantities of strategic information for rapid assimilation. The earliest applications of display screen images were developed in military applications, most notably the Semi-Automatic Ground Environment (SAGE) project of the US Air Force. It was not until 1962, however, when a young graduate student at the Massachusetts Institute of Technology (MIT), Ivan Sutherland, astonished the established computer science community with his Sketchpad program, that the capabilities of visual images were realized.

Sketchpad allowed a computer operator to use the computer to create, very rapidly, sophisticated visual models on a display screen that resembled a television set. The visual patterns could be stored in the computer's memory like any other data, and could be manipulated by the

UNIT III

DESIGNING RULES

Interaction design basic: introduction – The process of design – user focus – scenarios – navigation design – screen design and layout – iteration and prototyping – HCI in the software process: introduction – the software lifecycle – usability engineering – iterative design and prototyping – design rational – design rules: introduction – principles to support usability – standards – guidelines – golden rules and heuristics – HCI patterns.

3.1.INTERACTION DESIGN BASIC

what is design? A simple definition is:

achieving goals within constraints This does not capture everything about design, but helps to focus us on certain things:

Goals What is the purpose of the design we are intending to produce? Who is it for? Why do they want it? For example, if we are designing a wireless personal movie player, we may think about young affluent users wanting to watch the latest movies whilst on the move and download free copies, and perhaps wanting to share the experience with a few friends.

Trade-off Choosing which goals or constraints can be relaxed so that others can be met. For example, we might find that an eye-mounted video display, a bit like those used in virtual reality, would give the most stable image whilst walking along. However, this would not allow you to show friends, and might be dangerous if you were watching a gripping part of the movie as you crossed the road. Often the most exciting moments in design are when you get a radically different idea that allows you to satisfy several apparently contradictory constraints. However, the more common skill needed in design is to accept the conflict and choose the most appropriate trade-off. The temptation is to focus on one or other goal and optimize for this, then tweak the design to make it just satisfy the constraints and other goals. Instead, the best designs are where the designer understands the trade-offs and the factors affecting them. Paradoxically, if you focus on the trade-off itself the more radical solutions also become more apparent.

3.1.1 THE GOLDEN RULE OF DESIGN

Part of the understanding we need is about the circumstances and context of the particular design problem. We will return to this later in the chapter. However, there are also more generic concepts to understand. The designs we produce may be different, but often the raw materials are the same. This leads us to the golden rule of design: understand your materials.

In the case of a physical design this is obvious. Look at a chair with a steel frame and one with a wooden frame. They are very different: often the steel frames are tubular or thin L or H section steel. In contrast wooden chairs have thicker solid legs. If you made a wooden chair using the design for a metal one it would break; if you made the metal one in the design for the wooden one it would be too heavy to move.

For Human–Computer Interaction the obvious materials are the human and the computer.

understand computers

limitations, capacities, tools, platforms

understand people

psychological, social aspects, human error.

3.1.2 TO ERR IS HUMAN

It might sound demeaning to regard people as ‘materials’, possibly even dehumanizing. In fact, the opposite is the case: physical materials are treated better in most designs than people. This is particularly obvious when it comes to failures.

The news headlines: an air crash claims a hundred lives; an industrial accident causes millions of pounds’ worth of damage; the discovery of systematic mistreatment leads to thousands of patients being recalled to hospital. Some months later the public inquiries conclude: human error in the operation of technical instruments. The phrase ‘human error’ is taken to mean ‘operator error’, but more often than not the disaster is inherent in the design or installation of the human interface. Bad interfaces are slow or error-prone to use. Bad interfaces cost money and cost lives.

People make mistakes. This is not ‘human error’, an excuse to hide behind in accident reports, it is human nature. We are not infallible consistent creatures, but often make slips, errors and

omissions. A concrete lintel breaks and a building collapse. Do the headlines read ‘lintel error’? No. It is the nature of concrete lintels to break if they are put under stress and it is the responsibility of architect and engineer to ensure that a building only puts acceptable stress on the lintel. Similarly, it is the nature of humans to make mistakes, and systems should be designed to reduce the likelihood of those mistakes and to minimize the consequences when mistakes happen.

3.1.3 THE CENTRAL MESSAGE – THE USER

In this book you will find information on basic psychology, on particular technologies, on methods and models. However, there is one factor that outweighs all this knowledge. It is about attitude. Often it is said that the success of the various methods used in HCI lies not in how good they are, but in that they simply focus the mind of the designer on the user. This is the core of interaction design: put the user first, keep the user in the center and remember the user at the end.

3.2 THE PROCESS OF DESIGN

Often HCI professionals complain that they are called in too late. A system has been designed and built, and only when it proves unusable do they think to ask how to do it right! In other companies usability is seen as equivalent to testing – checking whether people can use it and fixing problems, rather than making sure they can from the beginning.

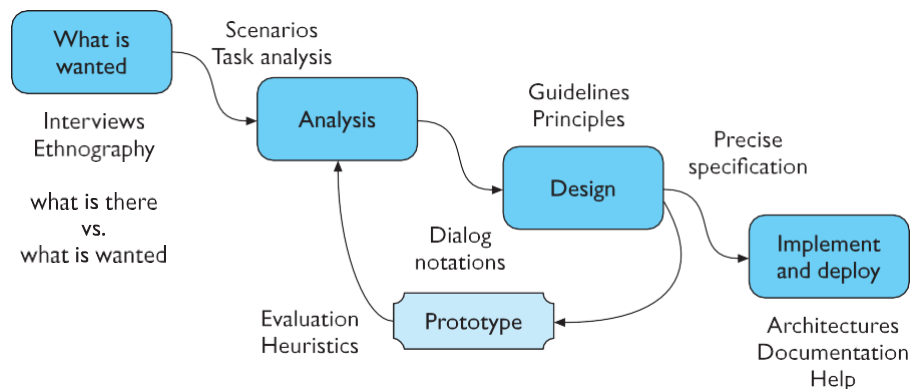


Figure 1 Interaction design process

Requirements – what is wanted the first stage is establishing what exactly is needed. As a precursor to this it is usually necessary to find out what is currently happening. For example, how do people currently watch movies? What sort of personal appliances do they currently use?

There are a number of techniques used for this in HCI: interviewing people, videotaping them, looking at the documents and objects that they work with, observing them directly. We don't have a chapter dedicated to this, but aspects will be found in various places throughout the book.

Analysis The results of observation and interview need to be ordered in some way to bring out key issues and communicate with later stages of design. we will look at scenarios, rich stories of interaction, which can be used in conjunction with a method like task analysis or on their own to record and make vivid actual interaction. These techniques can be used both to represent the situation as it is and also the desired situation.

Design Well, this is all about design, but there is a central stage when you move from what you want, to how to do it.

Iteration and prototyping Humans are complex and we cannot expect to get designs right first time. We therefore need to evaluate a design to see how well it is working and where there can be improvements.

Implementation and deployment Finally, when we are happy with our design, we need to create it and deploy it. This will involve writing code, perhaps making hardware, writing documentation and manuals – everything that goes into a real system that can be given to others.

3.3 USER FOCUS

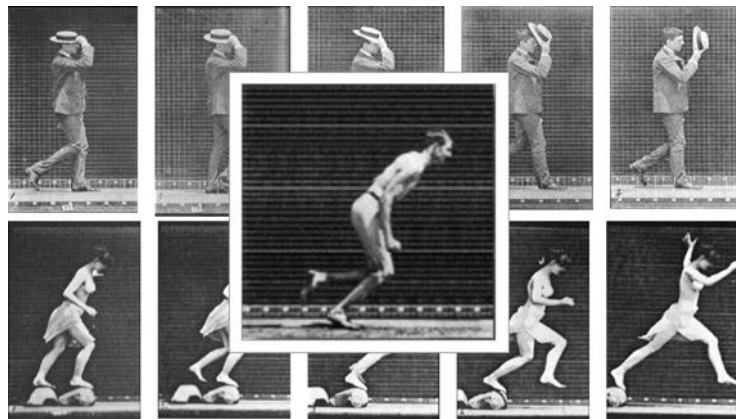


Figure 2 Eadweard Muybridge's time-lapse photography

Who are they?

Of course, the first thing to find out is who your users are. Are they young or old, experienced computer users or novices? As we saw with the stock control system, it may not be obvious who the users are, so you may need to ask this question again as you find out more about the system and its context.

Probably not like you!

When designing a system, it is easy to design it as if you were the main user: you assume your own interests and abilities. So often you hear a designer say ‘but it’s obvious what to do’. It may be obvious for her! This is not helped by the fact that many software houses are primarily filled with male developers.

Talk to them.

It is hard to get yourself inside someone else’s head, so the best thing is usually to ask them. This can take many forms: structured interviews about their job or life, open-ended discussions, or bringing the potential users fully into the design process.

Watch them.

Although what people tell you is of the utmost importance, it is not the whole story.

When black-belt judo players are asked how they throw an opponent, their explanations do not match what they actually do. Think about walking – how do your legs and arms move? It is harder than you would think! Although people have run since the earliest times, it was only with Eadweard Muybridge’s pioneering time-lapse photography in the 1870s that the way people actually walk, run and move became clear.

Use your imagination.

Even if you would like to involve many users throughout your design exercise this will not always be possible. It may be too costly, it may be hard to get time with them (e.g. hospital consultant), it may be that there are just too many (e.g. the web).

3.4 SCENARIOS

Brian would like to see the new film *Moments of Significance* and wants to invite Alison, but he knows she doesn't like 'arty' films. He decides to take a look at it to see if she would like it and so connects to one of the movie-sharing networks. He uses his work machine as it has a higher bandwidth connection, but feels a bit guilty. He knows he will be getting an illegal copy of the film, but decides it is OK as he is intending to go to the cinema to watch it. After it downloads to his machine he takes out his new personal movie player. He presses the 'menu' button and on the small LCD screen he scrolls using the arrow keys to 'bluetooth connect' and presses the 'select' button. On his computer the movie download program now has an icon showing that it has recognized a compatible device and he drags the icon of the film over the icon for the player. On the player the LCD screen says 'downloading now', with a per cent done indicator and small whirling icon.

During lunchtime Brian takes out his movie player, plugs in his earphones and starts to watch. He uses the arrow keys to skip between portions of the film and decides that, yes, Alison would like it. Then he feels a tap on his shoulder. He turns round. It is Alison. He had been so absorbed he hadn't noticed her. 'What are you watching', she says. 'Here, listen', he says and flicks a small switch. The built-in directional speaker is loud enough for both Brian and Alison to hear, but not loud enough to disturb other people in the canteen. Alison recognizes the film from trailers, 'surprised this is out yet' she says. 'Well actually...'. Brian confesses, 'you'd better come with me to see it and make an honest man of me'. 'I'll think about it', she replies.

Figure 3 Scenario for proposed movie player

Figure 3 gives an example of a scenario for the personal movie player. Like the persona it is perhaps more detailed than appears necessary, but the detail helps make the events seem real. The figure shows plain text, but scenarios can be augmented by sketches, simulated screen shots, etc. These sketches and pictures are called story-boards and are similar to the techniques used in film making to envisage plot-lines. Where the design includes physical artifacts, the scenarios can be used as a script to act out potential patterns of use. For example, we might imagine a digital Swiss army knife, which has a small LCD screen and uses the toothpick as a stylus. The knife connects to the internet via a wireless link through your phone and gives interesting tips from other Swiss army knife users.

You can see already how scenarios force you to think about the design in detail and notice potential problems before they happen. If you add more detail you can get to a blow-by-blow account of the user-system interactions and then ask 'what is the user intending now?'; 'what is the system doing now?'. This can help to verify that the design would make sense to the user and also that proposed implementation architectures would work. In addition, scenarios can be used to: Communicate

with others – other designers, clients or users. It is easy to misunderstand each other whilst discussing abstract ideas. Concrete examples of use are far easier to share. express dynamics
Individual screen shots and pictures give you a sense of what a system would look like, but not how it behaves. In the next section we will discuss ways of describing the patterns of interaction with a system. These are more complex and involve networks or hierarchies. In contrast scenarios are linear – they represent a single path amongst all the potential interactions. This linearity has both positive and negative points: Time is linear Our lives are linear as we live in time and so we find it easier to understand simple linear narratives. We are natural storytellers and story listeners.

3.5 NAVIGATION DESIGN

Table 1. levels of Interaction

PC application	Website	Physical device
Widgets	Form elements, tags and links	Buttons, dials, lights, displays
Screen design	Page design	Physical layout
Navigation design	Site structure	Main modes of device
Other apps and operating system	The web, browser, external links	The real world!

Widgets The appropriate choice of widgets and wording in menus and buttons will help you know how to use them for a particular selection or action.

Screens or windows You need to find things on the screen, understand the logical grouping of buttons.

Navigation within the application You need to be able to understand what will happen when a button is pressed, to understand where you are in the interaction.

Environment The word processor has to read documents from disk, perhaps some are on remote networks. You swap between applications, perhaps cut and paste.

Here we will consider two main kinds of issue:

local structure

looking from one screen or page out

global structure

structure of site, movement between screens.

3.6 LOCAL STRUCTURE

Much of interaction involves goal-seeking behavior. Users have some idea of what they are after and a partial model of the system. In an ideal world if users had perfect knowledge of what they wanted and how the system worked they could simply take the shortest path to what they want, pressing all the right buttons and links. However, in a world of partial knowledge users meander through the system. The important thing is not so much that they take the most efficient route, but that at each point in the interaction they can make some assessment of

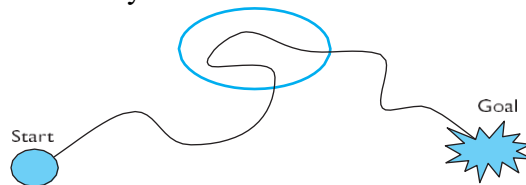


Figure 4 The personal movie player says 'downloading

whether they are getting closer to their (often partially formed) goal.

To do this goal seeking, each state of the system or each screen needs to give the user enough knowledge of what to do to get closer to their goal.

- knowing where you are
- knowing what you can do
- knowing where you are going – or what will happen
- knowing where you've been – or what you've done.

The screen, web page or device displays should make clear where you are in terms of the interaction or state of the system. Some websites show 'bread crumbs' at the top of the screen, the path of titles showing where the page is in the site (Figure 5). It is also important to know what you can do – what can be pressed or clicked to go somewhere or do something. Some web pages are particularly bad in that it is unclear.



Figure 5 Breadcrumbs. Screen shot frame reprinted by permission from Microsoft Corporation

3.7 SCREEN DESIGN AND LAYOUT

User-Centric Design:

User Personas: Understand target audience.

User Scenarios: Identify common use cases.

Hierarchy and Flow:

Information Hierarchy: Prioritize information.

User Flow: Design intuitive task flows.

Grid System:

Layout Grids: Use for alignment and spacing.

Responsive Grids: Design for different screens.

Consistency:

Design Patterns: Use consistent styles.

Visual Consistency: Cohesive visual design.

Whitespace:

Balanced Spacing: Ensure clarity.

Breathing Room: Avoid visual clutter.

Typography:

Readable Fonts: Choose legible fonts.

Hierarchy with Fonts: Use sizes and weights.

Color Scheme:

Branding Colors: Align with brand.

Contrast and Accessibility: Ensure readability.

Navigation:

Clear Paths: Intuitive navigation.

Consistent Navigation: Maintain uniformity.

Call-to-Action (CTA):

Contrasting CTAs: Make key actions stand out.

Descriptive CTAs: Clearly indicate actions.

Visual Elements:

Icons: Use for visual representation.

Images: High-quality and relevant.

Feedback and Validation:

Error Messages: Clear and concise.

User Feedback: Confirm actions.

Responsive Design:

Adapt to Screens: Ensure responsiveness.

Mobile-First: Design for mobile, scale up.

User Testing:

Prototyping: Test with prototypes.

Iterative Design: Improve based on feedback.

Accessibility:

Alt Text: Describe images.

Keyboard Navigation: Ensure accessibility.

Loading Time:

Optimized Assets: Faster loading.

Loading Feedback: Show progress.

Future Scalability:

Modular Design: Reusable components.

Scalable Systems: Plan for growth.

3.8 HCI IN THE SOFTWARE PROCESS

Within computer science there is already a large subdiscipline that addresses the management and technical issues of the development of software systems – called software engineering. One of the cornerstones of software engineering is the software life cycle, which describes the activities that take place from the initial concept formation for a software system up until its eventual phasing out and replacement. This is not intended to be a software engineering textbook, so it is not our major concern here to discuss in depth all of the issues associated with software engineering and the myriad life-cycle models.

The important point that we would like to draw out is that issues from HCI affecting the usability of interactive systems are relevant within all the activities of the software life cycle. Therefore, software engineering for interactive system design is not simply a matter of adding one more activity that slots in nicely with the existing activities in the life cycle. Rather, it involves techniques that span the entire life cycle.

3.9 THE SOFTWARE LIFE CYCLE

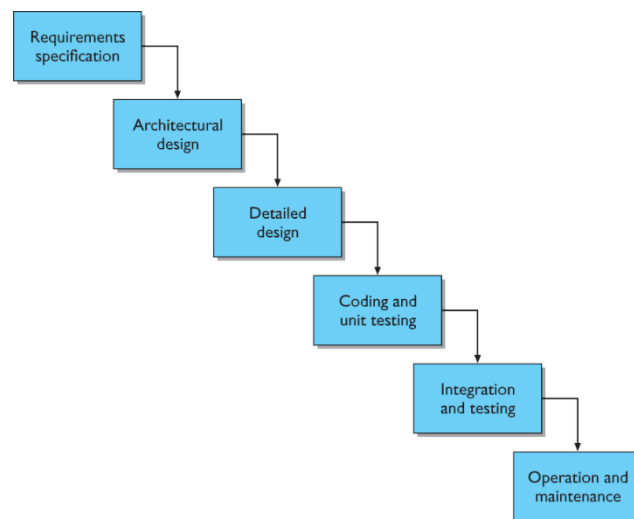


Figure 6.1. The activities in the waterfall model of the software life cycle

3.9.1 ACTIVITIES IN THE LIFE CYCLE

A more detailed description of the life cycle activities is depicted in Figure 6.1. The graphical representation is reminiscent of a waterfall, in which each activity naturally leads into the next. The analogy of the waterfall is not completely faithful to the real relationship between these activities, but it provides a good starting point for discussing the logical flow of activity. We describe the activities of this waterfall model of the software life cycle next.

Requirements specification

In requirements specification, the designer and customer try to capture a description of what the eventual system will be expected to provide. This is in contrast to determining how the system will provide the expected services, which is the concern of later activities. Requirements specification involves eliciting information from the customer about the work environment, or domain, in which the final product will function. Aspects of the work domain include not only the particular functions that the software product must perform but also details about the environment in which it must operate, such as the people whom it will potentially affect and the new product's relationship to any other products which it is updating or replacing.

Architectural design

As we mentioned, the requirements specification concentrates on what the system is supposed to do. The next activities concentrate on how the system provides the services expected from it. The first activity is a high-level decomposition of the system into components that can either be brought in from existing software products or be developed from scratch independently. An architectural design performs this decomposition. It is not only concerned with the functional decomposition of the system, determining which components provide which services. It must also describe the interdependencies between separate components and the sharing of resources that will arise between components.

There are many structured techniques that are used to assist a designer in deriving an architectural description from information in the requirements specification (such as CORE, MASCOT and HOOD). Details of these techniques are outside the scope of this book, but can be found in any good software engineering textbook. What we will mention here is that the majority of these techniques are adequate for capturing the functional requirements of the system – the services the

system must provide in the work domain – but do not provide an immediate way to capture other non-functional requirements – features of the system that are not directly related to the actual services provided but relate to the manner in which those services must be provided. Some classic examples of non-functional requirements are the efficiency, reliability, timing and safety features of the system.

Detailed design

The architectural design provides a decomposition of the system description that allows for isolated development of separate components which will later be integrated. For those components that are not already available for immediate integration, the designer must provide a sufficiently detailed description so that they may be implemented in some programming language. The detailed design is a refinement of the component description provided by the architectural design. The behavior implied by the higher-level description must be preserved in the more detailed description.

Typically, there will be more than one possible refinement of the architectural component that will satisfy the behavioral constraints. Choosing the best refinement is often a matter of trying to satisfy as many of the non-functional requirements of the system as possible. Thus the language used for the detailed design must allow some analysis of the design in order to assess its properties.

Coding and unit testing

The detailed design for a component of the system should be in such a form that it is possible to implement it in some executable programming language. After coding, the component can be tested to verify that it performs correctly, according to some test criteria that were determined in earlier activities. Research on this activity within the life cycle has concentrated on two areas. There is plenty of research that is geared towards the automation of this coding activity directly from a low-level detailed design. Most of the work in formal methods operates under the hypothesis that, in theory, the transformation from the detailed design to the implementation is from one mathematical representation to another and so should be able to be entirely automated. Other, more practical work concentrates on the automatic generation of tests from output of earlier activities which can be performed on a piece of code to verify that it behaves correctly.

A proof can be entirely justified by the rules of the mathematical language, in which case it is considered a formal proof. More common is a rigorous proof, which is represented within some mathematical language but which relies on the understanding of the reader to accept its correctness without appeal to the full details of the argument, which could be provided but usually are not. The difference between formality and rigor is in the amount of detail the prover leaves out while still maintaining acceptance of the proof.

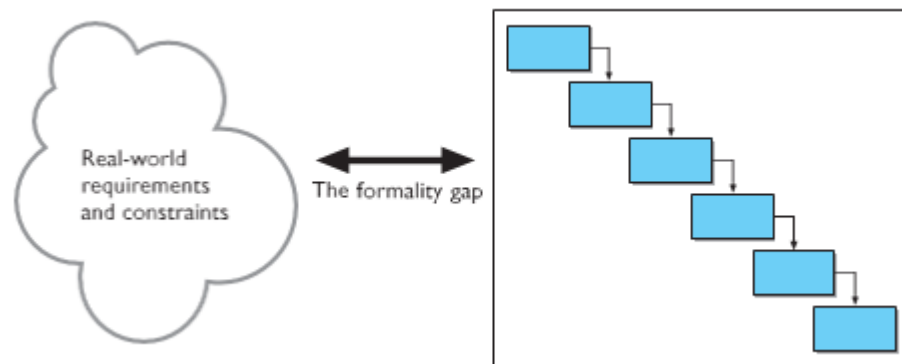


Figure 8. The formality gap between the real world and structured design

Proofs that are for verification of a design can frequently occur within one language or between two languages which both have a precise mathematical semantics. Time constraints for a design project and the perceived economic implications of the separate components usually dictate which proofs are carried out in full formality and which are done only rigorously (if at all). As research in this area matures and automated tools provide assistance for the mechanical aspects of proof, the cost of proof should decrease.

Validation proofs are much trickier, as they almost always involve a transformation between languages. Furthermore, the origin of customer requirements arises in the inherent ambiguity of the real world and not the mathematical world. This precludes the possibility of objective proof, rigorous or formal. Instead, there will always be a leap from the informal situations of the real world to any formal and structured development process. The formality gap means that validation will always rely to some extent on subjective means of proof.

3.10 USABILITY ENGINEERING

The ultimate test of a product's usability is based on measurements of users' experience with it. Therefore, since a user's direct experience with an interactive system is at the physical interface, focus on the actual user interface is understandable. The danger with this limited focus is that much of the work that is accomplished in interaction involves more than just the surface features of the systems used to perform that work. In reality, the whole functional architecture of the system and the cognitive capacity of the users should be observed in order to arrive at meaningful measures. But it is not at all simple to derive measurements of activity beyond the physical actions in the world, and so usability engineering is limited in its application.

In relation to the software life cycle, one of the important features of usability engineering is the inclusion of a usability specification, forming part of the requirements specification, that concentrates on features of the user-system interaction which contribute to the usability of the product. Various attributes of the system are suggested as gauges for testing the usability. For each attribute, six items are defined to form the usability specification of that attribute. Table 2 provides an example of a usability specification for the design of a control panel for a video cassette recorder (VCR), based on the technique presented by Whiteside, Bennett and Holtzblatt.

Table 2. Sample usability specification for undo with a VCR

Attribute:	Backward recoverability
Measuring concept:	Undo an erroneous programming sequence
Measuring method:	Number of explicit user actions to undo current program
Now level:	No current product allows such an undo
Worst case:	As many actions as it takes to program in mistake
Planned level:	A maximum of two explicit user actions
Best case:	One explicit cancel action

The recovery procedure can be in either a backward or forward sense. Current VCR design has resulted in interactive systems that are notoriously difficult to use; the redesign of a VCR provides a good case study for usability engineering. In designing a new VCR control panel, the designer wants to take into account how a user might recover from a mistake he discovers while trying to program the VCR to record some television program in his absence. One approach that the designer decides to follow is to allow the user the ability to undo the programming sequence, reverting the state of the VCR to what it was before the programming task began.

The backward recoverability attribute is defined in terms of a measuring concept, which makes the abstract attribute more concrete by describing it in terms of the actual product. So, in this case, we realize backward recoverability as the ability to undo an erroneous programming sequence. The measuring method states how the attribute will be measured, in this case by the number of explicit user actions required to perform the undo, regardless of where the user is in the programming sequence.

Table 3 Criteria by which measuring method can be determined

1.	Time to complete a task
2.	Per cent of task completed
3.	Per cent of task completed per unit time
4.	Ratio of successes to failures
5.	Time spent in errors
6.	Per cent or number of errors
7.	Per cent or number of competitors better than it
8.	Number of commands used
9.	Frequency of help and documentation use
10.	Per cent of favorable/unfavorable user comments
11.	Number of repetitions of failed commands
12.	Number of runs of successes and of failures
13.	Number of times interface misleads the user
14.	Number of good and bad features recalled by users
15.	Number of available commands not invoked
16.	Number of regressive behaviors
17.	Number of users preferring your system
18.	Number of times users need to work around a problem
19.	Number of times the user is disrupted from a work task
20.	Number of times user loses control of the system
21.	Number of times user expresses frustration or satisfaction

Determining the worst-case value depends on a number of things. Usually, it should be no lower than the now level. The new product should provide some improvement on the current state of affairs, and so it seems that at least some of the usability attributes should provide worst case values that are better than the now level. Otherwise, why would the customer bother with the new system (unless it can be shown to provide the same usability at a fraction of the cost)? The designers in the example have determined that the minimal acceptable undo facility would require the user to perform as many actions as he had done to program in the mistake. This is a clear improvement over the now level, since it at least provides for the possibility of undo. One way to provide such a capability would be by including an undo button on the control panel, which would effectively reverse the previous non-undo action. The designer's figure that they should allow for the user to do a complete restoration of the VCR state in a maximum of two explicit user actions, though they

recognize that the best case, at least in terms of the number of explicit actions, would require only one.

Tables 2 and 3, adapted from Whiteside, Bennett and Holtzblatt, provide a list of measurement criteria which can be used to determine the measuring method for a usability attribute and the possible ways to set the worst/best case and planned/ now level targets. Measurements such as those promoted by usability engineering are also called usability metrics.

Table 4 Examples of usability metrics from ISO 9241

Usability objective	Effectiveness measures	Efficiency measures	Satisfaction measures
Suitability for the task	Percentage of goals achieved	Time to complete a task	Rating scale for satisfaction
Appropriate for trained users	Number of power features used	Relative efficiency compared with an expert user	Rating scale for satisfaction with power features
Learnability	Percentage of functions learned	Time to learn criterion	Rating scale for ease of learning
Error tolerance	Percentage of errors corrected successfully	Time spent on correcting errors	Rating scale for error handling

The ISO standard 9241, described earlier, also recommends the use of usability specifications as a means of requirements specification. Table 4 gives examples of usability metrics categorized by their contribution towards the three categories of usability: effectiveness, efficiency and satisfaction.

3.10.1 PROBLEMS WITH USABILITY ENGINEERING

The major feature of usability engineering is the assertion of explicit usability metrics early on in the design process which can be used to judge a system once it is delivered. There is a very solid argument which points out that it is only through empirical approaches such as the use of usability metrics that we can reliably build

more usable systems. Although the ultimate yardstick for determining usability may be by observing and measuring user performance, that does not mean that these measurements are the best way to produce a predictive design process for usability.

The problem with usability metrics is that they rely on measurements of very specific user actions in very specific situations. When the designer knows what the actions and situation will be, then she can set goals for measured observations. However, at early stages of design, designers do not have this information. Take our example usability specification for the VCR. In setting the acceptable and unacceptable levels for backward recovery, there is an assumption that a button will be available to invoke the undo. In fact, the designer was already making an implicit assumption that the user would be making errors in the programming of the VCR. Why not address the origin of the programming errors, then maybe undo would not be necessary? We should recognize another inherent limitation for usability engineering, that is it provides a means of satisfying usability specifications and not necessarily usability. The designer is still forced to understand why a particular usability metric enhances usability for real people. Again, in the VCR example, the designer assumed that fewer explicit actions make the undo operation easier. Is that kind of assumption warranted?

3.11 ITERATIVE DESIGN AND PROTOTYPING

A point we raised earlier is that requirements for an interactive system cannot be completely specified from the beginning of the life cycle. The only way to be sure about some features of the potential design is to build them and test them out on real users. The design can then be modified to correct any false assumptions that were revealed in the testing. This is the essence of iterative design, a purposeful design process which tries to overcome the inherent problems of incomplete requirements specification by cycling through several designs, incrementally improving upon the final product with each pass.

The problems with the design process, which lead to an iterative design philosophy, are not unique to the usability features of the intended system. The problem holds for requirements specification in general, and so it is a general software engineering problem, together with technical and managerial issues.

On the technical side, iterative design is described by the use of prototypes, artifacts that simulate or animate some but not all features of the intended system. There are three main approaches to prototyping:

Throw-away the prototype is built and tested. The design knowledge gained from this exercise is used to build the final product, but the actual prototype is discarded. Figure 6.5 depicts the procedure in using throw-away prototypes to arrive at a final requirements specification in order for the rest of the design process to proceed.

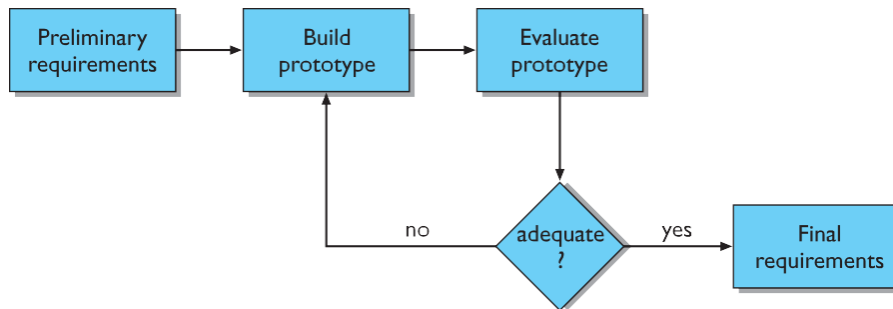


Figure 9 Throw-away prototyping within requirements specification

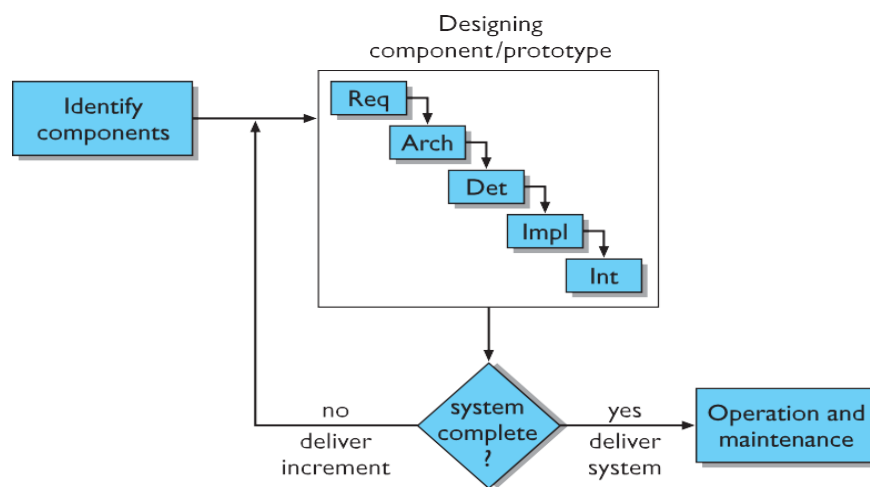


Figure 10. Incremental prototyping within the life cycle

Incremental The final product is built as separate components, one at a time. There is one overall design for the final system, but it is partitioned into independent and smaller components. The final product is then released as a series of products, each subsequent release including one more component.

Evolutionary Here the prototype is not discarded and serves as the basis for the next iteration of design. In this case, the actual system is seen as evolving from a very limited initial version to its final release. Evolutionary prototyping also fits in well with the modifications which must be made to the system that arise during the operation and maintenance activity in the life cycle.

programmer would have to know quite a bit of intimate detail of the hardware system in order to control even the simplest of interactive behavior. These high-level programming languages allow the programmer to abstract away from the hardware specifics and think in terms that are closer to the way the input and output devices are perceived as interaction devices.

3.11.3 WARNING ABOUT ITERATIVE DESIGN

Though we have presented the process of iterative design as not only beneficial but also necessary for good interactive system design, it is important to recognize some of its drawbacks, in addition to the very real management issues we have already raised. The ideal model of iterative design, in which a rapid prototype is designed, evaluated and modified until the best possible design is achieved in the given project time, is appealing. But there are two problems. That poses no problem, as long as both the designer and the ultimate users of the clock both share the knowledge of the discrepancy between possible clock displays and legal times. Such would not be the case for someone assuming a 24-hour time format, in which case the displays 00:30 and 13:45 would represent valid times in their model but not in the microwave's model. In this particular example, the subjects tested during the evaluation must have all shared the 12-hour time model, and the mismatch with the other users (with a 24-hour model) was only discovered after the product was being shipped. At this point, the only impact of iterative design was a change to the documentation alerting the reader to the 12-hour format, as it was too late to perform any hardware change.

The second problem is slightly more subtle, and serious. If, in the process of evaluation, a potential usability problem is diagnosed, it is important to understand the reason for the problem and not just detect the symptom. In the clock example, the designers could have noticed that some subjects with a 24-hour time model were having difficulty setting the time. Say they were trying to set the time for 14:45, but they were not being allowed to do that. If the designers did not know the subject's goals, they might not detect the 24/12 hour discrepancy. They would instead notice that the users were having trouble setting the time and so they might change the buttons used to set the time instead of other possible changes, such as an analog time dial, or displaying AM or PM on the clock dial to make the 12-hour model more obvious, or to change to a 24-hour clock.

3.12 DESIGN RATIONALE

In designing any computer system, many decisions are made as the product goes from a set of vague customer requirements to a deliverable entity. Often it is difficult to recreate the reasons, or rationale, behind various design decisions. Design rationale is the information that explains why a computer system is the way it is, including its structural or architectural description and its functional or behavioral description. In this sense, design rationale does not fit squarely into the software life cycle described in this chapter as just another phase or box. Rather, design rationale relates to an activity of both reflection (doing design rationale) and documentation (creating a design rationale) that occurs throughout the entire life cycle.

It is beneficial to have access to the design rationale for several reasons:

In an explicit form, a design rationale provides a communication mechanism among the members of a design team so that during later stages of design and/or maintenance it is possible to understand what critical decisions were made, what alternatives were investigated (and, possibly, in what order) and the reason why one alternative was chosen over the others. This can help avoid incorrect assumptions later.

Accumulated knowledge in the form of design rationales for a set of products can be reused to transfer what has worked in one situation to another situation which has similar needs. The design rationale can capture the context of a design decision in order that a different design team can determine if a similar rationale is appropriate for their product.

The effort required to produce a design rationale forces the designer to deliberate more carefully about design decisions. The process of deliberation can be assisted by the design rationale technique by suggesting how arguments justifying or discarding a particular design option are formed.

In the area of HCI, design rationale has been particularly important, again for several reasons:

There is usually no single best design alternative. More often, the designer is faced with a set of trade-offs between different alternatives. For example, a graphical interface may involve a set of actions that the user can invoke by use of the mouse and the designer must decide whether to present each action as a 'button' on the screen, which is always visible, or hide all of the actions in a menu which must be explicitly invoked before an action can be chosen. The former option

maximizes the operation visibility .But, the latter option takes up less screen space. It would be up to the designer to determine which criterion for evaluating the options was more important and then communicating that information in a design rationale.

Even if an optimal solution did exist for a given design decision, the space of alternatives is so vast that it is unlikely a designer would discover it. In this case, it is important that the designer indicates all alternatives that have been investigated. Then later on it can be determined if she has not considered the best solution or had thought about it and discarded it for some reason. In project management, this kind of accountability for design is good.

The usability of an interactive system is very dependent on the context of its use. The flashiest graphical interface is of no use if the end-user does not have access to a high-quality graphics display or a pointing device. Capturing the context in which a design decision is made will help later when new products are designed. If the context remains the same, then the old rationale can be adopted without revision. If the context has changed somehow, the old rationale can be re-examined to see if any rejected alternatives are now more favorable or if any new alternatives are now possible.

The structure-oriented approach does not capture historical information. Instead, it captures the complete story of the moment, as an analysis of the design space which has been considered so far. The final category of design rationale concentrates on capturing the claims about the psychology of the user that are implied by an interactive system and the tasks that are performed on them.

There are some issues that distinguish the various techniques in terms of their usability within design itself. We can use these issues to sketch an informal rationale for design rationale. One issue is the degree to which the technique impinges on the design process. Does the use of a particular design rationale technique alter the decision process, or does it just passively serve to document it? Another issue is the cost of using the technique, both in terms of creating the design rationale and in terms of accessing it once created. A related issue is the amount of computational power the design rationale provides and the level to which this is supported by automated tools. A design rationale for a complex system can be very large and the exploration of the design space changes over time. The kind of information stored in a given design rationale will affect how that vast amount of information can be effectively managed and browsed.

3.12.3 PSYCHOLOGICAL DESIGN RATIONALE

The final category of design rationale tries to make explicit the psychological claims of usability inherent in any interactive system in order better to suit a product for the tasks users have. This psychological design rationale has been introduced by Carroll and Rosson [62], and before we describe the application of the technique it is important to understand some of its theoretical background.

People use computers to accomplish some tasks in their particular work domain, as we have seen before. When designing a new interactive system, the designers take into account the tasks that users currently perform and any new ones that they may want to perform. When the new system is implemented, or becomes an artifact, further observation reveals that in addition to the required tasks it was built to support, it also supports users in tasks that the designer never intended. Once designers understand these new tasks, and the associated problems that arise between them and the previously known tasks, the new task definitions can serve as requirements for future artifacts.

Carroll refers to this real-life phenomenon as the task–artifact cycle. He provides a good example of this cycle through the evolution of the electronic spreadsheet. When the first electronic spreadsheet, VisiCalc, was marketed in the late 1970s, it was presented simply as an automated means of supporting tabular calculation, a task commonly used in the accounting world. Within little over a decade of its introduction, the application of spreadsheets had far outstripped its original intent within accounting. Spreadsheets were being used for all kinds of financial analysis, ‘what-if’ simulations, report formatting and even as a general programming language paradigm! As the set of tasks expands, new spreadsheet products have flooded the marketplace trying to satisfy the growing customer base. Another good example of the task–artifact cycle in action is with word processing, which was originally introduced to provide more automated support for tasks previously achieved with a typewriter and now provides users with the ability to carry out various authoring tasks that they never dreamed possible with a conventional typewriter. And today, the tasks for the spreadsheet and the word processor are intermingled in the same artifact.

The first step in the psychological design rationale is to identify the tasks that the proposed system will address and to characterize those tasks by questions that the user tries to answer in accomplishing them. For instance, Carroll gives an example of designing a system to help programmers learn the Smalltalk object-oriented programming language environment. The main

task the system is to support is learning how Smalltalk works. In learning about the programming environment, the programmer will perform tasks that help her answer the questions:

What can I do: that is, what are the possible operations or functions that this programming environment allows?

How does it work: that is, what do the various functions do?

n How can I do this: that is, once I know a particular operation I want to perform, how do I go about programming it?

For each question, a set of scenarios of user–system behavior is suggested to support the user in addressing the question. For example, to address the question ‘What can I do?’, the designers can describe a scenario whereby the novice programmer is first confronted with the learning environment and sees that she can invoke some demo programs to investigate how Smalltalk programs work. Once this system is running, observation of its use and some designer reflection is used to produce the actual psychological design rationale for that version of the system. The psychological claim of this demo system is that the user learns by doing, which is a good thing. However, there may also be negative aspects that are equally important to mention. The demo may not be very interactive, in which case the user clicks on it to initiate it and then just sits back and watches a graphic display, never really learning how the demo application is constructed in Smalltalk. These negative aspects can be used to modify later versions of the system to allow more interactive demos, which represent realistic, yet simple, applications, whose behavior and structure the programmer can appreciate.

By forcing the designer to document the psychological design rationale, it is hoped that she will become more aware of the natural evolution of user tasks and the artifact, taking advantage of how consequences of one design can be used to improve later designs.

3.13 DESIGN RULES

Designing for maximum usability is the goal of interactive systems design.

* Abstract principles offer a way of understanding usability in a more general sense, especially if we can express them within some coherent catalog.

- * Design rules in the form of standards and guidelines provide direction for design, in both general and more concrete terms, in order to enhance the interactive properties of the system.
- * The essential characteristics of good design are often summarized through ‘golden rules’ or heuristics.
- * Design patterns provide a potentially generative approach to capturing and reusing design knowledge

3.13.1 PRINCIPLES TO SUPPORT USABILITY

The most abstract design rules are general principles, which can be applied to the design of an interactive system in order to promote its usability. Derivation of principles for interaction has usually arisen out of a need to explain why a paradigm is successful and when it might not be. Principles can provide the repeatability which paradigms in themselves cannot provide. In this section we present a collection of usability principles. Since it is too bold an objective to produce a comprehensive catalog of such principles, our emphasis will be on structuring the presentation of usability principles in such a way that the catalog can be easily extended as our knowledge increases.

The principles we present are first divided into three main categories:

Learnability – the ease with which new users can begin effective interaction and achieve maximal performance.

Flexibility – the multiplicity of ways in which the user and system exchange information.

Robustness – the level of support provided to the user in determining successful achievement and assessment of goals.

In the following, we will subdivide these main categories into more specific principles that support them. In most cases, we are able to situate these more specific principles within a single category, but we have made explicit those cases when a principle falls into two of the above categories.

you have received a new message. If you do not attend to that message immediately, you may forget about it. If, however, some persistent visual information informs you of the incoming message (say, the flag goes up on your electronic mailbox), then that will serve as a reminder that an unread message remains long after its initial receipt.

3.13.16 RECOVERABILITY

Users make mistakes from which they want to recover. Recoverability is the ability to reach a desired goal after recognition of some error in a previous interaction. There are two directions in which recovery can occur, forward or backward. Forward error recovery involves the acceptance of the current state and negotiation from that state towards the desired state. Forward error recovery may be the only possibility for recovery if the effects of interaction are not revocable (for example, in building a house of cards, you might sneeze whilst placing a card on the seventh level, but you cannot undo the effect of your misfortune except by rebuilding). Backward error recovery is an attempt to undo the effects of previous interaction in order to return to a prior state before proceeding. In a text editor, a mistyped keystroke might wipe out a large section of text which you would want to retrieve by an equally simple undo button. Recovery can be initiated by the system or by the user. When performed by the system, recoverability is connected to the notions of fault tolerance, safety, reliability and dependability, all topics covered in software engineering. However, in software engineering this recoverability is considered only with respect to system functionality; it is not tied to user intent.

3.13.17 RESPONSIVENESS

Responsiveness measures the rate of communication between the system and the user. Response time is generally defined as the duration of time needed by the system to express state changes to the user. In general, short durations and instantaneous response times are desirable. Instantaneous means that the user perceives system reactions as immediate. But even in situations in which an instantaneous response cannot be obtained, there must be some indication to the user that the system has received the request for action and is working on a response.

As significant as absolute response time is response time stability. Response time stability covers the invariance of the duration for identical or similar computational resources. For example, pull-

down menus are expected to pop up instantaneously as soon as a mouse button is pressed. Variations in response time will impede anticipation exploited by motor skill.

3.13.18 TASK CONFORMANCE

Since the purpose of an interactive system is to allow a user to perform various tasks in achieving certain goals within a specific application domain, we can ask whether the system supports all of the tasks of interest and whether it supports these as the user wants. Task completeness addresses the coverage issue and task adequacy addresses the user's understanding of the tasks.

It is not sufficient that the computer system fully implements some set of computational services that were identified at early specification stages. It is essential that the system allows the user to achieve any of the desired tasks in a particular work domain as identified by a task analysis that precedes system specification. Task completeness refers to the level to which the system services can be mapped onto all of the user tasks. However, it is quite possible that the provision of a new computer-based tool will suggest to a user some tasks that were not even conceivable before the tool. Therefore, it is also desirable that the system services be suitably general so that the user can define new tasks.

3.14 STANDARDS

Standards for interactive system design are usually set by national or international bodies to ensure compliance with a set of design rules by a large community. Standards can apply specifically to either the hardware or the software used to build the interactive system. Smith points out the differing characteristics between hardware and software, which affect the utility of design standards applied to them:

Underlying theory Standards for hardware are based on an understanding of physiology or ergonomics/human factors, the results of which are relatively well known, fixed and readily adaptable to design of the hardware. On the other hand, software standards are based on theories from psychology or cognitive science, which are less well formed, still evolving and not very easy to interpret in the language of software design. Consequently, standards for hardware can directly relate to a hardware specification and still reflect the underlying theory, whereas software standards would have to be more vaguely worded.

Change Hardware is more difficult and expensive to change than software, which is usually designed to be very flexible. Consequently, requirements change for hardware do not occur as frequently as for software. Since standards are also relatively stable, they are more suitable for hardware than software.

Historically, for these reasons, a given standards institution, such as the British Standards Institution (BSI) or the International Organization for Standardization (ISO) or a national military agency, has had standards for hardware in place before any for software. For example, the UK Ministry of Defense has published an Interim Defense Standard 00–25 on Human Factors for Designers of Equipment, produced in 12 parts:

Part 1 Introduction Part 2 Body Size

Part 3 Body Strength and Stamina Part 4 Workplace Design

Part 5 Stresses and Hazards Part 6 Vision and Lighting

Part 7 Visual Displays

Part 8 Auditory Information Part 9 Voice Communication Part 10 Controls

Part 11 Design for Maintainability Part 12 Systems

Only the last of these is concerned with the software design process. The international standard ISO 9241, entitled Ergonomic Requirements for Office Work with Visual Display Terminals (VDT)s, has 17 parts. Seven of these are concerned with hardware issues – requirements for visual display, keyboard layout, workstation layout, environment, display with reflections, display colors and non-keyboard input devices. Seven parts are devoted to software issues – general dialog principles, menu dialogs, presentation of information, user guidance, command dialogs, direct manipulation dialogs and form-filling dialogs. However, standards covering software issues are now being produced, for example, the draft standard ISO 14915 covers software ergonomics for multimedia user interfaces.

Usability The effectiveness, efficiency and satisfaction with which specified users achieve specified goals in particular environments.

Effectiveness The accuracy and completeness with which specified users can achieve specified goals in particular environments.

Efficiency The resources expended in relation to the accuracy and completeness of goals achieved.

Satisfaction The comfort and acceptability of the work system to its users and other people affected by its use.

3.15 GUIDELINES

Several books and technical reports contain huge catalogs of guidelines. A classic example was a very general list compiled by Smith and Mosier in 1986 at the Mitre Corporation and sponsored by the Electronic Systems Division of the US Air Force [325]. The basic categories of the Smith and Mosier guidelines are:

1. Data Entry
2. Data Display
3. Sequence Control
4. User Guidance
5. Data Transmission
6. Data Protection

Each of these categories is further broken down into more specific subcategories which contain the particular guidelines. A major concern for all of the general guidelines is the subject of dialog styles, which in the context of these guidelines pertains to the means by which the user communicates input to the system, including how the system presents the communication device. Most guidelines are applicable for the implementation of any one of these dialog styles in isolation. It is also important to consider the possibility of mixing dialog styles in one application. This equates to a mixing of dialog styles – a direct manipulation dialog being suitable for the action paradigm and a command language being suitable for the language paradigm. Mayhew provides guidelines and a technique for deciding how to mix dialog styles. For, the sake of consistency, this ordering guideline is to be followed for all operation invocation involving the explicit and separate indication of an operation and the object or arguments of that operation.

Another less straightforward example from the Apple guidelines refers to user control:

The user, not the computer, initiates and controls all actions.

Programming in the style of these GUIs involves the use of toolkits which provide high-level widgets, as we have mentioned earlier in this book and will discuss in more detail in Chapter 8. More importantly, each of these GUIs has its own look and feel, which describes their expected behavior. The style guides are intended to help a programmer capture the elements of the look and feel of a GUI in her own programming. Therefore, style guides for the look and feel of a GUI promote the consistency within and between applications on the same computer platform.

3.16 GOLDEN RULES AND HEURISTICS

So far we have considered a range of abstract principles and detailed guidelines, which can be used to help designers produce more usable systems. But all of these rules require a certain amount of commitment on the part of the designer, either to track down appropriate guidelines or to interpret principles. Is there a simpler way? A number of advocates of user-centered design have presented sets of ‘golden rules’ or heuristics. While these are inevitably ‘broad-brush’ design rules, which may not be always be applicable to every situation, they do provide a useful checklist or summary of the essence of design advice. It is clear that any designer following even these simple rules will produce a better system than one who ignores them. There are many sets of heuristics, but the most well used are Nielsen’s ten heuristics, Shneiderman’s eight golden rules and Norman’s seven principles.

3.16.1 SHNEIDERMAN’S EIGHT GOLDEN RULES OF INTERFACE DESIGN

Shneiderman’s eight golden rules provide a convenient and succinct summary of the key principles of interface design. They are intended to be used during design but can also be applied, like Nielsen’s heuristics, to the evaluation of systems. Notice how they relate to the abstract principles discussed earlier.

1. Strive for consistency in action sequences, layout, terminology, command use and so on.
2. Enable frequent users to use shortcuts, such as abbreviations, special key sequences and macros, to perform regular, familiar actions more quickly.

3. Offer informative feedback for every user action, at a level appropriate to the magnitude of the action.
4. Design dialogs to yield closure so that the user knows when they have completed a task.
5. Offer error prevention and simple error handling so that, ideally, users are prevented from making mistakes and, if they do, they are offered clear and informative instructions to enable them to recover.
6. Permit easy reversal of actions in order to relieve anxiety and encourage exploration, since the user knows that he can always return to the previous state.
7. Support internal locus of control so that the user is in control of the system, which responds to his actions.
8. Reduce short-term memory load by keeping displays simple, consolidating multiple page displays and providing time for learning action sequences.

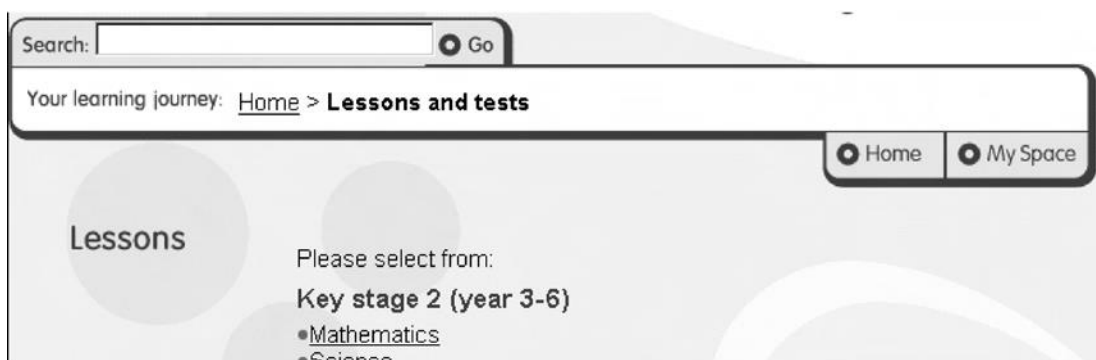
3.16.2 NORMAN'S SEVEN PRINCIPLES FOR TRANSFORMING DIFFICULT TASKS INTO SIMPLE ONES

1. Use both knowledge in the world and knowledge in the head. People work better when the knowledge they need to do a task is available externally – either explicitly or through the constraints imposed by the environment. But experts also need to be able to internalize regular tasks to increase their efficiency. So, systems should provide the necessary knowledge within the environment and their operation should be transparent to support the user in building an appropriate mental model of what is going on.
2. Simplify the structure of tasks. Tasks need to be simple in order to avoid complex problem solving and excessive memory load. There are a number of ways to simplify the structure of tasks. One is to provide mental aids to help the user keep track of stages in a more complex task. Another is to use technology to provide the user with more information about the task and better feedback. A third approach is to automate the task or part of it, as long as this does not detract from the user's experience. The final approach to simplification is to change the nature of the task so that it becomes something simpler. In all of this, it is important not to take control away from the user.

3. Make things visible: bridge the gulfs of execution and evaluation. The interface should make clear what the system can do and how this is achieved, and should enable the user to see clearly the effect of their actions on the system.
4. Get the mappings right. User intentions should map clearly onto system controls. User actions should map clearly onto system events. So, it should be clear what does what and by how much. Controls, sliders and dials should reflect the task – so a small movement has a small effect and a large movement a large effect.
5. Exploit the power of constraints, both natural and artificial. Constraints are things in the world that make it impossible to do anything but the correct action in the correct way. A simple example is a jigsaw puzzle, where the pieces only fit together in one way. Here the physical constraints of the design guide the user to complete the task.
6. Design for error. To err is human, so anticipate the errors the user could make and design recovery into the system.
7. When all else fails, standardize. If there are no natural mappings then arbitrary mappings should be standardized so that users only have to learn them once.

3.17 HCI Patterns

... NAVIGABLE SPACES or STEP BY STEP and a CONTROL PANEL are in place, which require the user to be able to move through the steps page by page.



From (www.learn.co.uk). An example of a toolbar that provides the option to return home and keeps track of the learning journey, providing the option to link back to a safe place.

❖❖❖

It is easy to get lost in the tangle of links in a website.

People don't use the web like a TV or magazine. They use the web to find what they are looking for and then stop. They may select a wrong link and not be able to find their way back to something relevant. They do not always keep track of where they've been. They may forget where they have been if they are interrupted when using the site.

You are more likely to explore a website if you are sure that you can easily get out of an undesired state or space; that assurance engenders a feeling of security. Backtracking out of a long navigation path can be very tedious.

Therefore:

Always include a way back to a place that acts as a 'vantage point' from where you can reorientate yourself.



This pattern can be used in conjunction with GO BACK ONE STEP and CONTINUE TO NEXT STEP.

Figure 7.3 An example pattern 'go back to a safe place' adapted from Tidwell's Common Ground collection. Courtesy of Jenifer Tidwell

The pattern also has references to other patterns, indicating both the context in which it can be applied (the top references) and the patterns that may be needed to complete it (the bottom references). This connects the patterns together into a language. Patterns in isolation have limited use, but by traversing the hierarchy, through these references, the user is assisted in generating a complete design.

Patterns and pattern languages are characterized by a number of features, which, taken as a whole, distinguish them from other design rules:

They capture design practice and embody knowledge about successful solutions: they come from practice rather than psychological theory.

They capture the essential common properties of good design: they do not tell the designer how to do something but what needs to be done and why.

They represent design knowledge at varying levels, ranging from social and organizational issues through conceptual design to detailed widget design.

They are not neutral but embody values within their rationale. Alexander's language clearly expresses his values about architecture. HCI patterns can express values about what is humane in interface design.

The concept of a pattern language is generative and can therefore assist in the development of complete designs.

They are generally intuitive and readable and can therefore be used for communication between all stakeholders.

Patterns are a relatively recent addition to HCI representations, in which there are still many research issues to resolve. For instance, it is not clear how patterns can best be identified or how languages should be structured to reflect the temporal concerns of interaction.

UNIT - 3

MODELS AND THEORIES

Cognitive models –Socio-Organizational issues and stake holder requirements –
Communication and collaboration models-Hypertext, Multimedia and WWW.

3.1. COGNITIVE MODELS

3.1.1. Cognitive Science

- Cognitive Science is the science of mind and behavior which is different from psychology.
- Cognitive science used to understand the knowledge acquisition and us is key to understanding the mind.
- So, **Cognitive science**- is a **scientific and interdisciplinary study of the mind with special emphasis** on the use and acquisition of knowledge and information.
- It includes **Artificial Intelligence, Psychology, Linguistics, Philosophy, Anthropology, Neuroscience and Education.**
- It implies both **interdisciplinary approach** (i.e. many scientific disciplines contribute to cognitive science) and a **computational approach** which explains information processing in terms of neural computations.
- Cognitive science grew out of the following three developments:
 - The invention of computer and design the program that could do the kind of tasks that humans do.
 - The development of information processing. The main goal of information processing is to specify the internal processing which involves perception, language, memory and thought.
 - The development of the theory of generative grammar and related branches in linguistics.
- Cognitive science contains five major areas: **Knowledge representation, language, learning, thinking, and perception.**
- Cognitive model bound from cognitive science.

- **Cognitions:** derived from latin base cognitio – i.e., “know together” which means the collection of mental processes and activities used in perceiving ,learning, remebering, thinking, and understanding, and the act of those processes.

3.1.2 Why congntive Sciecene is interdisciplinary?

To process the information, human information processor has to use the following steps:

- To acquire real time information about the surrounding environment using **perception** or recpetors.
- With the usage of some **languages**, making the use of information about syntax, semantics and phonology.
- To combine diferent sources of information and derive new information and test its consistency using **reasoning**.
- To make use of information in **action** planning and guidance
- To store and retreive the information using **memeory**

Hence, cognitive science is inter- disciplinary one.

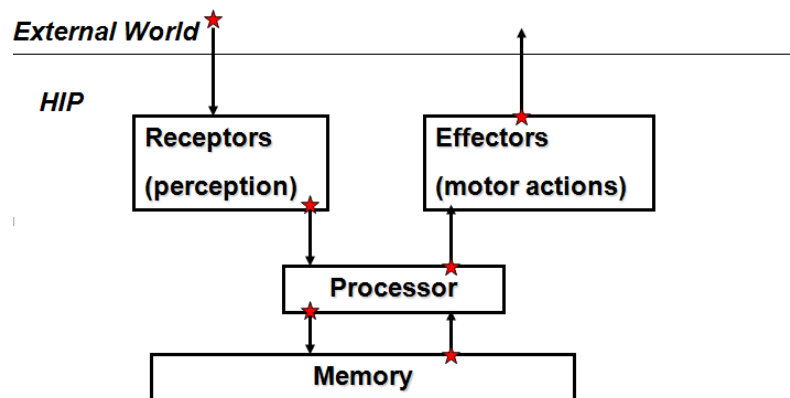


Fig.3.1. Cognitive Model for Human Information Processor (HIP)

3.1.3 Cognitive Model

- **Congnitive Model:** A theory that produces a computational model of how people perform tasks and solve problems by using psychological prinicples and emprical studies.
- It is both a research tool for theory building and and engineering tool for applying theroy.
- The aim of cognitive model in Human Computer Interaction (HCI) is to design and evolution of interface alternatives.
- Cognitive models are abstract, quantitative, approximate, and estimate from experimnts based on a theory of cognition.

- There is a need to model some aspects of user's understanding, knowledge, intentions or processing.
- The level of representation differs from technique to technique- from models of high level goals and the results of problem solving activities, to description of motor level activity such as keystrokes and mouse click.

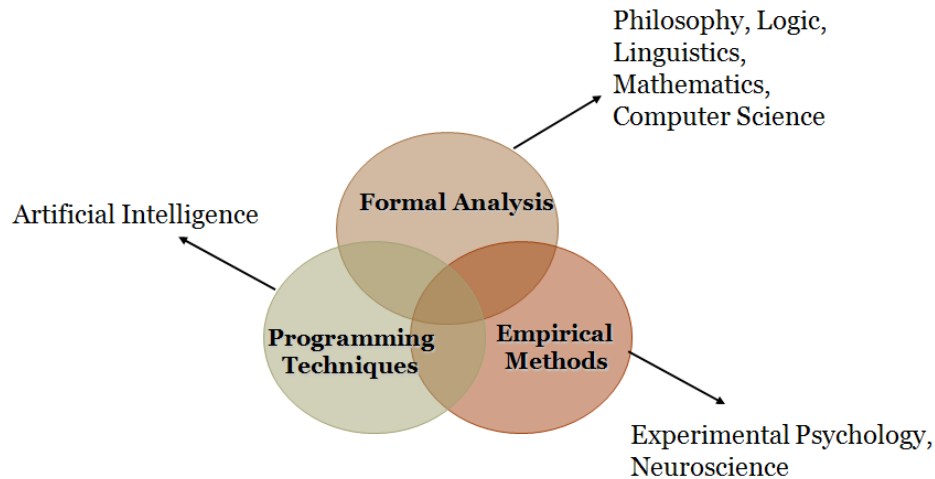


Fig.3.2. Cognitive Model

- The common categorization of cognitive models are:
 - Competence Vs Performance
 - Computational Flavour
 - No clear divide
- Competence Model
 - Competence models tend to be ones that can predict legal behavior sequences but generally do this without reference to whether they could actually be executed by users
 - Competence models, therefore, represent the kinds of behavior expected of a user, but they provide little help in analyzing that behavior to determine its demands on the user.
- Performance Model
 - performance models not only describe what the necessary behavior sequences are but usually describe both what the user needs to know and how this is employed in actual task execution.
 - Performance models provide analytical power mainly by focusing on routine behavior in very limited applications.
- Cognitive model for HCI are mainly classified in to three categories:
 - Hierarchical representation of the user's tasks and goal structure.

- Linguistic and grammatical model
- physical and device-level models.
- **Role of Cognitive Model in HCI**
 - It attempts to predict user performance based on a model of cognition in HCI.
 - HCI uses that model to predict how human would complete tasks using a particular user interface.
 - It limits the design space in HCI
 - HCI can take any specific design decision based on the answers of cognitive model
 - It is used to estimate both total task time and training time.
 - Also used to identify complex, error-prone stages of the design.
- **Advantages**
 - Don't need to implement or prototype
 - Don't need to test with real users
 - Theory has explanatory power which provides scientific foundation for design , like other engineering fields.

3.1.4. Goal and Task Structure

- It is based on divide and conquer
- It models the mental processing in which the user achieves goals by solving sub goals.
- **Goals** are intentions/aims (i.e. what you would like to be true).
- **Tasks** are actions (i.e., how to achieve it)
- Consider the example to produce a report on sales of introductory HCI text books.
- To achieve this goal, we divide in to several sub goals viz gathering the data together, producing the tables and histograms and writing the descriptive material
- Again such above mentioned sub goals are divided into further sub goals until some level of details is found at which we decide to stop.
- *Example:* again gathering the data together sub goal is divided into find the names of all introductory HCI books and then search the book sales database for these books.
 - gather data
 - . find book names
 - . . do keywords search of names database
 - *further sub-goals*



- . . . sift through names and abstracts by hand
- *further sub-goals*
- . . . search sales database - further sub-goals
- layout tables and histograms - further sub-goals
- write description - further sub-goals
- There are three techniques are used to model the Goals and Task Hierarchies:
 - Goals, Operators, Methods, and Selection (GOMS)
 - Cognitive Complexity Theory (CCT)
 - Hierarchical Task Analysis (HTA)
- **Issues for Goal Vs Task Structure:**
 - Granularity
 - Where do we start and stop?
 - Get down to routine learned behavior, not problem solving
 - Unit task – most abstract task is unit task which does not require any problem solving skills on the part of the user.
 - Conflict
 - There is more than one way to achieve the goal or solve the problem.
 - Treatment of Error

3.1.4.1. Goal, Operators, Methods and Selection (GOMS)

- GOMS provides a higher-level language for task analysis and UI modeling
- It generates a set of quantitative and qualitative measures based on description of the task and user interface
- It provides a hierarchy of goals and methods to achieve them
- The different GOMS variants use different terms, operate at various levels of abstraction, and make different simplifying assumptions
- GOMS is a formal representation of routine cognitive skill
- A description of knowledge required by an expert user to perform a specific task.
- It provides a description of what the user must learn.
- GOMS can be classified as a predictive, descriptive and prescriptive model:
 - **Predictive**
 - Predicts the time it will take user to perform the tasks under analysis
 - **Descriptive**
 - Represents the way a user performs tasks on a system

- It provides less work than user study
- It is easy to modify when UI is revised
- It is research tools to aid modeling process since it can still be tedious

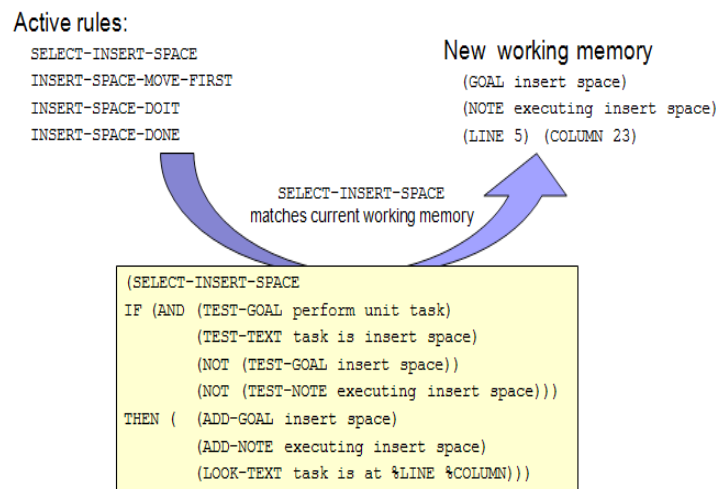
3.1.4.1. (g) *Disadvantages of GOMS:*

- Assumption made here in GOMS model is extreme expert behavior
- Here the Tasks must be goal-directed
- It does not model problem-solving, exploration, etc.
- It is very difficult to model behavior at this level of detail
- It is still hard to model anything but the simplest tasks
- It not easy for heuristic analysis, guidelines, or cognitive walkthrough
- GOMS is an interesting theoretical model, but is hardly ever used in practice

3.1.4.2. *Cognitive Complexity theory (CCT)*

- It was introduced by Kieras and polson.
- The basic premises of CCT is goal decomposition
- It enriches the model to provide more analytical power.
- There are two parallel descriptions:
 - User's goal
 - It is based on a GOMS like goal hierarchy
 - It is expressed by using some production rule. i.e if condition then action
 - Where condition is statement about the contents of working memory
 - If the condition is true then the production rule is said to be fire.
 - Action may consist of one or more elementary actions which may be either changes to the working memory, or external action such as key stroke.
 - Production rule may be written as program in LISP like language.
 - Device/ Computer goal
 - CCT uses generalized transition networks
 - It is a forms of state transition network
 - It is covered under dialogue models.
- Consider the example of an editing task using the UNIX vi text editor
- We had written CCT.
- Here the production rules are in long-term memory where 4 rules in the text on page 425.

- CCT as:
 (GOAL perform unit task)
 (TEXT task is insert space)
 (TEXT task is at 5 23)
 (CURSOR 8 7)
- Four rules to model inserting a space is:



- Some notes on CCT:
 - The rules did not fire in the order they were written
 - The rules are all active and at each moment any rule that has its conditions true may fire. Some rules may never fire; the same rule may fire repeatedly.
 - The rules can fire simultaneously in parallel model
 - The rules may be represented by experts

3.1.4.3. Problems and Extensions of goal hierarchies

- The description can be enormous in goal hierarchies
- Goal hierarchies are a post hoc technique where the risk is that it is defined by the computer dialog and not a user one
- Expert and also novice can also use this model at their level. Hence the representation of model for any problem is different by both experts and novice.

3.1.5 Linguistic Models

- Usually, Language is a tool for user can interact with computer.
- Understanding the user's behaviour and cognitive difficulty based on analysis of language between user and system.

- There are two types of Linguistic dialogue models
 - Backus–Naur Form (BNF)
 - Task–Action Grammar (TAG)

3.1.5. (a) *Backus-Naur Form(BNF)*

- It is a very common notation from computer Science
- BNF rules are used to describe the dialog grammar
- It is purely syntactic view of dialogue. It just ignores semantics
- BNF has been used widely to specify the syntax of computer programming language.
- There are two types of description used here: Terminals and Non Terminals.
 - Terminals
 - It shown in upper case letter
 - It represents the lowest level of user behaviour
 - *e.g.* CLICK-MOUSE, MOVE-MOUSE
 - Nonterminals
 - It is shown in lower case letter
 - It represents the higher level of abstraction
 - *e.g.* select-menu, position-mouse
 - Non-terminals are defined in terms of other non-terminals and terminals by a definition of the form
 - Name ::= expression
 - Where ::= symbol is read as ‘is defined as’
 - Non-terminals only appear on the left hand side.
- Consider the following example
 - An expression
 - contains terminals and nonterminals
 - combined in sequence (+) or as alternatives (|)

draw line ::= select line + choose points + last point

select line ::= pos mouse + CLICK MOUSE

choose points ::= choose one | choose one + choose points

choose one ::= pos mouse + CLICK MOUSE

last point ::= pos mouse + DBL CLICK MOUSE

pos mouse ::= NULL | MOVE MOUSE+ pos mouse

- Measurement with BNF
 - In order to measure and analyze the BNF we could consider the following factors:
 - Count the number of rules where the more rules an interface requires to use it, the more complicated it is. Not all the rules are so good.
 - Count the number of '+' and '|' operators. This would be in effect, penalize the more complex single rule.

3.1.5. (b) Task Action Grammar(TAG)

- BNF ignores the advantages of consistency both in the language's structure and in its use of command names and letters.
- TAG making consistency more explicitly.
- It uses parameterized grammar rules to emphasize consistency and encoding the user's world knowledge.
- In order to illustrate the consistency, we consider the three UNIX commands: cp(Copy), mv(Move) and ln(link)
- Each has two possible forms in BNF
- In BNF, three UNIX commands would be described as:
 - `copy ::= cp + filename + filename | cp + filenames + directory`
 - `move ::= mv + filename + filename | mv + filenames + directory`
 - `link ::= ln + filename + filename | ln + filenames + directory`
- No BNF measure could distinguish between this and a less consistent grammar in which
 - `link ::= ln + filename + filename | ln + directory + filenames`
- In TAG, this consistency of argument order can be made explicit using a parameter, or semantic feature for file operations.

Feature	Possible values
<i>Op</i>	{ <i>copy, move, link</i> }

- Rules here are:
 - `file-op[Op] ::= command[Op] + filename + filename`
 - `command[Op] + filenames + directory`
 - `command[Op = copy] ::= cp`
 - `command[Op = move] ::= mv`
 - `command[Op = link] ::= ln`

3.1.6. Physical and Device Models

- There are two types:
 - The Keystroke Level Model (KLM)
 - Buxton's 3-state model
- It is based on empirical knowledge of human motor system.
- User's task here is acquisition and execution. It only addresses the execution.
- This model is complementary with goal hierarchy.

3.1.6. (a) Keystroke Level Model (KLM)

- It is lowest level of GOMS
- It predicts the user performance based on deep cognitive understanding
- It is aimed at unit tasks within interaction
- The assumption is that these more complex tasks would be split into sub task before the user attempts to map them into physical action
- The tasks is split in to 2 phases:
 - Acquisition of the task – ie the user builds mental representation of the task
 - Execution of the task using the system's facilities.
- It is related to GOMS model
- The model decomposes the execution phase in to 5 different physical motor operators, a mental operator and a system response operator.
 - Physical motor:
 - K - keystroking
 - P - pointing
 - H - homing
 - D - drawing
 - Mental
 - M - mental preparation
 - System
 - R - response
- The execution of a tasks will involve interleaved occurrence of the various operators
- The execution time are empirically determined by
 - $T_{\text{execute}} = T_K + T_P + T_H + T_D + T_M + T_R$
- Consider the example, we are using mouse based editor where if we notice a single character error we will point at the error, delete the character and retype it, and then return to previous typing point. KLM model of previous example and its time calculation are as follows:

GOAL: ICONISE-WINDOW

[select

GOAL: USE-CLOSE-METHOD

. MOVE-MOUSE-TO- FILE-MENU

. PULL-DOWN-FILE-MENU

. CLICK-OVER-CLOSE-OPTION

GOAL: USE-CTRL-W-METHOD

PRESS-CONTROL-W-KEY]

- compare alternatives:
 - USE-CTRL-W-METHOD vs.
 - USE-CLOSE-METHOD
 - assume hand starts on mouse

USE-CTRL-W-METHOD		USE-CLOSE-METHOD	
H[to kbd]	0.40	P[to menu]	1.1
M	1.35	B[LEFT down]	0.1
K[ctrlW key]	0.28	M	1.35
		P[to option]	1.1
		B[LEFT up]	0.1
Total	2.03 s	Total	3.75 s

3.1.7. Cognitive Architecture Model

- All of cognitive models make assumptions about the architecture of the human mind.
- GOMS is based on divide and conquer method where the concept of taking problem and solving it by divide and conquer method
- CCT assumes the distinction between long and short term memory where production rule being stored in long term memory and matched against the contents of short term memory to determine which 'fire'.
- KLM based on model human processor architecture.

3.1.7. (a) Problem Space Model

- Rational behavior is characterized as behavior that is intended to achieve a specific goal.
 - This element of rationality is often used to distinguish between intelligent and machine-like behavior.

- In the field of artificial intelligence (AI), a system exhibiting rational behavior is referred to as a *knowledge-level* system.
- A knowledge-level system contains an *agent* behaving in an environment.
 - The agent has knowledge about itself and its environment, including its own goals.
 - It can perform certain actions and sense information about its changing environment.
 - The agent behaves in its environment, it changes the environment and its own knowledge.
- This model can view the overall behavior of the knowledge-level system as a sequence of environment and agent states as they progress in time.
 - The goal of the agent is characterized as a preference over all possible sequences of agent/environment states.
- The contrast of this rational behavior with another general computational model for a machine, which is not rational.
 - *For example*, it is common to describe a problem as the search through a set of possible states, from some initial state to a desired state.
 - The search proceeds by moving from one state to another possible state by means of operations or actions, the ultimate goal of which is to arrive at one of the desired states.
 - Once a programmer has identified a problem and a means of arriving at the solution to the problem (the algorithm), the programmer then represents the problem and algorithm in a programming language, which can be executed on a machine to reach the desired state.
 - The architecture of the machine only allows the definition of the search or problem space and the actions that can occur to traverse that space.
 - Termination is also assumed to happen once the desired state is reached.
 - The machine does not have the ability to formulate the problem space and its solution, mainly because it has no idea of the goal.
- It is the job of the programmer to understand the goal and so define the machine to achieve it.
- In order to realize the architecture of a knowledge-level system, we can adapt the state-based computational model of a machine.
 - The new computational model is the *problem space model*.
- Thus, a problem space consists of a set of states and a set of operations that can be performed on the states.
- Behavior in a problem space is a two-step process.

CATWOE

- The acronym of CATWOE is
 - **Clients:** those who receive output or benefit from the system
 - **Actors:** those who perform activities within the system
 - **Transformations:** the changes that are affected by the system
 - **Weltanschauung:** (from the German) or World View - how the system is perceived in a particular root definition
 - **Owner:** those to whom the system belongs, to whom it is answerable and who can authorize changes to it
 - **Environment:** the world in which the system operates and by which it is influenced

3.2.3. Participatory Design

- *Participatory design* is a philosophy
- It encompasses the whole design cycle.
- It is design in the workplace, where the user is involved not only as an experimental subject or as someone to be consulted when necessary but as a member of the design team.
- Users are therefore active collaborators in the design process, rather than passive participants whose involvement is entirely governed by the designer.
- It has three specific characteristics:
 - It is design and evaluation context or work oriented rather than system oriented.
 - It is characterized by collaboration: the user is included in the design team and can contribute to every stage of the design.
 - The approach is iterative: the design is subject to evaluation and revision at each stage.
- It uses the following methods to convey information between the user and designer:
- Brainstorming- The session provides a range of ideas from which to work. These can be filtered using other techniques.
- Storyboarding- means of describing the user's day-to-day activities as well as the potential designs and the impact they will have
- Workshop - used to fill in the missing knowledge of both user and designer and provide a more focussed view of the design.
- Pencil and paper exercises- allow designs to be talked through and evaluated with very little commitment in terms of resources.

Effective Technical and Human Implementation of Computer-based Systems (ETHICS)

- It was developed by Enid Mumford
- It is a participatory socio-technical approach
- it is distinct in its view of the role of stakeholders in the process
- Here, stakeholders are included as participants in the decision making process.
- ETHICS considers the process of system development as one of managing change: conflicts will occur and must be negotiated to ensure acceptance and satisfaction with the system.
- It has three levels of participation:
 - **Consultative** – the weakest form of participation where participants are asked for their opinions but are not decision makers.
 - **Representative** – a representative of the participant group is involved in the decision making process.
 - **Consensus** – all stakeholders are included in the decision-making process.

3.2.4. Ethnographic methods

- It is very influential in CSCW
- It is a form of anthropological study with special focus on social relationships
- It does *not* enter actively into situation
- It seeks to understand social culture

Contextual Inquiry

- It is an ethnographic approach developed by Holtzblatt
 - The model of investigator being apprenticed to user to learn about work
 - The investigation takes place in workplace - detailed interviews, observation, analysis of communications, physical workplace, artefacts
 - The number of models created:
 - sequence, physical, flow, cultural, artefact
 - models consolidated across users
- The output indicates task sequences, artefacts and communication channels needed and physical and cultural constraints

3.3. COMMUNICATION AND COLLABORATION MODELS

- Groupware systems, such as email or conferencing systems, involve more than one person.

- The field of computer-supported cooperative work (CSCW) encompasses both specific groupware systems and the effects of computers on cooperative working in general.
- Effective communication underlies much collaborative work and many systems aim to support communication at a distance.
- Face-to-face communication is often seen as the ideal to which computer-mediated communication should aim.

3.3.1. Face to Face Communication

- It is the most primitive and most subtle form of communication
- It is often seen as the paradigm for computer mediated communication
- It involves not just speech and hearing, but also the subtle use of body language and eyegaze.

3.3.1. (a) *Transfer effects and personal space*

- Face-to-face communication can carry forward our expectations and social norms by using computer-mediated forms of communication.
- People are very adaptable and can learn new norms to go with new media (for example, the use of 'over' for turn-taking when using a walkie-talkie).
- The success with new media is often dependent on whether the participants can use their existing norms.
- Furthermore, the rules of face-to-face conversation are not conscious.
- It may interpret failure as rudeness of colleague
- *e.g. personal space*
 - video may destroy mutual impression of distance
 - happily the 'glass wall' effect helps

3.3.1. (b) *Eye Contact and gaze*

- Normal conversation uses eye contact extensively, if not as intently.
- Our eyes tell us whether our colleague is listening or not; they can convey interest, confusion or boredom
- Video may spoil direct eye contact.
- A role in establishing rapport between the participants, eyegaze is useful in establishing the focus of the conversation.

3.3.1. (c) *Gestures and body language*

- Much of our communication is through our bodies gesture (and eye gaze) used for deictic reference

- where the first letter denotes the speaker (Alison or Brian) and the second letter labels the adjacency pair.

3.3.2. (b) Context

- Utterances are highly ambiguous
- We use context to disambiguate:
 - **Brian:** (*points*) that post is leaning a bit
 - **Alison:** that's the one you put in
- There are two types of context:
 - external context – reference to the environment
 - e.g., Brian's '*that*' – the thing pointed to
 - internal context – reference to previous conversation
 - e.g., Alison's '*that*' – the last thing spoken of
- Often contextual utterances involve indexicals:
 - *that, this, he, she, it*
- These may be used for internal or external context
- Also descriptive phrases may be used:
 - external: '*the corner post is leaning a bit*'
 - internal: '*the post you mentioned*'

3.3.2. (c) Topics, focus and forms of utterance

- The conversation is so dependent on context, it is important that the participants have a shared focus.
- Consider the example,

Alison: Oh, look at your roses : : :

Brian: mmm, but I've had trouble with greenfly.

Alison: they're the symbol of the English summer.

Brian: greenfly?

Alison: no roses silly!
- In the above conversation, Tracing topics is one way to analyse conversation.
 - Alison begins – *topic* is roses
 - Brian shifts topic to greenfly
 - Alison misses shift in focus ... *breakdown*

3.3.2. (d) Breakdown and repair

- Breakdown happens at all levels:
topic, indexicals, gesture
- Breakdowns are frequent, but
- redundancy makes detection easy
(Brian cannot interpret 'they're ... summer')
- correction after breakdown is called *repair*.
- people very good at repair
(Brain and Alison quickly restore shared focus)
- Electronic media may lose some redundancy
⇒ breakdown more severe

3.3.2. (e) Speech act theory

- A particular form of conversational analysis, *speech act theory*, has been both influential and controversial in CSCW.
- Not only is it an analytic technique, but it has been used as the guiding force behind the design of a commercial system, Coordinator.
- The basic premise of speech act theory is that utterances can be characterized by what they *do*.
- For example, if you say 'I'm hungry', this has a certain *propositional meaning* – that you are feeling hungry.
- Speech act theory concerns itself with the way utterances interact with the actions of the participants.
- The act of saying the words changes the state of the couple. Other acts include promises by the speaker to do something and requests that the hearer do something.
- These basic acts are called *illocutionary points*.

3.3.3. Text based Communication

- The major form of direct communication is text based in *asynchronous* groupware system.
- Text-based communication is familiar to most people, in that they will have written and received letters.
- There are four types of textual communication in current groupware:
 - discrete directed messages, no structure
 - linear messages added (in temporal order)
 - non-linear hypertext linkages

- spatial two dimensional arrangement

3.3.3. (a) *Back channels and affective state*

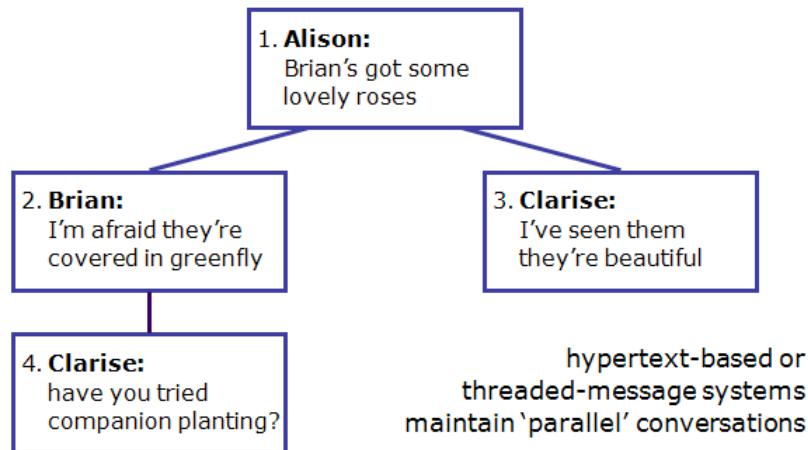
- One of the most profound differences between face-to-face and text-based communication is the lack of fine-grained channels.
- Text-based communication loses these back channels completely.
- There is no facial expression or body language in text based communication
⇒ weak *back channels*
- So, difficult to convey: *affective state* – happy, sad, ... , *illocutionary force* – urgent, important, ...
- Email users have developed explicit tokens of their affective state by the use of ‘flaming’ and ‘smilies’, using punctuation and acronyms; for example:
 - :-) – smiling face, happy
 - :-(– sad face, upset or angry
 - ;-)) – winking face, humorous
 - LOL – laughing out loud.

3.3.3. (b) *Grounding Constraints*

- Establishing common ground depends on *grounding constraints*
 - cotemporality – instant feedthrough
 - simultaneity – speaking together
 - sequence – utterances ordered
- Often weaker in text based communication
e.g., loss of sequence in linear text
- Network delays or coarse granularity ⇒ *overlap*
 - Bethan:** how many should be in the group?
 - Rowena:** maybe this could be one of the 4 strongest reasons
 - Rowena:** please clarify what you mean
 - Bethan:** I agree
 - Rowena:** hang on
 - Rowena:** Bethan what did you mean?
- Message pairs 1&2 and 3&4 composed simultaneously
lack of *common experience*
 - Rowena: 2 1 3 4 5 6

Bethan: 1 2 4 3 5 6

- N.B. breakdown of turn-taking due to poor back channels
- In order to maintain context, Recall *context* was essential for disambiguation
- Text loses external context, hence deixis
 - (but, linking to shared objects can help)
- (1) **Alison:** Brian's got some lovely roses
- (2) **Brian:** I'm afraid they're covered in greenfly
- (3) **Clarise:** I've seen them, they're beautiful
- Both (2) and (3) respond to (1)
- ... but *transcript* suggests greenfly are beautiful!
- Non Linear Conversation



3.3.3. (c) *Pace and granularity*

- The term *pace* is being used in a precise sense
- The pace of the conversation is the rate of such a sequence of connected messages and replies.
- Clearly, as the pace of a conversation reduces, there is a tendency for the *granularity* to increase.
- Pace of conversation – the rate of turn taking
 - face-to-face – every few seconds
 - telephone – half a minute
 - email – hours or days
- face-to-face conversation is highly interactive
 - initial utterance is vague

- feedback gives cues for comprehension
- lower pace $\Rightarrow$ less feedback
 $\Rightarrow$ less interactive
- People create *coping strategies* when things are difficult
- Coping strategies for slow communication attempt to increase granularity:
- *eagerness* – looking ahead in the conversation game
 - **Brian:** Like a cup of tea? Milk or lemon?
- *multiplexing* – several topics in one utterance
 - **Alison:** No thanks. I love your roses.
- Reviewability is another grounding constraint of communication, but this time one where text-based communication has the advantage over speech.

3.3.4. Group Working

3.3.4. (a) Group Dynamics

- Work groups constantly change:
 - – in structure – in size
- Several groupware systems have explicit rôles
 - But rôles depend on context and time
- *e.g.*, M.D. down mine under authority of foreman
 - and may not reflect duties
- *e.g.*, subject of biography, author, but now writer
- Social structure may change: democratic, autocratic, and group may fragment into sub-groups
- Groupware systems rarely achieve this flexibility
- Groups also change in composition
 - $\Rightarrow$ new members must be able to 'catch up'

3.3.4. (b) Physical environment

- Face-to-face working radically affected by layout of workplace
- *e.g.* meeting rooms:
 - recessed terminals reduce visual impact
 - inward facing to encourage eye contact
 - different power positions

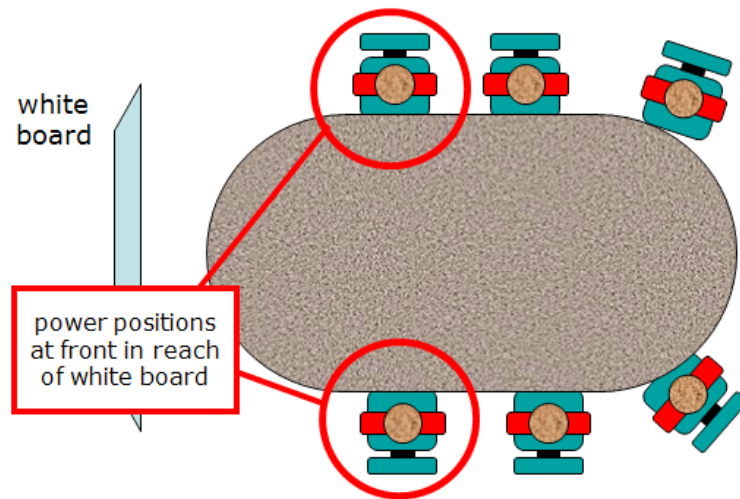


Fig.3.10. Power Position – Traditional Meeting room

- **Distributed cognition**
 - Traditional cognitive psychology in *the head*
 - Distributed cognition suggests look to *the world*
 - Thinking takes place in interaction
 - with other people
 - with the physical environment
- **Implications for group work:**
 - importance of mediating representations
 - group knowledge greater than sum of parts
 - design focus on external representation

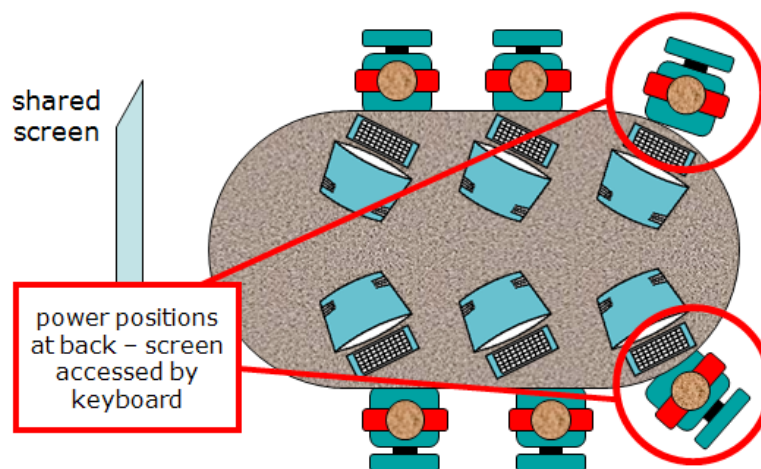


Fig.3.11. Power Position – Augmented Meeting room

OVERVIEW

We need to know what a system does in order to assess its usability.

- Standard software engineering formalisms can be used to specify an interactive system. These are of various types:
 - model based, such as Z, which describe the system's state and operations
 - algebraic formalisms, which describe the effects of sequences of actions
 - temporal and deontic logics, which describe when things happen and who is responsible.
- Special interaction models are designed specifically to describe usability properties, including:
 - predictability and observability – what you can tell about the system from looking at it
 - reachability and undo – what you can do with it.
- Most formal models and notations focus on events and changes that happen when they occur, but we need richer models to deal with:
 - interstitial behavior – the things that happen between events such as dragging an icon
 - physical objects in ubiquitous computing or virtual reality
 - the tension between precise time and more fuzzy human ideas of time.

17.1 INTRODUCTION

In the previous chapter we looked at the specification of dialog. In this chapter we will look at ways of modeling the semantics of an interactive system. The dialog just tells us about what user actions are legal at any point, but here we will be interested in what the user's actions *do* to the system.

There is some overlap. In the previous chapter we discussed some of the ways dialog is linked to semantics. Also, definitions of system semantics tend to define the acceptable dialog. However, a system-oriented description will not describe the dialog as well as a dedicated dialog notation, and a dialog notation may hardly define the system at all.

In Section 17.2, we will describe how standard formalisms can be used to specify interactive systems. Such formalisms are becoming part of software engineering practice and can be a way for interface designers to articulate their ideas and communicate them to system developers. These formal notations are used to specify the functionality and presentation of specific systems.

Section 17.3 looks at interaction models, formal models that describe general properties of systems. We will demonstrate how one such model, the PIE model, can be used to investigate generic principles such as WYSIWYG and undo.

Finally, in Section 17.4, we will discuss ways to model phenomena that involve more continuous interactions such as dragging an icon and also issues of time such as what we mean by 'let's meet this afternoon'.

Each modeling technique takes the view that it is not enough to design the 'interface' as a thin layer between the user and the system. Usability is affected by the whole system functionality: what is there, what can be done to it, and how it is presented at the interface.

17.2 STANDARD FORMALISMS

In this section we will discuss 'off-the-shelf' formalisms which can be used to specify interactive systems. The purpose of a formal specification is twofold: communication and analysis. Sections 17.2.1 and 17.2.2 will show how this influences their use for interface design.

We will then look at three brands of formalism. First we will look at model-oriented notations, Z in particular. We will use a graphics program as an example throughout this subsection and also Section 17.2.5 on algebraic specification. Finally, Section 17.2.6 looks at temporal and other logics. Another major brand of notation is the process algebras, such as CSP. However, these are most suited to dialog design and have been discussed in Chapter 16.

Real time

Temporal logics only deal with time in the sense that they represent the succession of events – one thing happens before another. They do not represent actual durations and times in hours, minutes and seconds. Clearly, there are important user interface aspects which require a real-time statement such as the following:

When the ellipse option is selected it must be highlighted within 100 ms and the new object must appear on the screen within 1 second

Programming, specifying and reasoning about real-time behavior is a very active research area, and a variety of notations have been developed, such as *real-time logic* and timed versions of the process algebras (CSP and CCS). However, as yet, none has been extensively used within HCI. One barrier to effective use is that general properties are difficult to state exactly – we usually want a response time of ‘around’ 100 milliseconds. Also, the real-time notations have often been developed with time-critical applications in mind, whereas, for many human response time issues, there is a gradual degradation in performance as the time increases, rather than a critical time after which the system might as well not bother. On the other hand, there are some timing issues where critical bounds do occur: for example, in hand–eye coordination tasks, delays of even a few hundred milliseconds can destroy performance totally.

Whether or not we use a formalism to describe real-time properties of a design, we must always remember that these issues are important, and we are not using an ‘infinitely fast machine’ (see Chapter 2 and [93]).

Deontics – responsibility and freedom

Specifications say what the designer thinks should happen. Specifications of the system say what it should do, but real requirements are often about the world – ‘the system should have a secure backup every week’. One could write a formal version of this statement using some form of temporal logic, but when we come to design a system to satisfy it we have trouble. We can design a ‘backup’ program which puts all the data onto tape, and which uses sophisticated error checking and redundancy to make sure the saved data are secure. We can even design a user interface which is resilient to mistakes on the operator’s part, but which is so clear that such mistakes never happen. But all this is to no avail if the operator does not put the backup tape into the system! Traditional specifications say *what* happens, but do not mention *who* is responsible for making it happen.

Consider the following statements:

Hotel rules: the guests are to be in the hotel by midnight

Prison rules: the inmates are to be locked in their cells from 9pm to 6am

The two statements are similar in form, and their propositional meanings are almost identical. If we swap a few words they transform into one another. However, *from context* we know that the first statement expresses a requirement on the hotel guests, whereas the second expresses a requirement on the prison warders.

Deontic logics address these issues by including the concept of responsible agents (human, corporate and computer) and the mutual responsibilities between them. The most common deontic operators are *permission* (**per**) and *obligation* (**obl**). These both take two arguments: the first is who has the permission or obligation, and the second what it is they are permitted or obliged to make true. For example, we can refine the temporal logic statement about printing including these operators. The agents are the user ‘Jane’ and the laser printer ‘lp3’:

$$\begin{aligned} \text{owns}(\text{Jane}, \text{file 'fred'}) &\Rightarrow \text{per}(\text{Jane}, \text{request}(\text{Jane}, \text{'print fred'})) \\ \text{request}(\text{Jane}, \text{'print fred'}) &\Rightarrow \text{obl}(\text{lp3}, \text{prints the file 'fred'}) \end{aligned}$$

The first formula says that if Jane owns the file ‘fred’ then she is permitted to request the command ‘print fred’. The second says that if she requests the command, then the printer is obliged to print ‘fred’. The first statement is a little clumsy because it has to be phrased in terms of the proposition *request*(Jane, ‘print fred’). If we also include the idea of actions, such as *request*(‘print fred’), which can be performed (**performs**) by agents, then the requirements can be rephrased using a modified obligation operator:

$$\begin{aligned} \text{owns}(\text{Jane}, \text{file 'fred'}) &\Rightarrow \text{per}(\text{Jane}, \text{request('print fred')}) \\ \text{performs}(\text{Jane}, \text{request('print fred')}) &\Rightarrow \text{obl}(\text{lp3}, \text{print (the file 'fred')}) \end{aligned}$$

Such a statement is not far different from the statement of the requirements in English, and makes quite clear the balance of responsibility.

Deontic logics are becoming popular in requirements engineering. It is thus possible that as an interface designer, you may be asked to produce a system which satisfies some (suitably explained) deontic specification. Even if this is not the case, it is worth noting, formally or informally, exactly what expectations you have of your users. In practice, one ought to work with several levels of expectation. For example, if you are designing an automatic bank teller, it may be true that customers are obliged to use only their own cards, and to use them in a particular fashion. However, you do not want the system to crash too terribly if a customer does not behave in this fashion. As interactive systems begin to involve more than a single user (see Chapters 14 and 19), it becomes more important to keep track of these responsibilities and freedoms, who must do what and who can do what. If the user of a single-user system does not behave in the ‘obliged’ fashion then the impact is personal (although not necessarily acceptable). However, in a multi-user system, we want to restrict the bad effect on others – the system should still maintain its obligations to them and allow them to perform those things they are permitted to do.

17.3 INTERACTION MODELS

Interactive systems ought to be ‘*what you see is what you get*’ (WYSIWYG), *consistent*, have a universal *undo* facility . . . the list goes on. But, if a supplier says that its word processor ‘Sludge-Word’ is WYSIWYG, how do we test this? The screen fonts look very impressive, but are they the same as on the page? Perhaps the system appears

WYSIWYG for simple jobs but this breaks down when things become more complex. What does WYSIWYG really mean?

It was to address these issues that the methodology described in this section was developed. Whereas the formal notations described in Section 17.2 describe specific systems, the aim in this section is to define *interaction models*, which are generic, formal models of interactive systems. Using such a generic model, one can define principles in a formal way which are then applicable to a range of systems. In particular, by regarding Sludge-Word as an instance of the general model we can verify (or refute) some of its supplier's claims.

The particular model we will describe, the *PIE model*, was designed to attack WYSIWYG-like properties. It would be nice to say that after reading this section, you will know exactly what WYSIWYG means. Unfortunately, it is too wide and varied a term to be formalized. However, we will describe several principles from Chapter 7, more limited in scope, which can be formalized. Of these some cover areas within the general area of WYSIWYG, namely *observability* – what you can tell about the current state of the system from the display – and *predictability* – what you can tell about its future behavior. In addition, we will define principles concerning the control of the system by the user, such as *reachability* – you can get anywhere from anywhere – and *undo* – the ability to perform backwards error recovery.

As we mentioned in Chapter 7, we certainly cannot define ‘usability’ totally; we cannot say that if a system obeys a set of formal principles then it will be usable. Nevertheless, some of the formal principles are necessary for usability: any system which breaks them is bound to have problems. The formal principles form a ‘safety net’ to prevent some of the worst mistakes in an interactive system, but do not ensure a good design. That depends on a good designer.

This ‘safety net’, although valuable, is not the principal benefit of using interaction models. Their chief value lies in the insights gained by considering properties of interaction, away from the surface clutter of real or imagined systems. These insights become part of the background with which you approach new areas, whether or not a formal approach is explicitly taken there. Furthermore, some of these insights can be abstracted into informal principles, which, though derived by formal analyses, can, once stated, be justified in their own right. It is a strange paradox that some of the informal concepts, which are obtained by such formal analysis, are not themselves fully formalizable.

17.3.1 The PIE model

The PIE model is a *black-box model*. It does not try to represent the internal architecture and construction of a computer system, but instead describes it purely in terms of its inputs from the user and outputs to the user. For a simple single-user system, typical inputs would be from the keyboard and mouse, and outputs would be the computer's display screen and the eventual printed output (Figure 17.2).

The difference between the ephemeral *display* of a system and the permanent *result* is central to the PIE model. We will call the set of possible displays D and the

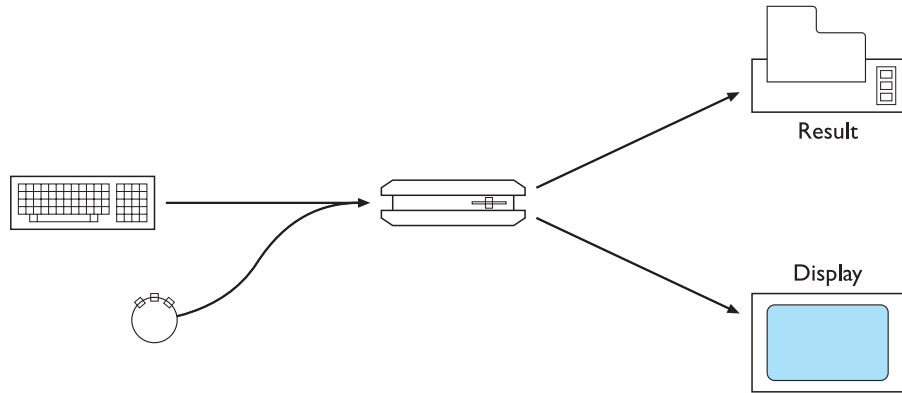


Figure 17.2 Inputs and outputs of a single-user system

set of possible results R . In order to express principles of observability, we will want to talk about the relation between display and result. Basically, can we determine the result (what you will get) from the display (what you see)?

For a formal statement of predictability it helps (but is not essential) to talk about the internal state of the system. This does not counter our claim to have a black-box model. First, the state we define will be opaque; we will not look at its structure, merely postulate it is there. Secondly, the state we will be discussing is not the actual state of the system, but an idealization of it. It will be the minimal state required to account for the future *external behavior*. We will call this the *effect* (E). Functions *display* and *result* obtain the current outputs from this minimal state:

$$\text{display} : E \rightarrow D$$

$$\text{result} : E \rightarrow R$$

The current display will be literally what is now visible. The current result is actually not what *is* available, but what the result would be if the interaction were finished. For example, with a word processor, it is the pages that would be obtained if one printed the current state of the document.

A single-user action we will call a *command* (from a set C). The history of all the user's commands is called the *program* ($P = \text{seq } C$), and the current effect can be calculated from this history using an *interpretation function*:

$$I : P \rightarrow E$$

Arguably the input history would be better labeled H , but then the PIE model would lose its acronym! If we put together all the bits, we obtain a diagram of sets and functions (Figure 17.3), which looks rather like the original illustration.

In principle, one can express all the properties one wants in terms of the interpretation function, I . However, this often means expressing properties quantified over all possible past histories. To make some of the properties easier to express, we will also use a state transition function *doit*:

$$\text{doit} : E \times P \rightarrow E$$

17.3.3 Reachability and undo

In Chapter 16, we discussed connectivity issues for dialogs. We could check a dialog description to see if there were any blind alleys, which once you had chosen them, would never let you back to the rest of the dialog. Systems can have similar problems at a semantic level.

In a commercial program debugger used by one of the authors, there is a window listing all the variables. If a variable is a complex structure, then hitting the ‘insert’ key while the cursor is over the variable will expand the variable showing all its fields. If you only want a few of the fields to be displayed, you can move the cursor over the unwanted fields and press the ‘delete’ key and the field is removed. These operations can be repeated over complex hierarchical structures. If you remove a field and then wish you had not, you can always press ‘insert’ again over the main variable and all the fields will be redisplayed. Even this breaks somewhat the principle of *commensurate effort*, but worse is to come. The ‘delete’ key also works for top-level variables, but once one of these is removed from the display there is *nothing* you can do to get it back, short of exiting the debugger and rerunning it from scratch.

A principle that stops this type of behavior is *reachability*. A system is reachable if from any state the system is in, you can get to any other state. The formal statement of this is as follows:

$$\forall e, e' \in E \cdot (\exists p \in P \cdot \text{doit}(e, p) = e')$$

Unlike the predictability principles, there are no awkward caveats. The only problem is that, if anything, it is too weak. For instance, a word processor could have a delete key, but no way to move the cursor about, so you always type at the end of the document. Now you can, of course, get from any document state to any other, you simply delete the whole text and retype what you want. However, if you had just typed in a whole letter and then noticed a mistake on the first line, you would not be pleased! So, ideally, one wants an independent idea of ‘distance’ between states and to make the difficulty of the path between them commensurate with the distance – small changes should be easy. Despite this, the principle on its own would have been strong enough to prevent the behavior of the debugger!

One special case of reachability is when the state you want to get to is the one you have just been in, that is *undo*. We expect undo to be easy, and ideally have a single undo button that will always undo the effect of the last command. We can state this requirement very easily:

$$\forall c \in C \cdot \text{doit}(e, c \frown \text{undo}) = e$$

This says exactly what we wanted. We start in a state e . We then do any command c and follow it by the special command *undo*. The state is then the same as we began in.

Stop! Before patting ourselves on the back for so clearly defining undo, we should check that this requirement for undo is consistent. Indeed, it is consistent – so long as there are at most two states. That is, the above undo requirement is only possible

for systems which *do* virtually nothing! The reason for this is that *undo* is itself a command and can undo itself. Take any state e and choose any command x . Let e_x be the state you get to after command x . That is $e_x = \text{doit}(e, x)$. Now we can apply the undo requirement to state e_x :

$$\text{doit}(e_x, \text{undo}) = \text{doit}(e, x \frown \text{undo}) = e$$

So, the *undo* command in state e_x gets us back to e . That is as expected. But what does *undo* do if we are in state e ? Again we can employ the undo principle remembering that $e = \text{doit}(e_x, \text{undo})$:

$$\text{doit}(e, \text{undo}) = \text{doit}(e_x, \text{undo} \frown \text{undo}) = e_x$$

This uses the undo principle when the command c is *undo* itself. However, our choice of command x was arbitrary, so if we had chosen another command, say y , we would have concluded that $\text{doit}(e, \text{undo}) = e_y$. This means that $e_x = e_y$, and in general anything we do from state e gets us to the same state. So at most we have two states, a toggle, with all the commands flipping back and forth between them. The only alternative is that the system does nothing.

We will not go on to describe the details of better undo requirements, the interested reader can find that elsewhere. The basis of most workable undo systems is that *undo* is not just any old command, but is treated differently. The simplest fix to the above undo principle is to restrict the commands to anything *except undo*!

The lesson from the above is clear. It is easy to say you want something which sounds quite reasonable. A formal description of the requirement may well reveal that, as in the case of *undo*, it is inconsistent – that is, *no* system could be built which satisfies the requirement.

17.3.4 Other interaction models

The PIE model was the inspiration for a wide variety of different models. Some of these are similar, but take a slightly different standpoint. For example, one model has results only available at some states. This actually corresponds to the intuition that a printed document is only available when you invoke the print function, or that an updated file is only available in the file system when you ask the editor to save. However, that model also only had the display available at some states – less intuitive.

There is also a range of models focussed at specific domain areas. The PIE model is a very general model of deterministic single-user systems. Other models address areas including:

Windowed systems This model describes the interference properties between windows. The distinctive feature is that the user is effectively regarded as having different personae when interacting with different windows. This reflects the observation that when swapping between tasks (associated with windows), the user should not have to keep track of dependencies and is therefore acting like several users.

Timing What happens when the system is not fast enough and the user's actions get ahead of the system's responses? The formal model describes the relationship between the steady state behavior – what would happen if the machine was infinitely fast and there were no delays – and the actual temporal behavior. This analysis has highlighted deficiencies in the input–output model of window managers and operating systems.

Attention This looks at ways of describing which parts of the display and result are used during any particular task. It uses *templates* for both, which model the selective attention during the task. The designer states what the expected templates are for any task, and then salience of the templates can be verified by experiment. These can then be tested using psychological experiments.

Non-determinism If you are ignorant of certain information in a system, it may appear to behave non-deterministically. Many properties of different models, for example the predictability of PIEs, map onto problems of non-determinism. Methods of handling this non-determinism from one domain can then suggest similar methods in what would otherwise appear disparate areas. The fact that supposedly deterministic interfaces often appear random suggests that, where it is helpful for other purposes, the interface can be made deliberately non-deterministic.

Dynamic pointers Interface objects often have special positions, such as the cursor or marked blocks. They may also have pointers into them, such as hypertext links. Dynamic pointers are a formalism that describes such positional information in a uniform manner. In particular, they help to manage the changes in pointers as the underlying object is updated. They also describe the mapping between positional information at different levels, for example when mapping mouse positions on the screen to their appropriate positions in application objects.

These are all principally single-user models. However, there has been some work in applying these formal models to group systems (such as those in Chapter 19). In particular, this has led to significant new insight into the meaning of undo in a group context.

17.4 CONTINUOUS BEHAVIOR

Although many of the systems and interfaces studied in rich media and in novel interfaces embody continuous real-time interaction, there are few models of this in the HCI literature. Possibly this is because of the conceptual dominance of discrete models. At a low level, computer systems are clearly discrete with step-by-step programs and users' actions being converted into streams of events. Even apparently smooth actions like dragging a mouse are treated as a series of individual 'mouse has moved' events.

There are two types of continuity:

continuous values – for example, the measurement of a weight;

continuous time – for example, a switch that could be sampled at any time.

Many sensors have both these properties: for example, the temperature of a room is both a continuous value and can be sampled at any time. It is the latter of these that we will consider in this section, as many standard notations such as Z deal with continuous values, whereas continuous time is less common.

In this section we will first look again at the PIE model and see how it can be extended to deal with mouse location.

17.4.1 Dealing with the mouse

The PIE model is very asymmetric between input and output. Some sorts of output are difficult to deal with, for example a beep used when something goes wrong. This is not clear in the formalism itself, but if we look at the principles, it is obvious that the display is expected to be persistent. If you go away and have a cup of tea, the display is still there when you get back. The beep is not. On the input side, mouse movement is not easy: one can regard each movement as being a command in C , but this is unnatural. Certainly, the user would not be conscious of each pixel movement!

A beep is similar to a keystroke: each is an *event*. The mouse's position is similar to the screen: they both have an observable value at any moment – we say each of them is a *status*. The PIE model is then an event-in/status-out model: the user does events (from C) and the system responds with a status (*display*). This suggests variations of the PIE model that have different combinations of event and status.

One version has events and status for its input, but still a purely status output (any-in/status-out). The (single command) state transition function then depends on both the command and the current mouse position (M):

$$doit : E \times C \times M \rightarrow E$$

The display function is now not just a function of the state, but it too depends on the mouse position:

$$display : E \times M \rightarrow D$$

This allows for the display to include the mouse cursor, which moves with the mouse (pretty obvious really) and also allows for the change in the display when the mouse drags an object.

This model only describes a special subclass of systems, those that are *trajectory independent*. The mouse position only has a permanent effect at the moment a command (keystroke or mouse button) happens; the intermediate positions are forgotten. For example, the model cannot describe freehand drawing. It is possible to extend the model to include more general systems; however, the subclass is

interesting in itself. With the exception of drawing and similar tasks, such as ‘air-brush’ painting, most mouse-based systems are trajectory independent. Furthermore, the few that are not exhibit usability problems, suggesting that trajectory dependency is itself a general usability principle for all but exceptional situations.

17.4.2 Formal aspects of status–event analysis

Probably the earliest continuous time models in the formal user interface literature are the variants of status–event analysis [94,102] of which the variant of the PIE model above is a simple example. Status–event analysis (S–E) distinguishes events that occur at specific moments of time from status phenomena that have (typically changing) values over a period of time. Examples of events include keystrokes, beeps, and the stroke of midnight in the story of Cinderella. Examples of status phenomena include the current computer display, the location of the mouse pointer, the internal state of the computer and the weather. We will discuss status–event analysis in more detail in Chapter 18; here we will just look at the more formal aspects.

Perhaps one of the most significant features of S–E is its treatment of interstitial behavior. Whereas discrete models focus solely on what happens at the moments when events occur, S–E puts equal emphasis on the more fluid interaction between events. In many GUI systems this is what gives the ‘feel’ of interaction, for example, dragging, scrolling, etc., and in rich media this is likely to be the main purpose of interaction!

Status–event analysis is really a conceptual framework for viewing interaction, but does have several concrete models, both descriptive models (variants of the PIE) and specification notations. These all have the general form of a state-transition style description of events and a more continuous description of interstitial behavior:

action:

$$\text{user-event} \times (\text{current/history of}) \text{ input-status} \times \text{state} \\ \rightarrow \text{response-event} \times (\text{new}) \text{ state}$$

interstitial behavior:

$$(\text{current/history of}) \text{ input-status} \times \text{state} \rightarrow \text{output-status}$$

The treatment of the input status at events and during interstitial behavior distinguishes interactions of markedly different kinds, for example, the trajectory dependency described above (Section 17.4.1).

Another crucial aspect of S–E, which is not apparent in discrete systems, is status-change events. These occur when a status phenomenon crosses some form of trigger threshold. For example, when a temperature reaches some value, or the time (on the clock) is a certain time. The nature of these thresholds is application dependent and may be dynamic. Furthermore, there are a whole range of issues about how status-change events become system-level events: polling, active sensors, etc. Typically, at an abstract level of specification one would just say ‘when this happens . . .’, but as

Unit-5

5.1 Groupware

Groupware is technology designed to facilitate the work of groups. This technology may be used to communicate, cooperate, coordinate, solve problems, compete, or negotiate. While traditional technologies like the telephone qualify as groupware, the term is ordinarily used to refer to a specific class of technologies relying on modern computer networks, such as email, newsgroups, videophones, or chat.

Groupware technologies are typically categorized along two primary dimensions:

1. whether users of the groupware are working together at the same time ("realtime" or "synchronous" groupware) or different times ("asynchronous" groupware), and
2. whether users are working together in the same place ("colocated" or "face-to-face") or in different places ("non-colocated" or "distance").
- 3.

Groupware is a term for applications written to support the collaboration of several users.

Groupware can support different activities:

- direct interpersonal communication
- ideas generation and decision making
- sharing computer objects.

Groupware can be classified by

when and where the participants are working
the function it performs for cooperative work

The Time/Space Matrix

Classify groupware by: when the participants are working, at the same time or not

where the participants are working, at the same place or not

	Same place	Different place
Same time	Face-to- face conversation	Telephone
Different time	Sticky note	Letter

Fig- Time/Space Matrix

Common names for axes: time: synchronous/asynchronous
place: co-located/remote

The time/space matrix is a very useful shorthand to refer to the particular circumstances a groupware system aims to address. Basically, we look at the participants and ask whether they are in the same place or not, and whether they are operating at the same time or not. Figure 19.1 shows how various non-computer communication technologies fit into the time/space matrix. The axes are given different names by different authors. The space dimension is also called the geographical dimension and is divided into co-located (same place) and remote (different place). Many of the earliest groupware systems were aimed at overcoming the barriers of distance, for example email and video conferencing. More recently, systems have arisen which aim to augment face-to-face meetings and other co-located cooperation. The time axis is often divided into synchronous and asynchronous systems; so we would refer to a telephone as a synchronous remote communication mechanism, whereas sticky notes are asynchronous co-located.

Classification by Function

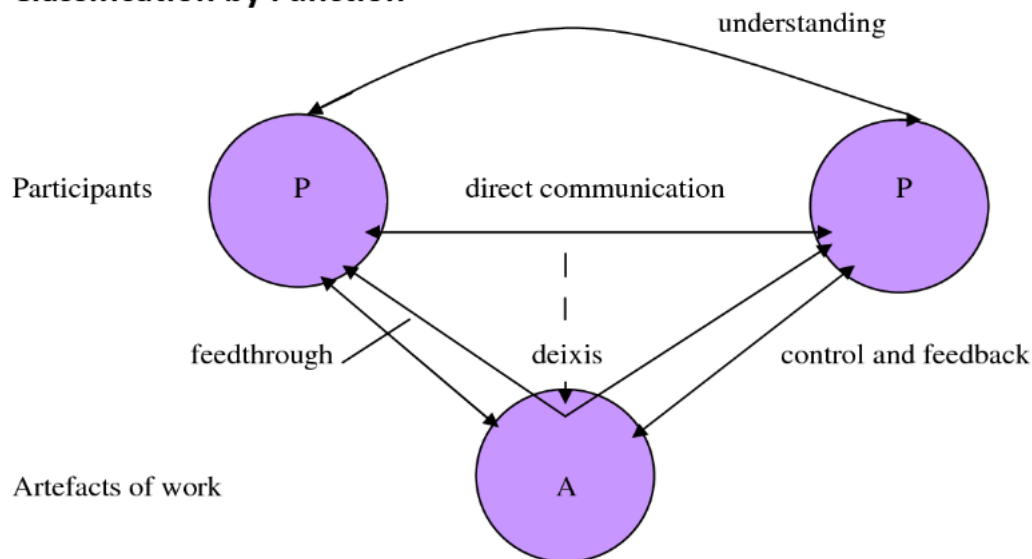


Fig- Cooperative work framework

Cooperative work involves: Participants who are working

Implicit in the term 'cooperative work' is that there are two or more participants. These are denoted by the circles labeled 'P'. They are engaged in some common work, and to do so interact with various tools and products. Some of these are physically shared (for example, two builders holding the ends of a measuring tape), but all are shared in the sense that they contribute to the cooperative purpose. These tools and other objects are denoted by the circle labeled 'A' – the artifacts of work. The participants communicate with one another as they work, denoted by the arrow between them. In real life this may be by speech (the builders with the tape), or letter (a lawyer and client); in fact, this direct communication may be in any of the categories of the time/space matrix. Part of the purpose of communication is to establish a common understanding of the task the participants are engaged in. This understanding may be implicit in the conversation, or may be made explicit in diagrams or text.

We will classify groupware systems by the function in this framework which they primarily support: computer-mediated communication supporting the direct communication between participants; meeting and decision support systems capturing common understanding;

shared applications and artifacts supporting the participants' interaction with shared work objects – the artifacts of work.

5.2 COMPUTER-MEDIATED COMMUNICATION

Email and bulletin boards-

Consider the stages during the sending of a simple email message:

1. **Preparation** You type a message at your computer, possibly adding a subject header.
2. **Dispatch** You then instruct the email program to send it to the recipient.
3. **Delivery** At some time later, anything from a few seconds (for LAN-based email) to hours (for some international email via slow gateways) it will arrive at the recipient's computer.
4. **Notification** If the recipient is using the computer a message may be displayed, saying that mail has arrived, or the terminal may beep.
5. **Receipt** The recipient reads the message using an email program, possibly different from that of the sender.

In the simple email example, there was just one recipient. However, most email systems also allow a set of recipients to be named, all of whom receive the message.

Like letters, these recipients may be divided into the direct recipients (often denoted by a To: field) and those who receive copies (Cc:). These two types of recipient are treated no differently by the computer systems – the distinction serves a social purpose for the participants. This is a frequent observation about any groupware system – the system should support people in their cooperation.

Structured message systems

asynchronous/remote 'super' email – cross between email and a database sender – fills in special fields recipient – filters and sorts incoming mail based on field contents ... but – work by the sender – benefit for the recipient

A common problem with email and electronic conferencing systems is overload for the recipient. As distribution lists become longer, the number of email messages received begins to explode. This is obvious: if each message you send goes to, on average, 10 people, then everyone will receive, on average, 10 messages for each one sent. The problem is similar to that caused for paper mail by photocopiers and mail merge programs. If we consider that newsgroups may have hundreds of subscribers, the problem becomes extreme. Happily, most newsgroups have only a few active contributors and many passive readers, but still the piles of unread electronic mail grow. Various forms of structured message system have been developed to help deal with this overload, perhaps the most well known being the Information Lens [225]. This adopts some form of filtering in order to sort items into different categories, either by importance or by subject matter. As well as a text message, normal email has several named fields: To, From, Subject. Structured message systems have far more, domain-specific, fields. The sender of the message chooses the appropriate message type, say a notification of a seminar. The system then presents a template which includes blank fields pertinent to the message type, for example Time, Place, Speaker and Title. Figure shows a typical structured message similar to those in Lens.

The named fields make the message more like a typical database record than a normal email message. Thus the recipient can filter incoming mail using databaselike queries. This can be used during normal reading – 'show me all messages From "abowd" or with Status "urgent"'. Alternatively, users may set up filtering agents to act on their behalf. Such an agent is a sort of electronic secretary; it is programmed with rules based on the field contents and can perform actions such as moving the message into a specific mailbox, deleting the message or informing the user.

Type: Seminar announcement
To: all From: Alan Dix
Subject: departmental
seminar Time: 2:15 Wednesday
Place: D014 Speaker: W.T. Pooh
Title: The Honey Pot
Text: Recent research on socially constructed meaning has focussed on the image of the Honey Pot and its dialectic interpretation within an encultured hermeneutic. This talk ...

Figure- Structured message

txt is gr8 –

While groupware developers produce more and more complex structured message systems, in the wider world people have voted with their fingers and adopted unstructured informal text messaging systems: instant messaging while online and SMS or paging through mobile phones.

Whereas email is more like an exchange of letters, IM is more like conversation with short exchanges, often not even complete sentences.

Hi, u there
yeh, had a good night last night?
uhu want to meet later

SMS stands for Short Message Systems, but few users know this or care – they simply send an ‘SMS’ or ‘text’ a friend. The SMS phenomenon is a salutary lesson to any of us who believe we can design the future. It was originally an ‘extra’ service, making use of a technical fix that allowed mobile phone companies to send messages to mobile phones to inform of voicemail or update internal settings.

Video conferences and communication

The idea of video phones has been around for a long time, from Flash Gordon’s days onwards, and early video phones have been available for at least 20 years.

Video conferences support specific planned meetings. However, one of the losses of working in a different site from a colleague is the chance meetings whilst walking down a corridor or drinking tea. Several experimental systems aim to counter this, giving a sense of social presence at a distance. One solution is the video window or video wall, a very large television screen set into the wall of common rooms at different sites [134]. The idea is that as people wander about the common room at one site they can see and talk to people at the other site – the video wall is almost like a window or doorway between the sites.

Virtual collaborative environments

Virtual reality techniques are now being used to allow participants to meet within a virtual world. Each participant views a virtual world using desktop or immersive VR. Within the virtual worlds the other participants appear, often rendered as a simple cube-based figure perhaps with a fixed or even video image texture mapped onto it. The representation of a participant in virtual space is called an embodiment

As participants move around in virtual space then their embodiments move correspondingly. Thus one can approach another within the virtual environment, initiate discussion and focus on common objects within the virtual environment. These systems attempt to mimic as much as possible of the real world in order to allow the participants to use their existing real-world social skills within the virtual world. For example, when other people speak, the volume may be adjusted so that those a long way away or behind you (in virtual space) are quieter than those close and in front. This is clearly very important in a heavily populated virtual environment. Given the virtual environment is within a computer, it makes sense to allow participants to bring other computer-based artifacts into the virtual environment. However, many of these are not themselves 3D objects, but simple text or diagrams. It is possible to map these onto flat virtual surfaces within the 3D virtual world (or even have virtual computer screens!). Text is especially difficult to read when rendered in perspective and so some environments take advantage of the fact that this is a virtual world to present such surfaces face on to all participants. But now we have a world the appearance of which is participant dependent.

5.3 MEETING AND DECISION SUPPORT SYSTEMS

In any conversation, the participants must establish a common understanding about the task they are to perform, and generate ideas. In some areas this is a secondary activity: you are discussing the job you are doing, the ideas support that job. However, there is a class of activities where the job is itself generating ideas and understanding. This is typically the case in a research environment, in design tasks, in management meetings and brainstorming sessions.

We will discuss three types of system where the generation and recording of ideas and decisions is the primary focus.

Argumentation tools -which record the arguments used to arrive at a decision and support principally asynchronous co-located design teams.

Meeting rooms -which support face-to-face groups (synchronous co-located) in brainstorming and management meetings.

Shared drawing surfaces- which can be used for synchronous remote design meetings.

5.3.1 Argumentation tools

argumentation tools is the family of tools developed to support the argumentation model called issue-based information system (IBIS).

This system has node types including 'issues', 'positions' and 'arguments', and these are linked together by relationships such as 'argument supports position'. Notice that argumentation tools may allow a range of interaction styles from asynchronous, when the designers use it one at a time, to fully synchronous, when several use it at once. Although there is no reason why the systems should not be used for distant collaboration they are typically used by groups within the same office, and in the case of one-at-a-time use, on one machine. They are thus largely, but not solely, asynchronous co-located groupware systems.

5.3.2 Meeting rooms

The advantages of email, bulletin boards and video conferences are obvious – if you are a long way apart, you cannot have a face-to-face meeting. Similarly, it is obvious why one should want to record decisions during a long-lived design process. The need for meeting rooms is less obvious. These are specially constructed rooms, with extensive computer equipment designed to support face-to-face

meetings. Given face-to-face meetings work reasonably well to start with, such rooms must be very well designed if the equipment is to enhance rather than disrupt the meeting.

The general layout consists of a large screen, regarded as an electronic whiteboard, at one end of the room, with chairs and tables arranged so that all the participants can see the screen. This leads to a U- or C-shaped arrangement around the screen, and in the biggest room even several tiers of seating. In addition, all the participants have their own terminals, which may be recessed into the tabletop to reduce their visual effect.

5.3.3 Shared work surfaces

The idea of a shared screen forming an electronic whiteboard is not confined to face-to-face meetings. One can easily imagine using the same software which runs in a meeting room, working between several sites. That is, we can take the synchronous co-located meeting room software and use it for synchronous remote meetings. As before, each participant's screen shows the same image, and the participants can write on the screen with the same sort of floor control policies as discussed earlier. There are additional problems. First, the participants will also require at least an audio link to one another and quite likely video as well. Remember that the social protocols used during lenient floor control, not to mention the discussion one has during a meeting, are difficult or impossible without additional channels of communication. As well as the person-to-person communications, the computer networks may have trouble handling the information. If there are delays between one person writing something on the board and another seeing it, the second participant may write to the same location. A situation which is easily avoided in the co-located meeting could become a major problem when remote.

5.4 SHARED APPLICATIONS AND ARTIFACTS

Shared PCs and shared window systems

Most of the groupware tools we have discussed require special collaboration-aware applications to be written. However, shared PCs and shared window systems allow ordinary applications to be the focus of cooperative work. Of course, you can cooperate simply by sitting together at the same computer, passing the keyboard and mouse between you and your colleague. The idea of a shared PC is that you have two (or more) computers which function as if they were one. What is typed on one appears on all the rest. This sounds at first just like a meeting room without the large shared screen. The difference is that the meeting rooms have special shared drawing tools, but the shared PC is just running your ordinary program. The sharing software monitors your keystrokes and mouse movements and sends them to all the other computers, so that their systems behave exactly like yours. Their keystrokes and movements are similarly relayed to you. As far as the application is concerned there is one keyboard and one mouse.

Shared PCs and window systems have two main uses. One is where the focus is on the documents being processed, for example if the participants are using a spreadsheet together to solve a financial problem. The other is technical support: if you have a problem with an application, you can ring up your local (or even remote) technical guru, who will connect to your computer, examine where you are and offer advice. Compare this scenario with trying to explain over the phone why your column is not formatting as you want.

Shared editors

A shared editor is an editor (for text or graphics) which is collaboration aware, that is it knows that it is being shared. It can thus provide several insertion points or locking protocols more tuned to the

editor's behavior. The software used in meeting rooms can be thought of as a form of shared editor and many of the issues are the same, but the purpose of a shared editor is to collaborate over normal documents. Shared editors may be text based or include graphics. For simplicity, we shall just consider text. Even so, there are a wide range of design options.

Co-authoring systems

Co-authoring is much longer term, taking weeks or months. Whereas shared editing is synchronous, co-authoring is largely asynchronous, with occasional periods of synchronous work. This may involve shared editing, but even if it does this is only one of the activities. Authors may work out some sort of plan together, apportion work between them, and then exchange drafts commenting on one another's work. In fact, this is only one scenario and if there is one consistent result from numerous studies of individual and collaborative writing, it is this: everyone and every group is different.

Shared diaries

We want to find a time for a meeting to discuss the book we are writing: when should it be? Four diaries come out and we search for a mutually acceptable slot. We eventually find a free slot, but decide we had better double check with our desk diaries and the departmental seminar program. This sort of scenario is repeated time and again in offices across the world. The idea of a shared diary or shared calendar is simple. Each person uses a shared electronic diary, similar to that often found on PCs and pocket organizers. When you want to arrange a meeting, the system searches everyone's diaries for one or more free slots.

There are technical problems, such as what to do if no slots are free (often the norm). The system can return a set of slots with the least other arrangements, which can then form a basis for negotiation. Alternatively, the participants can mark their appointments with levels of importance. The system can then assign costs to breaking the appointments, and find slots with least cost. Mind you, someone may regard all their appointments as critical.

Communication through the artefact

In each of the last four systems – shared PCs and windows, shared editors, coauthoring systems and shared diaries – the focus has been upon the artifacts on which the participants are working. They act upon the artifacts and communicate with one another about the artifacts. However, as well as observing their own actions on the artifacts, the participants are aware of one another's actions. This awareness of one another's actions is a form of communication through the artifact. This can happen even where the shared artifact is not 'real' groupware. For instance, shared files and databases can be a locus for cooperation. Sales figures may be entered into the company database by a person in one department and then used as part of a query by another employee. At a loose level, the two are cooperating in jobs, but the database, and the information in it, may be their only means of communication. Such communication is one-way, and is thus a weak form of collaboration, but often important. For example, casework files are a central mechanism for communication and cooperation in many areas from taxation to social work. However, the facilities for cooperation in a typical shared file store or database are limited to locking, and even that may be rudimentary

5.5 FRAMEWORKS FOR GROUPWARE

In this section we will discuss several frameworks for understanding the role of groupware. One use for these is as a classification mechanism, which can help us discuss groupware issues. In addition, they both suggest new application areas and can help structure the design of new systems.

5.1 Time/space matrix and asynchronous working

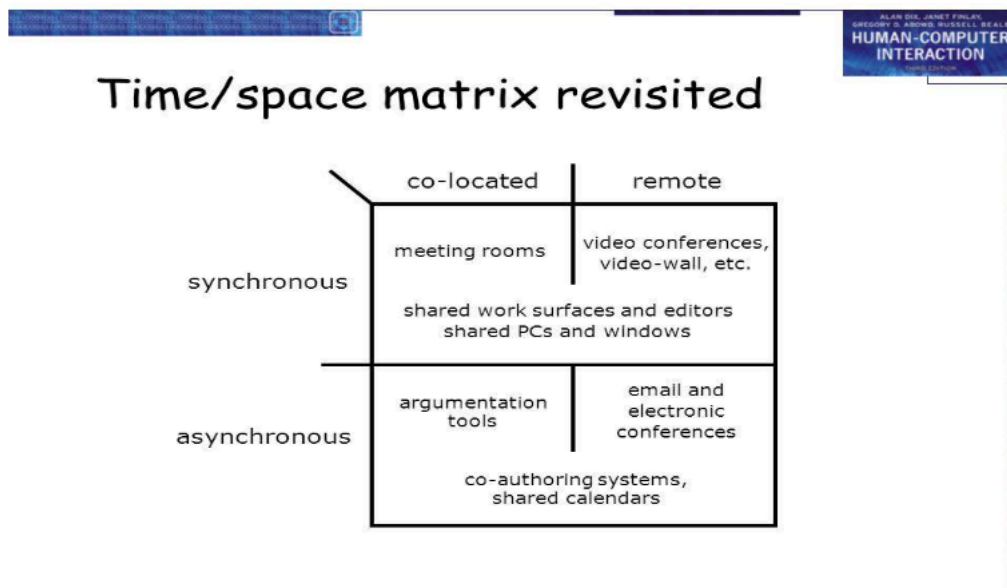


Figure- Groupware in the time/space matrix

This matrix has become a common language amongst the CSCW community. It can also be useful during design as one of the earliest decisions is what sort of interaction you are planning. The design space for synchronous interaction is entirely different from that for asynchronous. However, the synchronous/asynchronous distinction is not as simple as it at first seems. In Section 19.2 we simply said 'whether [the participants] are operating at the same time or not'. However, for an email system, it makes no difference whether or not people are operating at the same time. Indeed, even in Figure 19.1, when we classified letter writing as different time/different place, a similar objection could be brought. The difference between email systems and (most) co-authoring systems is that the latter have a single shared database. Thus when people work together, they know they are working together, and, depending on the locking regime used, can see each other's changes. An email system, on the other hand, may take some time to propagate changes. Perhaps, a better distinction is to look at the data store and classify systems as synchronized when there is a real-time computer connection, or unsynchronized when there is none. For unsynchronized systems it makes little difference whether or not the participants are operating at the same time. Also location is not very significant. (A co-located unsynchronized system is possible; imagine two computers in the same room with no network, which are periodically brought up to date with one another by floppy disk transfer.) If we consider synchronized systems, then the actual time of use becomes more important. If the participants are operating at the same time (concurrent access), we have real-time interaction as seen in meeting rooms (co-located) or video conferences (remote).

5.2 Shared information

Electronic conferences and shared workspaces share information primarily for communication, whereas a document is shared for the purpose of working. Both raise similar issues concerning the degree of sharing required.

Granularity -

Shared information

Granularity of sharing

- **chunk size**
 - small – edit same word or sentence
 - large – section or whole document
- **update frequency**
 - frequent – every character
 - infrequent – upon explicit 'send'

Levels of sharing –

As well as varying in terms of how much is shared, systems vary as to what is shared. For example, two people may be viewing the same part of a Database, but one person sees it presented as a graph, and the other in tabular form.

Types of object –

The kind of object or data we are cooperating over obviously affects the way we share them. This is particularly important in the *unsynchronized* case or where there is a danger of *race conditions*. That is, where two participants perform updates simultaneously and there is confusion as to which comes first.

5.3 Integrating communication and work

Direct communication supported by email, electronic conferences and video connections;

Common understanding supported by argumentation tools, meeting rooms and shared work surfaces;

Control and feedback from shared artifacts supported by shared PCs and windows, shared editors, co-authoring systems and shared diaries.

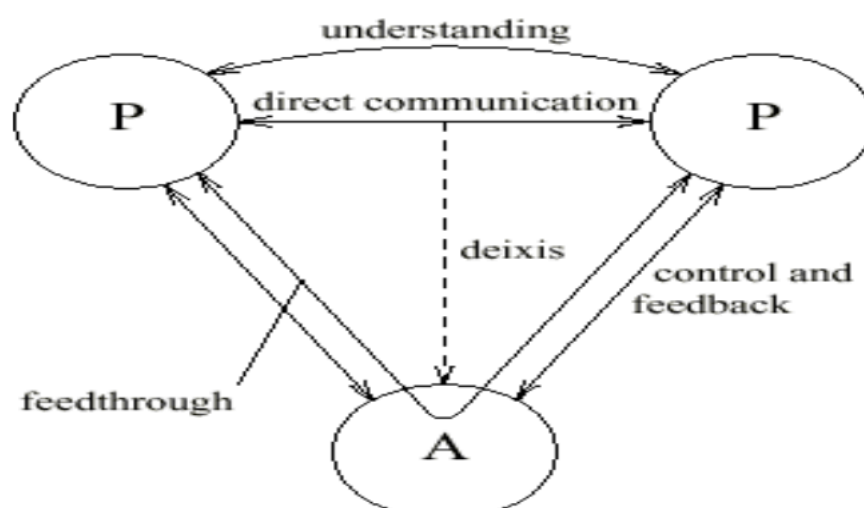


Fig - Cooperative work framework

The first new arc represents *deixis*. The participants needed to refer to items on the shared screen, but could not use their fingers to point. In general, direct communication about a task will refer to the artifacts used as part of that task. The other new arc runs between the

participants, but through the artifact. This reflects the *feedthrough* where one participant's manipulation of shared objects can be observed by the other participants.

communication through the artifact can be as important as direct communication between the participants. Although systems have been classified by the arc which they most directly support, many support several of these aspects of cooperative work. In particular, if the participants are not co-located, many systems will supply some alternative means of direct communication. For example, shared window systems are often used in conjunction with audio or video communications. However, these channels are very obviously separate, compared with, say, the *TeamWorkStation* where the shared work surface video images of hands are overlaid. In particular, this close association of direct communication with the artifacts makes deixis more fluid – the participants can simply point and gesture as normal.

5.4 Awareness

An important issue in groupware and CSCW is awareness – generally having some feeling for what other people are doing or have been doing. Awareness is usually used to refer to systems that demand little conscious effort or attention as opposed to, say, something that allows you to explicitly find out what others are doing.

There are a number of different kinds of awareness. First we may want to know who is there (a) – are they available, at their desk, in the building or busy? For example, instant messaging systems often have some form of buddy list. Your friends are listed there and the messenger window shows whether they are logged into the system, if they appear to be idle (not typed for a while), or whether they have left some sort of status message. Always-on video or audio connections also give a sense of being 'around' your remote colleagues and effortlessly having a feeling for what they are doing, whether they are busy or in a meeting.

You may also want to be aware of what is happening to shared objects (b). For example, you might request that the system informs you when shared objects are updated. Although knowing *what* has changed is important, ideally we would like to know *why* it has changed. Of course we could ask, but if we know how the change happened (c), then we may be in a better position to infer the reasons for the change. Actually, very few systems do this except in so far as they allow users to add annotations to explain changes. These two together (b and c) are called workplace awareness.

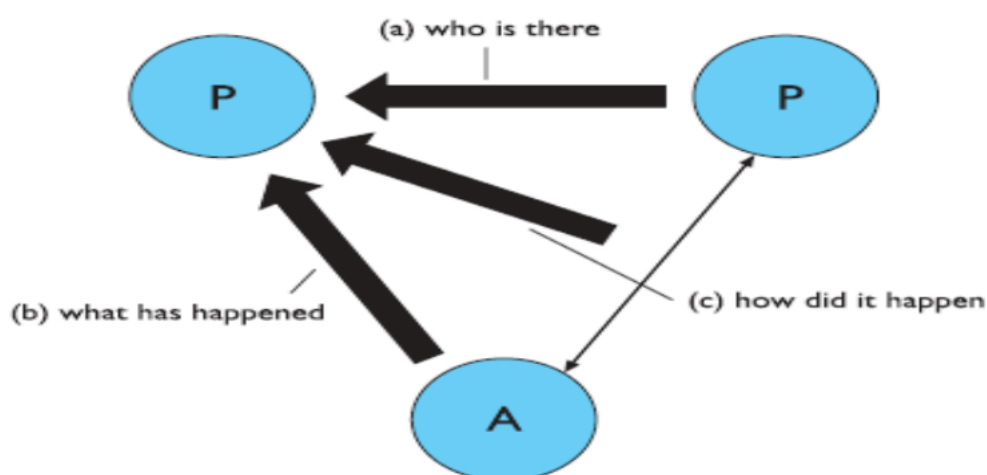


Fig- Forms of awareness

5.5 IMPLEMENTING SYNCHRONOUS GROUPWARE

Groupware systems are intrinsically more complicated than single-user systems. Issues like handling updates from several users whilst not getting internal data structures or the users'

screens in a mess are just plain difficult. These are made more complicated by the limited bandwidth and delays of the networks used to connect the computers, and by the single-user assumptions built into graphics toolkits.

5.5.1 Feedback and network delays

When editing text, a delay of more than a fraction of a second between typing and the appearance of characters is unacceptable. For text entry, a slightly greater delay is acceptable as you are able to type ahead without feedback from the screen. Drawing, on the other hand, demands even faster feedback than text editing. Groupware systems usually involve several computers connected by a network. If the feedback loop includes transmission over the network, it may be hard to achieve acceptable response times. To see why, consider what happens when the user types a character:

1. The user's application gets an event from the window manager.
2. It calls the operating system . . .
3. which sends a message over the network, often through several levels of protocol.
4. The message is received by the operating system at the remote machine,
5. which gives it to the remote application to process.
- 6–8. the reply returns (as steps 2–4)
9. and the feedback is given on the user's screen.

This process requires two network messages and four context switches between operating system and application programs in addition to the normal communication between window manager and application. However, even this is just a minimum time and other factors can make the eventual figure far worse. Network protocols with handshaking can increase the number of network messages to at least four (two messages plus handshakes). If the application is running on a multi-tasking machine, it may need to wait for a time slice or even be swapped out! Furthermore, the network traffic is unlikely to be just between two computers: in meeting rooms we may have dozens of workstations. Clearly, any architectural design for cooperative systems must take the potential for network delays very seriously.

5.5.2 Architectures for groupware

There are two major architectural alternatives for groupware, *centralized* and *replicated*, with variations upon them both. In a centralized or *client-server architecture* each participant's workstation has a minimal program (the client) which handles the screen and accepts the participant's inputs. The real work of the application is performed by the server, which runs on a central computer and holds all the application's data. Client-server architectures are probably the simplest to implement as we have essentially one program, with several front ends. Furthermore, if you use X Windows then there are standard facilities for one program to access several screens

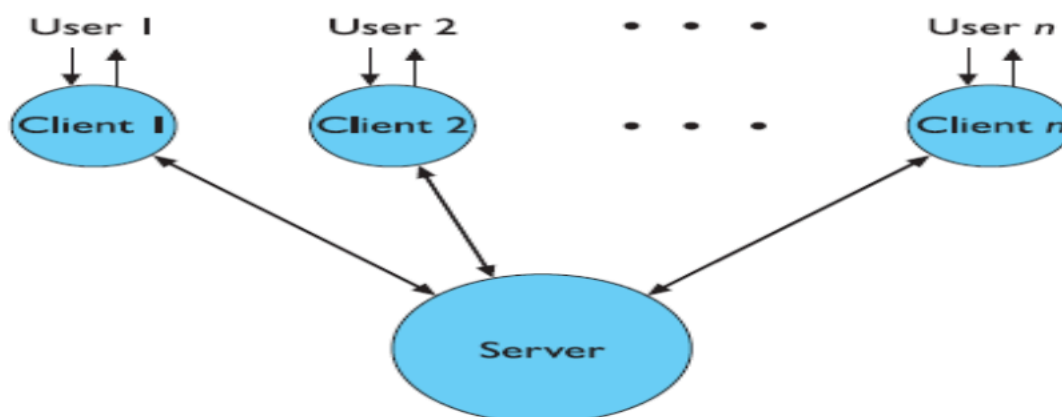


Fig- Client-server architecture

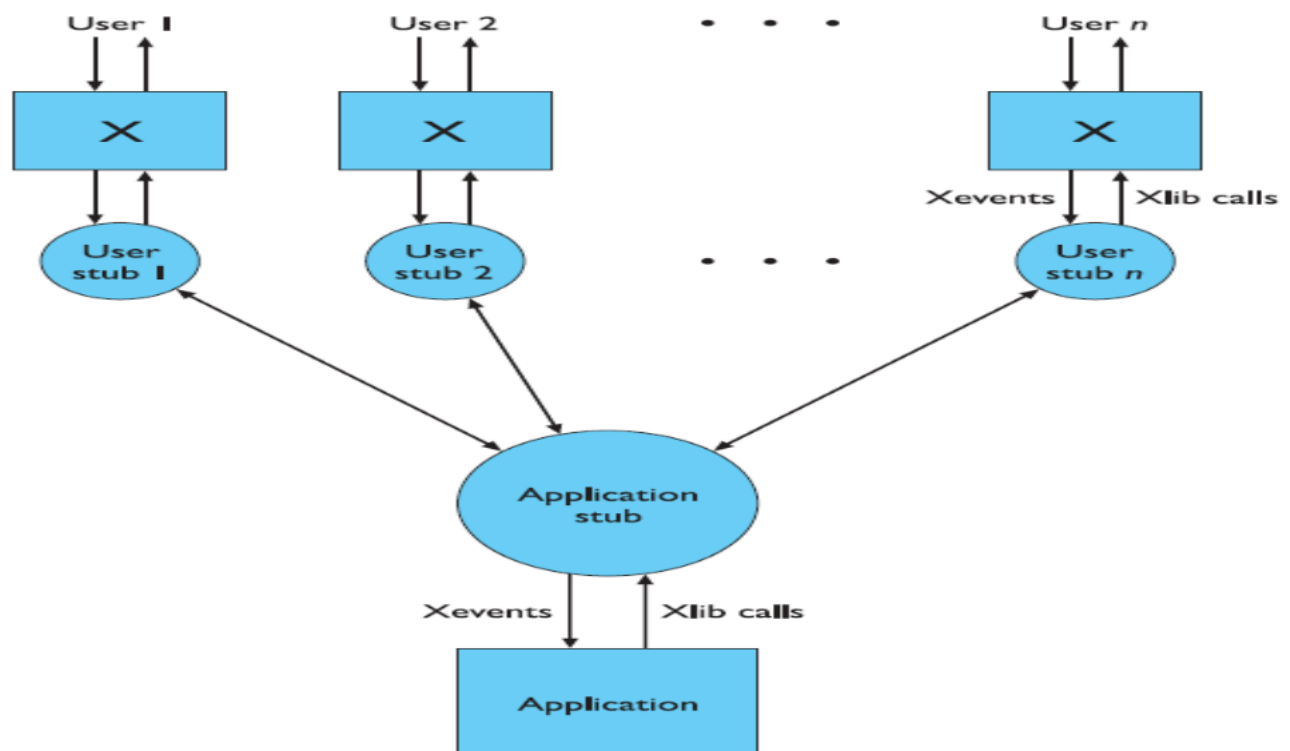
As a special case, the server may run on one of the users' workstations and subsume the client there. Typically, this would be the user who first invoked the shared application. This arrangement is a *master-slave architecture*, the master being the merged server-client and the slaves the remainder of the clients. The user of the master will have a particularly fast response compared with the other users.

The second major architecture is *replicated*. Each user's workstation runs its own copy of the application. These copies communicate with one another and attempt to keep their data structures consistent with one another. Each replicate handles its own user's feedback, and must also update the screen in response to messages from other replicates. The intention is often to give the impression of a centralized application, but to obtain the performance advantages of distribution.

Compared with a client-server architecture, the replicated architecture is difficult to program. The main advantage of a replicated architecture over the client-server is in the local feedback. However, the clients are often not completely dumb and are able to handle a certain amount of feedback themselves. Indeed, the server often becomes merely a central repository for shared data with the clients having most of the application's functionality. On the other hand, a replicated architecture will rarely treat all the replicates identically. If a user tries to load or save a document, that action does not want to be replicated. Either one of the replicates is special, or there is a minimal server handling movement of data in and out of the system. So we see that there is a

Continuum between the client-server and the replicated architecture.

5.5.3 Shared window architectures



Shared window systems have some similarities with general groupware architectures, but also some special features. Recall that there is a single-user application which is being shared by several participants on different workstations. The single-user application normally interacts with the user via a window manager, say X (Figure). The shared window manager works by intercepting the calls between the application and X. Where the application would

normally send graphics calls to X, these are instead routed to a special application stub. This then passes the graphics calls from the X library (Xlib) on to a user stub on each participant's workstation. A copy of X is running on each workstation and the user stub passes the graphics calls to the local copy of X. Similarly the users' keystrokes and other actions cause X events which are passed to the user stub and thence through the application stub to the application. In fact, the nature of X's own client-server approach can make the user stub unnecessary, the application stub talking directly to an X 'server' on each workstation.

The input side has to include some form of floor control, especially for the mouse. This can be handled by the application stub which determines how the users' separate event streams are merged. For example, it can ignore any events other than those of the floor holder, or can simply allow users' keystrokes to intermingle. If key combinations are used to request and relinquish the floor, then the application stub can simply monitor the event streams for the appropriate sequences. Alternatively, the user stub may add its own elements to the interface: a floor request button and an indication of other participants' activities, including the current floor holder. The problem with a client-server-based shared window system is that graphics calls may involve very large data structures and corresponding network delays. One can have replicated versions where a copy of the application sits on each workstation and stubs communicate between one another. But, because the application is not *collaboration aware*, problems such as race conditions and reading and saving files become virtually intractable. For this reason, most shared window systems take a master-slave approach where the application runs on the first user's workstation and subsequent users get slave processes. The delays for other users are most noticeable when starting up the application.

5.5.4 Feedthrough and network traffic

Feedthrough & traffic

- Need to inform all other clients of changes
- Few networks support broadcast messages, so ...
 n participants $\Rightarrow n-1$ network messages!
- Solution: increase granularity
 - reduce frequency of feedback
 - but ...
poor feedthrough $\Rightarrow$ loss of shared context
- Trade-off: timeliness vs. network traffic

5.5.5 Graphical toolkits

Typical graphics toolkit or window manager, such as menus, buttons, dialog boxes and text and graphics regions. These are useful for creating single-user interfaces, and one would like to use the same components to build a groupware system. Unfortunately, the single-user assumptions built into such toolkits can make this very difficult.

Some widgets may take control away from the application. For instance, a pop-up menu may be invoked by a call such as

```
sel = do_pop_up("new", "open", "save", "exit", 0);
```

The call to `do_pop_up_menu` constructs the pop-up menu, waits for the user to enter a selection, and then returns a code indicating which choice the user made (1 for 'new', 2 for

‘open’, etc.). Of course, during this time the application cannot monitor the network. This can be got round, by careful programming, but is awkward.

5.5.6 Robustness and scalability

If you are producing a shared application to test an idea, or for use in an experiment, then you can make a wide range of assumptions, for instance a fixed number of participants. Also, the occasional crash, although annoying, is not disastrous. However, if you expect a system to be used for protracted tests or for commercial production then the standards of engineering must be correspondingly higher. Four potential sources of problems are:

1. Failures in the network, workstations or operating systems.
2. Errors in programming the shared application.
3. Unforeseen sequences of events, such as race conditions.
4. The system does not scale as the number of users or rate of activity increases.

Server faults

The most obviously disastrous problem in a client–server-based system is a server crash, whether hardware or software. Fortunately, this is most amenable to standard solutions. Most large commercial databases have facilities (such as transaction logging) to recover all but the most recent changes. If the groupware system is not built using such a system then similar solutions can be applied; for example, you can periodically save the current state using two or three files in rotation. The *last* entry in each file is the date of writing and the system uses the most recent file when it restarts, ignoring any partially written file. Remember, though, that this might mean your server going ‘silent’ for a few seconds each time it saves the file – can your clients handle this? In really critical situations one can have multiple servers and copies of the data, so that a backup server can take over after a crash of the primary server.

Workstation faults

More often, individual workstations, or the programs running on them, will crash. This is partly because there are more of them, and partly because their code is more complex. In particular, these programs are handling all the user’s interactions and are built upon complex (and frequently flaky) graphical toolkits. Of course, one tries to program carefully and avoid these errors, but experience shows that they will continue to occur. The aim is to confine the fault to the particular user concerned and to recover from the fault as quickly as possible. When thinking about a client–server architecture, there are three ‘R’s for the server:

Robust A client failure should not destroy or ‘hang’ the server. In particular, *never* have the server wait for a response from the client – it may never come. The server should either be *event driven* or *poll* the clients using non-blocking network operations.

Reconfigure The server must detect that the client has failed and reconfigure the rest of the system accordingly. The client’s failure can be detected by standard network failure codes, or by timing out the client if it is silent for too long. Reconfiguring will involve resetting internal data structures, and informing other participants that one of them is unavailable and why. Do not let them think their colleague is just being rude and not replying!

Resynchronize When the workstation/client recovers, the server must send sufficient information for it to catch up. A server may normally broadcast incremental information (new messages, etc.), so make sure that the server keeps track of all the information needed to send to the recovered client. This is very similar to the case of a new participant joining the groupware session.

Algorithm faults

Some application failures do not crash the application and may therefore be more difficult to detect. For instance, data structures between replicates or between client and server may become inconsistent. Obviously this *should not* happen, if the algorithms are correct. Indeed, such critical algorithms should be given the closest scrutiny and perhaps proved correct using formal methods. However, one should be prepared for errors and, where possible, include

3.4 HYPERTEXT, MULTIMEDIA AND THE WORLD WIDE WEB

3.4.1. Understanding Hyper text

- What is the hyper?- it includes rich contents: graphics, audio, video, computation, and interaction
- The traditional texts share a common linear nature.
- This linearity is partly because of the nature of the media used
- Text imposes strict linear progression on reader.

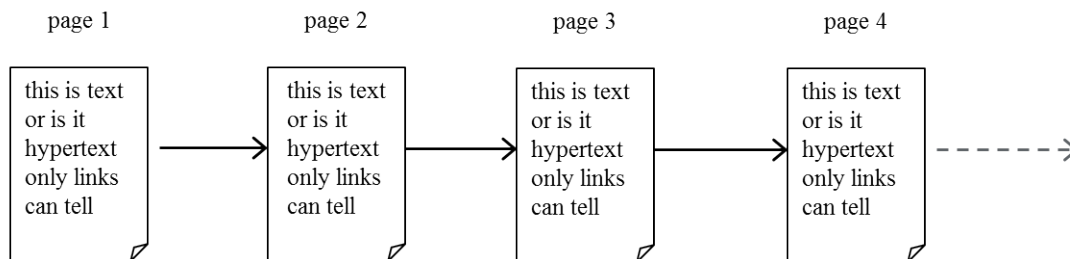


Fig.3.12. Linear Progression Structure of Text

- There are classes of activities where the reader needs to establish their own path through a text. For example, in the online documentation
- The learners may want to follow their own paths through material during some forms of investigative learning.
- Hence, the linear form of a traditional text is a difficult one.
- For example, when using manuals, the user may not understand all the terms used in the text, and will have to keep going back to a different series of pages to look up the definitions, returning to the original pages.
- Hypertext attempts to get around these limitations of text by structuring it into a mesh rather than a line
- This allows a number of different pages to be accessed from the current one
- Hypertext systems incorporate diagrams, photographs and other media as well as text. Such systems are often known as *multimedia* or *hypermedia* systems
- A hypertext system comprises a number of pages and a set of *links* that are used to connect pages together.
- The links can join any page to any other page, and there can be more than one link per page.

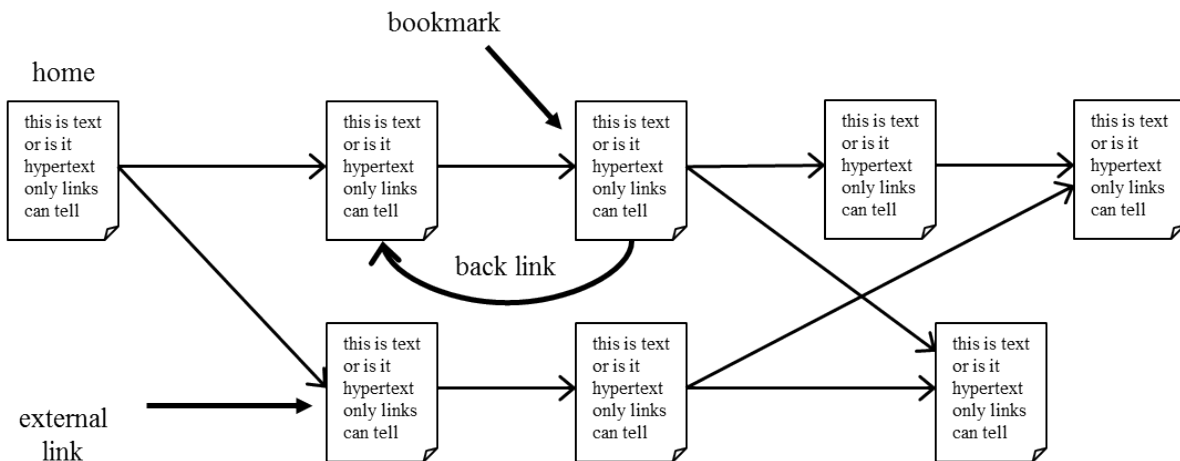


Fig.3.13. Non Linear Progression Structure of Hyper Text

- Hyper media is not just the text which includes hypertext systems and additional media such as illustration, photographs, audio and video
- Links can exist at the end of pages, with the user choosing which one to follow, or can be embedded within the document itself.
- Simply clicking on an unknown word takes the user to the relevant place in the glossary.
- The positions of these links are known as *hot-spots* since they respond to mouse clicks.
- Hot-spots can also be embedded within diagrams, pictures or maps, allowing the user to focus his attention on aspects that interest him.

3.4.1.1. Rich Content

- Hypertext systems may also include more dynamic material such as animation, video and audio clips, and even full computer applications.

Animation

- Adding motion to images
 - For things that change in time
 - Digital faces – seconds tick past or warp into the next
 - Analogue face – hands sweep around the clock face
 - Live displays: e.g. current system load
 - For showing status and progress
 - Flashing carat at text entry location
 - Busy cursors (hour-glass, clock, spinning disc)

- Progress bars
- For education and training
 - Let students see things happen ... as well as being interesting and entertaining images in their own right
- For data visualisation
 - Abrupt and smooth changes in multi-dimensional data visualised using animated, coloured surfaces
 - Complex molecules and their interactions more easily understood when they are rotated and viewed on the screen
- for animated characters
 - Wizards and help

Video and Audio

- Video or audio material as part of hypertext systems for education, entertainment
- Both audio and video material are expensive and time consuming to produce video and audio editing as standard
- Digital video cameras bring the production of audio/video material in to reach of many.
- In addition to that the standard formats such as QuickTime allow this material to be embedded in web pages for easy distribution. But, the production of quality video requires extensive experience
- Conceivably the biggest problem with audio and video is still the memory requirements.
- Longer video sequences are, of course, more linear than plain text. This may be acceptable if the hypertext is acting effectively as an index for video material.
- For example, one might have a collection of silent movies online and access them through a website.
- Hence, the techniques required to gain maximum benefit from moving images are very different from those that are used for static or minimal motion displays, and designers do not have enough experience to start applying the relevant technology at the relevant time.
- It may well be that computer interface designers will have to study the techniques of the film makers and cartoonists before they start to discover the real benefits that these techniques can provide.

Computation, Intelligence, and Interaction

- The good thing about a computer is that not only can it *show* things that have previously been prepared, it can also *do* things.

- However, the web search can look through all the chapters and find any words you want.
- More interactive hypermedia may contain embedded games or applications. For example, puzzle from the website
- Hypermedia running on the user's own computer may interact closely with other applications.
- For example, on a Macintosh HyperCard stacks can control applications using AppleEvents, or on a Windows platform hypermedia can include ActiveX components

Delivery Technology

- On the computer
 - Some hypertexts, in particular help systems, are downloaded or installed permanently on a computer.
 - This has the advantage of instant access and such applications need not use a standard viewer but may include their own bespoke browsing software.
 - CD-ROM or DVD based hypermedia
- On the web
 - It is really ubiquitous!
 - The world wide web offers a rich environment for the presentation of information.
 - Documents can be constructed that are very different from paper versions; basic text can be augmented through the use of hypertext links to other documents, while graphics can easily be incorporated as pictures, photographs, icons, page dividing bars, or backgrounds.
 - Pages can also have hypertext links embedded into different regions, which take the user to a different page or graphic if they are clicked on; these are known as active image maps.
 - These features allow web pages to become interactive, acting as the interface to the information as well as its holder.
 - In many countries, near universal internet access is not just web pages!
 - *e.g.* many applications have web-base documentation
- On the Move
 - Mobile phones, PDAs (personal digital assistants), and notebook computers have all increased the demand to have hypermedia available on the move.
 - Furthermore across many countries governments have sold franchises for high-bandwidth mobile services.

- Notebook computers can use just the same mechanisms as desktop computers, using CD-ROM or DVD for standalone material, or connecting to the web through wireless access points or through modems linked to mobile phone networks.
- However, the fact that the computer is mobile means that location can be used as a key into context-aware hypermedia showing different content depending on location.
- Delivery
 - CD-ROM or DVD (like desktop)
 - Cached content (*e.g.* AvantGo)
 - WiFi access points or mobile phone networks
 - WAP – for mobile phone, tiny web-like pages
- Context – who and where
 - tourist guides, directed advertising

Application Areas

- Rapid prototyping
 - Create live storyboards
 - Mock-up interaction using links
- Help and Documentation
 - Allows hierarchical contents, keyword search or browsing
 - Just in time learning
 - What you want when you want it
 - (*e.g.* technical manual for a photocopier)
 - Technical words linked to their definition in a glossary
 - links between similar photocopiers
- Education
 - Animation and graphics allow students to see things happen
 - Sound adds atmosphere and means diagrams can be looked at while hearing explanation
 - Non-linear structure allows students to explore at their own pace
 - E-learning
 - Letting education out of the classroom!!
 - *e.g.* eClass

3.4.2. *Issues/Finding things in Hyper text*

3.4.2.1 *Lost in Hyperspace*

- Even though the non-linear structure of hypertext is very powerful, it can also be confusing.
- It is easy to lose track of where you are, a problem that has been called ‘lost in hyperspace’.
- There are two elements to this feeling of ‘lostness’.
 - **Cognitive and related to content**
 - The reader can browse the text in any order. Each page or node has to be written virtually independently, but, of course, in reality it cannot be written entirely without any assumption of prior knowledge.
 - Once the reader encounters fragmentary information, it cannot be properly integrated, leading to confusion about the topic.
 - **Navigation and Structure**
 - The hypertext may have a hierarchical or other structure, the user may navigate by hyperlinks that move across this main structure.
 - It is easy to lose track of where you are and where you have been.
- The solution to the former issue is to design the information better.
- The solution to the latter is to give users better ways of understanding where they are and of navigating in the hypertext.

3.4.2.2. *Designing Structure*

- In a paper format one is stuck with a single structure, which can lead to tensions: for example, the fact that in this book structural design is discussed in several places.
- If multiple structures are used, you have to consider what to do about the common material.
- For example, if we examine a car hypermedia text under ‘engine compartment’ and get to the fuel pump, this would also appear in the functional view under ‘fuel system’. Such common elements may be replicated.
- This has the advantage that the material can be presented in ways that make sense given their context, but it can also lead to inconsistencies.
- Alternatively we may make links across the hierarchy at some level; for example, the engine compartment may have a diagram of the engine with a labeled arrow saying ‘fuel pump (fuel system)’, which takes you to the description of the pump in the fuel system part of the hypertext.
- In all cases it is important that the structure and the naming of parts is meaningful for the user

3.4.2.3 Making Navigation Easier

- No matter how well designed the site structure is, there will still be problems: because the user does not understand the structure; or because the user has individual needs that the designer has not foreseen; or because even a good structure is not perfect.
- However, there are various things that can make it easier for users.
- Solution is to provide the following
 - maps
 - give an overview of the structure
 - show current location – you are here!
 - recommended routes
 - guided tour or bus tour metaphor
 - linear path through non-linear structure
 - levels of access
 - summary then progressive depth
 - supporting printing!
 - needs linearised content, links back to source

3.4.2.4 History, Bookmark and External Link

- Hypertext viewers and web browsers usually have some sort of history mechanism to allow you to see where you have been, and a more stack-based system using the ‘back’ button that allows you to backtrack through previously visited pages.
- The back button may be used where a user has followed a hyperlink and then decided it was to the wrong place, or alternatively, when browsing back and forth from a central page that contains lots of links.
- The latter is called *hub and spoke* browsing
- For longer-term revisiting, browsers typically support some form of bookmarking of favorite pages.
- Both this and, on the web, external links from other people’s sites mean that users may enter your hypertext at locations other than the top level or home page.
- On the web this is called *deep linking*.
- Many websites rely on the user remembering where they have come from to make sense of a page.
- If a page does not adequately show where it fits, then a user coming to it from outside may have no idea what site it is from, or why they are reading the material.

3.4.2.5 Indices, Directories and Search

- Index
 - often found in help, documentation, ... even books
 - selective: not an exhaustive list of words used
- Directories
 - on web index would be huge! so hand chosen sites
 - e.g. [open directory project](#), [Yahoo!](#)
- Web Search Engines
 - 'Crawl' the web following links from page to page
 - Build full word index (but ignore common 'stop' words)
 - Looks up in index when you enter keywords to find pages

3.4.3. Web Technology and Issues

- The web consists of a set of protocols built on top of the *internet* that, in theory, allow multimedia documents to be created and read from any connected computer in the world.
- The web supports hypertext, graphics, sound and movies, and, to structure and describe the information, uses a language called *HTML* (hypertext markup language) or in some cases, XML (extensible markup language).
- HTML is a markup language that allows hypertext links, images, sounds and movies to be embedded into text, and it provides some facilities for describing how these components are laid out.
- HTML documents are interpreted by a viewer, known as a *browser*; there are many browsers.
- Some of the commercial browsers such as *Netscape Navigator*, *Microsoft Internet Explorer* and *Opera*.
- These offer a graphical interface to the document, controlled by the mouse.
- Hypertext links are shown by highlighting the text that acts as the link in an alternative color, and are activated by clicking on the link.
- A further color is used to indicate a link that has already been visited.
- Hypertext links can also be embedded into regions within an image.
- Challenges in Web - lost in hyperspace, information overload

3.4.3.1. Web Servers and Clients

- A conventional PC program runs and is displayed on one computer, whereas the web is *distributed*.

- Different parts of it run on different computers, often in different countries of the world.
- They are linked, of course, by the internet, an enormous global computer network
- The pages are stored on web servers that may be on a company's own premises or in special data centers.
- Because they are networked, the webmaster for a site can upload pages to the server from wherever.



Fig.3.14. Working of Web Server and Client

3.4.3.2. Network Issues

- The fact that the web is networked raises a series of issues that can impact on usability.
- Network capacity is called *bandwidth*.
- This is a measure of the amount of information that can pass down the channel in a given time.
- However, bandwidth is not the only important measure.
- There is also the time it takes for a message to get across the network from your machine to the web server and back.
- This delay is called *latency*.
- Latency is caused by several factors – the finite speed of electrical or optical signals (no faster than the speed of light), and delays at routers along the way that take messages from one computer network and pass them on.
- This latency may not always be the same, varying with the exact route through the network traveled by a message, the current load on the different routers, etc. Variability in the latency is called *jitter*.

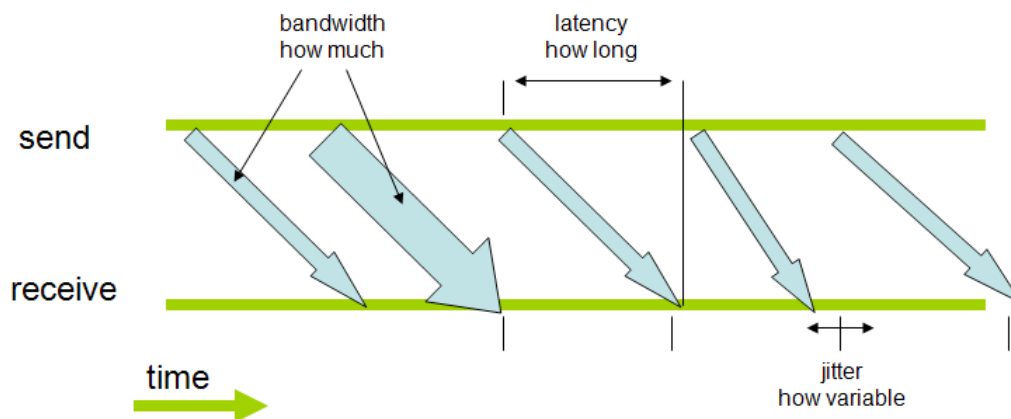


Fig.3.15. Bandwidth, Latency and Jitter

3.4.4. Static Web Contents

Message and Medium

- “content is king”
 - the catch phrase of dot.com era ... but widely ignored
 - the message ... content should be
 - appropriate to the audience, timely, reliable ,
 - generally worth reading !
- the medium ... page and site design
 - good design – essential to attract readers
... but won't hide bad material!
 - bad design – may mean good material never seen
 - printable!

Text

- text style
 - generic styles universal: serif, sans, fixed, bold, *italic*
 - specific fonts too, but vary between platforms
 - cascading style sheets (CSS) for fine control
... but beware older browsers and fixed font sizes
 - colour ... often abused!
- positioning
 - easy .. left, right justified or centred

- precise positioning with DHTML ... but beware platforms ...
- screen size
- mathematics ... needs special fonts, layout, ...

Graphics

- use with care ...
 - N.B. file size and download time
 - ... this image = 1000 words of text
 - affected by size, number of colours, file format
 - backgrounds ... often add little, hard to read text
- speeding it up
 - caching – reuse same graphics
 - progressive formats:
 - image appears in low res and gets clearer
- formats
 - JPEG – for photos
 - higher compression but ‘lossy’
 - get ‘artefacts’
 - GIF for sharp edges
 - lossless compression
 - PNG supported by current web browsers
- and action
 - animated gifs for simple animations
 - image maps for images you can click on

Icons

- on the web just small images
 - for bullets, decoration
 - or to link to other pages
 - lots available!
- design ... just like any interface
 - need to be understood
 - designed as collection to fit ...

- under construction
 - a sign of the inherent incompleteness of the web
 - or just plain lazy ??

Web Colour

- how many colours?
 - PC monitors – millions – 24 bits per pixel
... but the ‘same’ colour may look very different
 - N.B. usually only 72–96 dpi
 - older computers, PDAs, phones ...
 - perhaps only 16 bits or 8 bits per pixel ... 256 colours
 - or even greyscale
- colour palettes
 - choose useful 256 colours
 - different choices, but Netscape ‘web safe’ 216 are common
 - each GIF image has its own palette – use for fast download

Movies and Sound

- **problems**
 - **size and download... like graphics but worse!**
 - **may need special plug-ins**
 - **audio not so bad, some compact formats (MIDI)**
- **streaming video**
 - **play while downloading**
 - **can be used for ‘broadcast’ radio or TV**

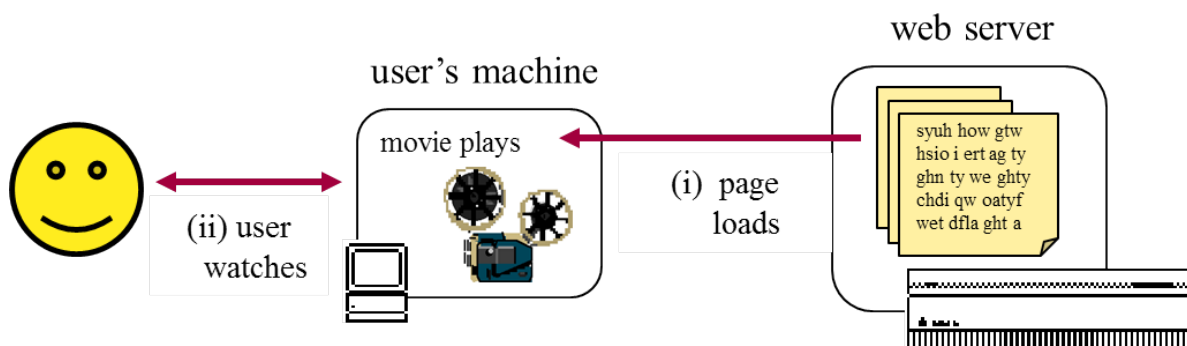


Fig.3.16. Animated GIF or Movie needs to download completely

3.4.5 Dynamic Web Contents

- In the early days, the web was simply a collection of (largely text) pages linked together.
- The material was static or slowly changing and much of it authored and updated by hand.
- As HCI researchers and designers, we can neither ignore nor uncritically accept new technology in the web.
- The active web is here, our job is to understand it and to learn how to use it appropriately

What happens and where?

- The ‘what happens where’ question is the heart of architectural design.
- It has a major impact on the pace of interaction, both *feedback*, how fast users see the effects of their own actions, and *feedthrough*, how fast they see the effects of others’ actions.
- Also, where the computation happens influences where data has to be moved to with corresponding effects on download times and on the security of the data.
- **User View**
 - One set of issues is based on what the end-user sees, the end-user here being the web viewer.
 - **What changes?** This may be a media stream (video, audio or animation) which is changing simply because it is the fundamental nature of the medium. It may be the presentation or view the user has of the underlying content; for example, sorting by different categories or choosing text-only views for blind users
 - **By whom?** Who effects the changes? In the case of a media stream or animation, the changes are largely automatic – made by the computer. The other principal sources of change are the site author and the user. However, in complex sites users may see each other’s changes – feedthrough.
 - **How often?** Finally, what is the pace of change? Months, days, or while you watch?
- **Technology and Security**
 - The fundamental question here is where ‘computation’ is happening. If pages are changing, there must be some form of ‘computation’ of those changes. Where does it happen?
 - **Client** One answer is in the user’s web-browsing client enabled by Java applets, various plug-ins such as Flash, scripting using JavaScript or VBScript with dynamic HTML layers, CSS and DOM (Domain Object Model).
 - **Server** A second possibility is at the web server using CGI scripts (written in

Perl, C, UNIX shell, Java or whatever you like!), Java Servlets, Active Server Pages or one of the other server-specific scripting languages such as PHP. In addition, client-side Java applets are only allowed to connect to networked resources on the same machine as they came from. This means that databases accessed from clientside JDBC (Java database connectivity) must run on the web server (see below).

- **Another machine** Although the pages are *delivered* from the web server, they may be *constructed* elsewhere. For hand-produced pages, this will usually be on the page author's desktop PC. For generated pages, this may be a PC or a central database server.
- **People** Of course, as noted earlier, the process of production and update may even involve people!

3.3.5.1 Fixed Content- Local Interaction and Changing views

- fixed content
- use Java applets, Flash, JavaScript+DHTML
- pros: rapid feedback
- cons: only local, no feedthrough
- after interaction ... what does 'back' do ??

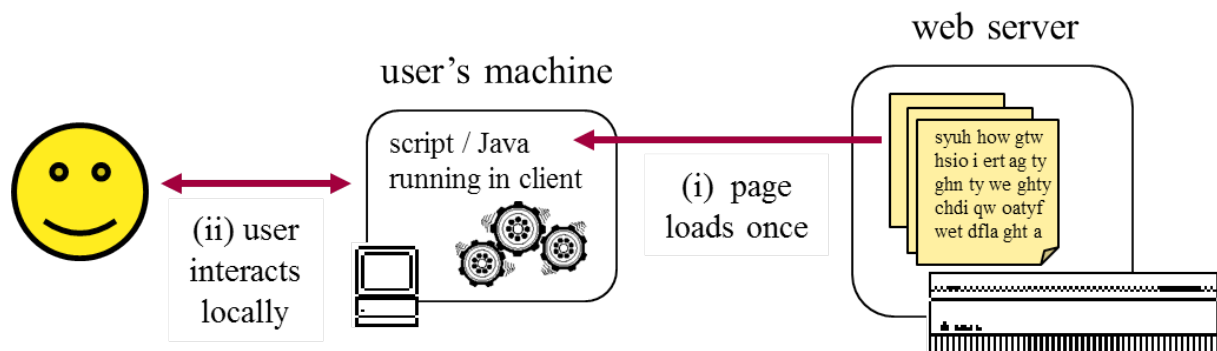


Fig.3.17. Java Applet or Javascript Running locally

REVIWE QUESTIONS WITH ANSWER

PART A – 2 MARKS

1. Define Cognitive Science

Cognitive science- is a scientific and interdisciplinary study of the mind with special emphasis on the use and acquisition of knowledge and information.

2. List out the fields included in Cognitive Science

Artificial Intelligence, Psychology, Linguisitcs, Philosphy, Anthropology, Nueorscience and Education.

3. What is Cognition?

It is derived from latin base cognitio – i.e., “know together” which means the collection of mental processes and activities used in perceiving ,learning, remebering, thinking, and understanding, and the act of those processes.

4. Why Cognitive Science is interdisciplinary?

In order to process the information, human information processor has to use the following steps:

- To acquire real time information about the surrounding environment using **perception** or recpetors.
- With the usage of some **languages**, making the use of information about syntax, semantics and phonology.
- To combine diferent sources of information and derive new information and test its consistency using **reasoning**.
- To make use of information in **action** planning and guidance
- To store and retreive the information using **memeory**
- Hence, cognitive science is inter- discipilinary one.

5. Define Cognitive Model

A theory that produces a computational model of how people perform tasks and solve problems by using psychological prinicples and emprical studies.